

中国图书分类号 TP391;TP311

国际图书分类号 004

密级 公开

单位代码 10613

西南交通大学

硕士学位论文

面向政务领域的大模型数据查询与分析方 法研究与实现

学科专业 计算机科学与技术

学号 2023*****

作者姓名 ###

指导教师 ***

学院 计算机与人工智能学院

2026 年 3 月 20 日

A Thesis Submitted to
Southwest Jiaotong University for Master Degree

**Research and Implementation of Data Query and
Analysis Methods Based on Large Language Models
for the Government Affairs Domain**

Discipline Computer Science and
Technology

Student ID 2023*****

Author ###

Supervisor ***

School School of Computing and
Artificial Intelligence

March. 20, 2026

西南交通大学

学位论文独创性声明

本人郑重声明：所呈交的学位论文是本人在导师指导下进行的研究工作及取得的研究成果。据我所知，除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得西南交通大学或其它教育机构的学位或证书而使用过的材料。其他人员对本研究所做的任何贡献均已在论文中作了明确的说明并表示谢意。

本学位论文的主要创新点如下：

1. 提出了基于逻辑增强与异构协同的政务数据查询生成方法。设计逻辑增强模式 LA-Schema，将业务术语映射、逻辑计算公式和值域约束显式注入数据库模式表示，解决了政务场景中术语歧义和隐含统计口径导致的查询语义偏差问题；构建逻辑映射生成器与渐进分治生成器组成的异构协同候选生成机制，结合执行反馈修复与候选判别模型，提升了复杂政务查询的准确性与稳定性。

2. 提出了基于分析规划与查询增强的政务数据分析方法。引入分析规划模式 AP-Schema，将开放式分析需求转化为涵盖分析类型、指标、维度、操作分层与可视化规格的结构化中间表示；设计查询增强的任务分解机制，将涉及统计口径和复杂关联的操作下沉至查询层完成，在可靠取数基础上通过轻量分析算子链与可视化规则生成规范化分析结果，实现了查询能力与分析能力的结构化协同。

3. 设计并实现了面向政务场景的智能问数系统。将上述查询方法与分析方法整合为可交互可追溯的统一服务，验证了所提方法在实际政务场景中的可落地性。

作者签名：

日期：2026 年 3 月 25 日

西南交通大学

学位论文使用授权

本学位论文作者完全了解西南交通大学有关保留、使用学位论文的规定，同意学校有权保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅。本人授权西南交通大学可以将学位论文的全部或部分内容编入有关数据库进行检索及下载，可以采用影印、缩印、扫描等复制手段保存、汇编本学位论文。

本学位论文属于

1. 保密□，在 年解密后适用本授权书
2. 不保密□，使用本授权书。

(请在以上方框内打“√”)

作者签名：_____ 导师签名：_____

日期： 年 月 日

摘 要

面向数字政府建设背景下持续积累的结构化政务数据，本文围绕非技术人员在政务场景中普遍面临的“查数难、分析难、协同难”问题，研究大语言模型驱动的数据查询与分析方法。针对政务术语与物理字段映射复杂、统计口径与计算逻辑隐含、查询与分析职责边界不清等特点，本文以“可靠查询、规范分析、系统落地”为总体目标，构建面向政务领域的智能问数与数据分析技术链路。

围绕上述目标，本文提出了基于逻辑增强与异构协同的政务数据查询生成方法，通过构建领域知识库、动态样例库与逻辑增强模式 LA-Schema，将业务术语映射、逻辑计算公式和值域约束显式注入查询生成过程，并结合逻辑映射生成器、渐进分治生成器、执行反馈修复与候选判别模型，实现面向复杂政务业务语义的可靠查询生成。进一步地，本文提出了基于分析规划与查询增强的政务数据分析方法，引入分析规划模式 AP-Schema，将开放式分析需求转化为结构化规划，通过查询增强的任务分解机制复用可靠取数能力，并结合轻量分析算子链与可视化规则生成规范化分析结果。在此基础上，本文设计并实现了一个面向政务场景的智能问数系统，将查询方法与分析方法组织为可交互、可追溯的统一服务。

实验结果表明，本文方法在自建政务数据集 GovSQL 上取得了 91.47% 的执行准确率，较最强基线 XiYan-SQL 提升 7.64 个百分点；在政务分析任务上，流程成功率、任务完成率和结果正确率分别达到 92.9%、85.7% 和 95.8%，显著优于代码生成和工具迭代等主流分析范式，能够稳定生成图表、表格与文字洞察；所实现系统支持自然语言问数、分析结果生成与过程追溯，并在 BIRD 与 Spider 公共数据集上保持较强竞争力。

研究表明，本文形成了面向政务领域从自然语言需求理解、可靠取数到规范分析与系统实现的完整技术链路，能够为非技术人员开展政务问数与分析提供方法基础与系统支撑，也为大语言模型在垂直领域数据智能应用中的研究与落地提供了参考。

关键词：政务数据，大语言模型，Text-to-SQL，智能问数，分析规划，数据分析

ABSTRACT

In the context of digital government construction, vast amounts of structured government affairs data have been continuously accumulated. This thesis investigates LLM-driven data query and analysis methods to address the pervasive challenges of data accessibility, analytical complexity, and collaborative silos faced by non-technical personnel in government scenarios. Considering the complex mapping between government terminology and physical database fields, the implicit statistical calibers and computational logic, and the unclear boundary between query and analysis responsibilities, this thesis aims to build an intelligent data querying and analysis technology pipeline for the government affairs domain, guided by the overarching goal of “reliable query, standardized analysis, and systematic implementation”.

Toward this goal, this thesis proposes a government affairs data query generation method based on logic enhancement and heterogeneous collaboration. By constructing a domain knowledge base, a dynamic example library, and a Logic-Augmented Schema (LA-Schema), business terminology mappings, logical computation formulas, and value domain constraints are explicitly injected into the query generation process. Combined with a logical mapping generator, a progressive divide-and-conquer generator, execution feedback-driven query repair, and a fine-tuned candidate discrimination model, the method achieves reliable query generation for complex government business semantics. Furthermore, this thesis proposes a government affairs data analysis method based on analysis planning and query augmentation. An Analysis Plan Schema (AP-Schema) is introduced to transform open-ended analysis requirements into structured plans. Through a query-augmented task decomposition mechanism, the reliable data retrieval capability is reused, and standardized analysis results are generated via lightweight analysis operator chains and visualization rules. Built upon these methods, this thesis designs and implements an intelligent data querying system for government scenarios, organizing the query and analysis methods into a unified, interactive, and traceable service.

Experimental results demonstrate that the proposed method achieves an execution accuracy of 91.47% on the self-constructed government affairs dataset GovSQL, outperforming the strongest baseline XiYan-SQL by 7.64 percentage points. On government analysis tasks, the pipeline success rate, task completion rate, and result accuracy

reach 92.9%, 85.7%, and 95.8%, respectively, significantly outperforming mainstream analysis paradigms such as code generation and tool-iterative agents, with stable generation of charts, tables, and textual insights. The implemented system supports natural language data querying, analysis result generation, and process traceability, while maintaining strong competitiveness on the BIRD and Spider public datasets.

This research establishes a complete technology pipeline for the government affairs domain, spanning from natural language requirement understanding, reliable data retrieval, to standardized analysis and system implementation. It provides a methodological foundation and system support for non-technical personnel to conduct government data querying and analysis, and also offers a reference for the research and deployment of large language models in vertical domain data intelligence applications.

Keywords: Government Affairs Data, Large Language Model, Text-to-SQL, Intelligent Data Querying, Analysis Planning, Data Analysis

目 录

第 1 章 绪论	1
1.1 研究背景与意义	1
1.1.1 研究背景	1
1.1.2 研究意义	2
1.2 国内外研究现状	3
1.2.1 基于大模型的 Text-to-SQL 研究现状	3
1.2.2 基于大模型的代码生成与自动数据分析研究现状	4
1.2.3 政务数据管理与应用平台研究现状	5
1.2.4 现有研究评述	6
1.3 论文研究内容	7
1.4 论文章节安排	8
1.5 本章小结	10
第 2 章 相关理论与技术	11
2.1 大语言模型与工具增强基础	11
2.1.1 Transformer 与指令对齐	11
2.1.2 上下文学习、链式推理与结构化输出	12
2.1.3 工具调用与代码执行	12
2.2 面向可靠查询生成的关键技术	13
2.2.1 任务定义与核心挑战	13
2.2.2 Schema/值检索与领域知识增强	13
2.2.3 候选生成、执行反馈与结果选择	14
2.3 面向数据分析的规划与结果生成技术	14
2.3.1 分析任务链条与中间规划	15
2.3.2 代码生成、分析算子与可视化组织	15
2.3.3 查询与分析分层协同	16
2.4 面向系统实现的工作流编排与服务协同	16
2.4.1 工作流图编排、状态管理与异常回传	16
2.4.2 轻量级 Agent 协同与角色分工	17
2.5 本章小结	17
第 3 章 基于逻辑增强与异构协同的政务数据查询生成方法	18

3.1	引言	18
3.2	总体框架设计	19
3.3	离线知识准备	20
3.3.1	向量库构建	20
3.3.2	政务领域知识库构建	21
3.3.3	动态样例库构建	22
3.4	基于逻辑增强的模式链接	24
3.5	异构候选生成	27
3.5.1	逻辑映射生成器	27
3.5.2	渐进分治生成器	28
3.6	基于执行反馈的查询修复	29
3.7	候选判别模型	30
3.7.1	训练样本构建	31
3.7.2	判别模型训练	32
3.7.3	配对判别与最优候选选择	33
3.8	实验与结果分析	34
3.8.1	数据集构建与实验设置	34
3.8.2	评估指标与对比基线	36
3.8.3	总体性能对比	37
3.8.4	消融实验	38
3.8.5	异构协同与候选判别分析	39
3.8.6	效率分析	41
3.8.7	公共数据集实验结果	42
3.9	本章小结	43
第 4 章	基于分析规划与查询增强的政务数据分析方法	44
4.1	引言	44
4.2	总体框架设计	45
4.3	分析规划构建	47
4.3.1	规划模式定义	47
4.3.2	约束与完备性	48
4.3.3	规划生成	49
4.4	基于查询增强的分析任务分解方法	51
4.4.1	查询层与分析层的任务划分	51

4.4.2 任务分解准则与层级判定	52
4.4.3 查询结果表示与层间衔接	53
4.5 基于分析算子的结果生成方法	54
4.5.1 分析算子设计与编排	54
4.5.2 结果生成流程与输出组织	55
4.6 实验与结果分析	57
4.6.1 实验设置与评价指标	57
4.6.2 结果对比	59
4.6.3 分类型性能对比	60
4.6.4 模块消融实验	61
4.6.5 AP-Schema 规划质量评估	62
4.7 本章小结	63
第 5 章 政务智能问数系统设计与实现	64
5.1 引言	64
5.2 系统总体架构	64
5.3 应用层与智能服务层实现	65
5.3.1 应用层接口设计	65
5.3.2 任务状态管理与异常回传	67
5.4 系统数据层存储	68
5.4.1 业务数据源接入与元数据抽取	68
5.4.2 系统元数据库实现	69
5.4.3 结果、日志与缓存管理	70
5.5 系统功能实现与效果展示	71
5.6 本章小结	74
结论与展望	75
6.1 主要研究结论	75
6.2 研究不足	76
6.3 未来展望	76
参考文献	77

第 1 章 绪论

1.1 研究背景与意义

1.1.1 研究背景

数字政府建设正在持续推动政务数据资源沉淀，并使数据逐步成为政府治理的重要生产要素。2022 年，国务院印发《关于加强数字政府建设的指导意见》，明确提出以数据资源开发利用贯通政府管理、公共服务和科学决策各环节，充分释放数据要素价值^[1]。2023 年，中共中央、国务院印发《数字中国建设整体布局规划》，进一步将数字政府和数据资源体系建设上升为国家战略部署^[2]。2025 年，国家发展改革委、国家数据局、工业和信息化部联合印发《国家数据基础设施建设指引》，从数据流通利用、算力底座、网络支撑和安全防护等方面提出总体建设方向，强调以基础设施能力支撑数据高效流通与应用开发^[3]。在上述政策持续推动下，各级政府部门不断推进数据归集、治理与共享交换，财政预算、公共资源、社会治理、经济运行等场景积累了大量结构化政务数据，政务数据已经成为支撑政府业务协同、治理监测和科学决策的重要基础^[4-5]。

然而政务数据规模的快速增长并未自然转化为高效的数据使用能力。当前政务数据应用仍主要依赖两种模式：一是由技术人员编写 SQL 语句完成数据库查询，再由业务人员进行人工解读和汇总；二是通过预置报表和 BI 看板查看固定口径的统计结果。前者对使用者的数据库知识、统计分析能力和业务理解能力要求较高，难以支撑非技术岗位开展自主问数；后者虽然降低了操作门槛，但通常只能提供预定义维度下的静态展示，难以满足临时性、多维度和深层次的数据探索需求^[4-5]。与此同时，政务任务普遍具有跨部门、跨层级和跨区域协同特征，围绕协同治理形成的数据又呈现出显著的时空性、层级性和业务约束性。已有研究表明，面向政府大规模协同任务的数据管理与统计分析本身就具有较高复杂度，传统离线处理和割裂式系统难以及时响应复杂场景下的动态分析需求^[6]。这使得政务数据应用长期面临“数据可得但难用、报表可看但难析、需求可提但响应滞后”的现实困境。

近年来以 GPT-4^[7]、Qwen^[8]和DeepSeek^[9]为代表的大语言模型在自然语言理解、代码生成和复杂任务处理等方面表现出更强能力^[10]，推动了自然语言问数、自动代码生成和自动分析等方向的发展。在数据查询层面，Text-to-SQL 技术已经能够将自然语言问题转换为结构化查询语句^[11]；在数据分析层面，围绕代码执行、分析代理和自动可视化的研究不断出现，显示出大语言模型在生成表格、图表和

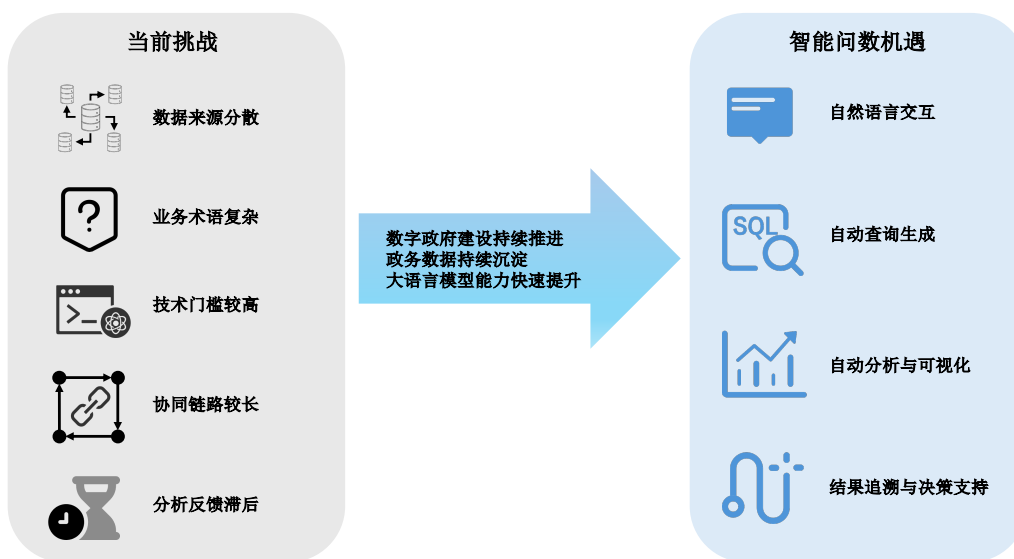


图 1-1 政务数据应用面临的挑战与智能问数带来的机遇示意图

分析结论方面的应用潜力^[12-13]。从国内应用看，大语言模型已逐步进入交通、民生服务和政务服务等垂直领域，基于自然语言交互的智能问数模式正从概念验证走向场景化探索^[14-16]。但政务场景中的数据语义、统计口径和结果规范性要求显著高于通用商业场景，直接迁移通用方法往往仍面临领域知识融入不足、分析流程缺乏结构化约束、查询与分析环节协同不畅等问题。

基于上述场景、瓶颈和技术现状，本文聚焦于政务领域智能问数与数据分析这一研究主题，关注的问题不是单一的“自然语言转 SQL”，而是如何围绕政务业务语义构建一条从自然语言需求理解、可靠查询生成、规范分析执行到系统交互落地的完整技术链路。概括而言，本文的研究目标是面向政务场景，构建“可靠查询 + 规范分析 + 系统落地”的一体化智能问数方法体系，使非技术人员也能够基于自然语言完成问数、查数和分析数。围绕这一目标，本文需要依次回答三个关键问题：第一，如何提升面向复杂政务业务逻辑的 Text-to-SQL 生成准确性与稳定性；第二，如何将开放式分析需求转化为结构化分析规划，并在可靠取数基础上完成规范化分析生成；第三，如何将上述方法组织为可交互、可追溯、可落地的政务智能问数系统。

1.1.2 研究意义

本文的研究兼具理论意义与实践意义。

理论意义。（1）推动领域知识增强的 Text-to-SQL 方法研究。现有 Text-to-SQL 方法多在 Spider^[17]、BIRD^[18]等通用数据集上验证，虽然在通用场景中取得显著进展，但对于政务场景中普遍存在的业务术语歧义、字段缩写复杂和隐含计算逻辑

等问题仍支持不足^[19]。本文通过逻辑增强模式和政务领域知识库的结合,探索将领域先验知识显式注入模式表示与生成流程的实现路径,为垂直领域 Text-to-SQL 方法研究提供参考。

(2) 探索查询生成与数据分析的结构化协同方法。现有研究多围绕查询生成或分析生成分别展开,对分析需求、查询约束和结果组织之间关系的统一描述仍显不足。本文通过构建分析规划模式 AP-Schema,将自然语言分析需求映射为结构化规划,并通过查询增强机制建立查询层与分析层之间的协同关系,为“先查准、再分析”的层级协同提供方法基础。

(3) 丰富面向政务场景的智能数据分析研究。政务数据分析不仅要求结果正确,还要求分析口径清晰、结果展示规范、结论表达可追溯。本文围绕趋势分析、对比分析、排名分析和结构分析四类典型政务分析任务设计分析算子链和结果组织方式,有助于拓展大语言模型与自动数据分析技术在政务场景中的研究边界^[12,20]。

实践意义。(1) 降低政务数据使用门槛。通过自然语言交互替代传统的 SQL 编写和复杂报表配置,可使行政管理、政策研究和业务监督等岗位的非技术人员直接表达数据需求,缩短“提出需求,等待开发,返回结果”的协作链路,提高政务数据使用的实时性与自主性。

(2) 提升治理决策中的数据支撑深度。相较于仅返回原始查询结果,本文研究的智能问数方法进一步覆盖数据分析、图表生成和洞察摘要等过程,可将政务数据应用从“取到数”延展到“看懂数、用好数”,从而为经济运行监测、财政绩效评估、公共资源配置和协同治理等场景提供更深入的数据支撑。

1.2 国内外研究现状

围绕本文研究主题,现有相关工作主要可归纳为三个方向:基于大模型的 Text-to-SQL 研究、基于大模型的代码生成与自动数据分析研究,以及政务数据管理与应用平台研究。三类研究分别对应“如何查到正确数据”“如何围绕数据形成规范分析结果”“如何在政务场景中组织和落地数据应用”三个层面。因而,本节不仅梳理已有工作的进展与优势,也进一步识别其在政务场景中的能力边界,以说明本文研究的必要性与切入点。

1.2.1 基于大模型的 Text-to-SQL 研究现状

Text-to-SQL 旨在将自然语言问题自动转换为可在关系数据库上执行的结构化查询语句,是自然语言数据库接口的重要研究方向^[11]。从发展脉络看,早期方法更多依赖人工规则、模板和领域词典;随后,Seq2SQL 和 SyntaxSQLNet 等工作推动

研究转向数据驱动的语义映射^[21-22]。这一阶段提升了自然语言查询的泛化能力，但在复杂嵌套、模式链接和跨库迁移方面仍存在明显不足。

在预训练模型阶段，研究重点进一步转向问题语义表示与数据库模式结构建模的融合。RAT-SQL、BRIDGE、PICARD 和 RESDSQL 分别从关系建模、内容锚定、语法约束和 Schema 筛选等角度提升了查询生成可靠性^[23-26]。这些工作表明，仅依赖端到端生成难以稳定处理复杂数据库问题，结构感知与约束机制是提高可用性的关键。

进入大语言模型阶段后，研究重点进一步转向提示组织、样例检索、任务分解、多路径生成与候选判别等策略。DAIL-SQL 和 DIN-SQL 说明高质量示例组织与任务分解有助于提升复杂查询生成效果^[27-28]；CHASE-SQL、XiYan-SQL、OpenSearch-SQL、Alpha-SQL、OmniSQL、CHESS 和 CodeS 等工作则分别从多路径候选构造、动态检索、搜索式生成、数据增强和专用模型构建等角度提升复杂查询的准确率与稳定性^[29-35]。国内学位论文研究也开始关注基于大语言模型的 Text-to-SQL 方法及其在政务数据赋能场景中的应用^[36-38]。

总体来看，现有 Text-to-SQL 研究已经显著提升了通用场景中的自然语言问库能力，但面向政务场景仍存在三方面不足。第一，政务数据库中字段名常以缩写、代码或历史系统命名方式存在，业务术语与物理字段之间存在多对多映射关系，通用 Schema 链接方法难以准确建立语义对齐。第二，预算执行率、同比增长率、结构占比等核心政务指标往往依赖隐含统计口径和计算逻辑，而这些逻辑通常不在原始 Schema 中显式表达，单纯依赖模型隐式推理容易产生“语法正确、业务错误”的结果。第三，政务问数既包含大量高频标准需求，也包含低频但复杂的长尾需求，单一生成策略较难同时兼顾生成效率与复杂查询准确性。上述不足构成了本文第三章需要重点解决的问题。

1.2.2 基于大模型的代码生成与自动数据分析研究现状

相较于 Text-to-SQL 主要解决“如何获得正确数据”，自动数据分析更关注“如何围绕数据组织分析过程并形成可理解结果”。近年随着代码生成模型和分析代理框架的出现，大语言模型开始被用于数据处理、结果解释与可视化生成等任务。

在代码生成方面，Codex^[39]验证了在大规模代码语料上训练的语言模型可以根据自然语言描述生成高质量程序；此后，以 Code Llama、StarCoder 和 WizardCoder 为代表的开源代码模型持续提升了代码补全、脚本生成和复杂编程任务求解能力^[40-42]。上述能力使大语言模型能够生成 Python、SQL 及数据处理脚本，为自动分析场景中的数据提取、统计计算和结果可视化奠定基础。

在自动数据分析框架方面, Data Interpreter^[12]通过规划与代码执行引擎结合实现端到端数据分析; InsightPilot^[20]通过分析操作序列辅助用户逐步发现数据洞察; LIDA^[43]和Chat2VIS^[44]分别从自然语言到可视化生成和对话式图表生成角度推进了自动可视化研究; TaskWeaver^[45]采用代码优先的 Agent 架构支持复杂分析任务执行; Data-Copilot^[46]、TableGPT2^[47]和ChartGPT^[48]则分别在自动化数据交互、表格理解和图表生成方面展示了大模型支持的数据分析潜力。

上述工作展示了大语言模型用于数据分析的可行性,但从本文面向的政务智能问数任务看,仍存在三方面不足。第一,很多方法直接从自然语言需求生成代码、图表或结论,中间规划表示不够显式,不利于校验分析对象、指标和约束是否一致。第二,查询生成与分析计算往往被放在同一流程中,数据库理解、SQL 生成和结果解释之间缺少清晰分工,难以稳定复用可靠取数能力。第三,现有系统多面向通用表格分析、商业分析或开放式数据科学任务,对政务场景中常见的趋势研判、横向比较、结构刻画和排名识别等规范化分析范式支持仍然有限。由此可见,如何引入结构化分析规划、明确查询与分析分层并适配政务任务,是本文第四章的主要切入点。

1.2.3 政务数据管理与应用平台研究现状

政务数据管理与应用平台建设经历了从信息资源整合、共享交换,到数据中台、BI 平台,再到引入大模型能力的智能化探索过程^[4-5]。在基础设施层面,各级政府已经构建起较为成熟的数据共享交换平台、主题数据库和业务协同系统,政务数据汇聚、治理和跨部门流通能力持续增强。相关研究也开始从平台型政府和政务数据治理等角度讨论如何通过数字基础设施提升政府的数据能力、业务能力和协同能力^[49]。

在应用层面,数据中台、专题数据库、商业智能系统和可视化看板仍是当前政务数据应用的主流形态。此类平台在统一数据口径、指标计算复用和固定场景展示方面具有明显优势,能够较好支撑例行监测、专题报送和领导驾驶舱等业务需求。然而,这类平台通常需要在建设阶段预设指标、维度、统计口径和图表形式,用户只能在既定框架内进行查看和筛选,难以针对临时性、探索性和跨主题问题开展自主问数与分析^[5,49]。

面向更复杂的政务协同场景,已有工作进一步表明,政务数据应用不仅是数据汇聚和看板展示问题,还涉及复杂任务组织、多层级协同和安全控制等工程挑战。王璐璐等围绕政府大规模协同任务提出时空数据管理与分析研究思路,强调政务协同任务下数据统计与分析的复杂性^[6]。赵大燕等则从安全治理角度提出面

向政务协同的访问控制模型，说明政务协同系统在灵活接入之外还必须兼顾权限边界与数据安全^[50]。这些研究为理解政务场景的数据复杂性和系统复杂性提供了重要参考。

在大语言模型应用方面，国内近年开始探索政务大模型与垂直智能体在政务服务、知识问答、民生热线和垂类应用中的落地路径^[14,16,51-53]。总体上看，现有探索更多聚焦于政策咨询、文本理解、智能问答或单点应用增强，对“自然语言问数、可靠取数、规范分析、结果追溯”这一完整链路关注仍然不足。换言之，政务平台建设已在数据治理和固定展示层面形成一定基础，但从“静态展示”向“智能理解与分析”的演进仍存在明显空间。

1.2.4 现有研究评述

综合上述研究可见，现有工作分别在自然语言查询、自动分析执行和政务平台建设三个方面奠定了重要基础，但仍未形成面向政务智能问数的完整方法闭环。表1-1对三个研究方向的核心特征进行对比归纳。

表 1-1 相关研究方向对比

研究方向	代表方法	主要优势	面向政务场景的主要局限
基于大模型的 Text-to-SQL	DAIL-SQL、CHASE-SQL、XiYan-SQL 等 ^[27,29-30]	Schema 理解与跨库迁移能力强	领域知识注入不足，难处理术语映射、隐含口径与长尾查询
自动数据分析	Data Interpreter、LIDA、TaskWeaver 等 ^[12,43,45]	代码驱动的数据处理与可视化生成	缺少结构化中间规划，查询与分析职责混杂
政务数据管理与应用平台	数据共享交换平台、数据中台、BI 看板等	数据汇聚治理与固定场景展示成熟	灵活分析能力不足，缺少自然语言交互与一体化落地

基于上述分析，本文归纳出三个关键研究缺口：第一，政务 Text-to-SQL 缺少领域知识显式增强机制，难以稳定实现可靠取数；第二，自动数据分析缺少查询增强与结构化规划机制，查询生成与分析执行混合处理，缺少可验证的中间规划表示；第三，现有政务数据平台缺少从方法到系统的一体化智能问数落地能力。换言之，只有同时解决查询正确性、分析规范性和系统协同性，政务智能问数才能真正从“可演示”走向“可应用”。

上述三个研究缺口中，前两个属于方法层面的核心问题，分别对应本文的两项方法创新：第三章研究面向复杂政务业务逻辑的可靠查询生成方法，第四章研究

面向政务洞察任务的结构化数据分析方法；第三个属于系统整合层面的问题，对应第五章的政务智能问数系统设计与实现，旨在将方法创新转化为可落地的系统能力。由此，本文形成了“方法创新驱动、系统落地验证”的完整研究链路。

1.3 论文研究内容

围绕上述研究缺口，本文以“可靠查询生成、规则分析生成、系统落地实现”为主线，开展面向政务领域的大语言模型数据查询与分析方法研究。总体思路不是让大语言模型端到端自由生成全部结果，而是按照“先查准、再分析、后落地”的路径逐层求解：先保证查询结果可靠，再保证分析过程规范，最后将方法封装为可交互系统。本文整体研究框架与技术链路如图1-2所示。

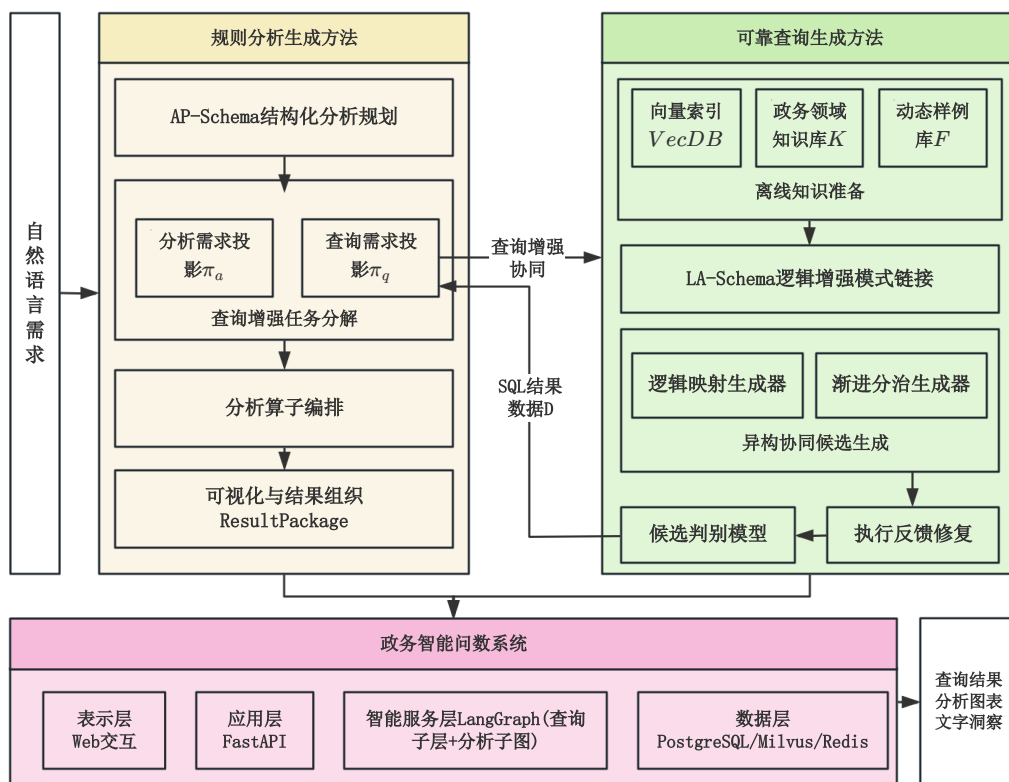


图 1-2 本文整体研究框架与技术链路示意图

本文的核心研究内容可从方法创新与系统实现两个层面加以概括。

方法层面

本文在方法层面提出两项核心创新，分别解决政务场景下“如何可靠查数”和“如何规范分析”两个关键问题。

(1) **基于逻辑增强与异构协同的政务数据查询生成方法（第三章）**。针对政务数据查询中业务术语语义模糊、逻辑计算隐含和复杂查询结构长尾分布等问题，本文提出一种面向政务场景的 Text-to-SQL 方法。该方法在离线阶段构建向量索引数据库、政务领域知识库和动态样例库，在在线推理阶段设计逻辑增强模式 LA-Schema，将业务术语映射、逻辑计算公式和值域约束显式注入数据库模式；同时设计逻辑映射生成器和渐进分治生成器组成的异构协同候选生成机制，并结合执行反馈驱动查询修复和基于微调的候选判别模型，提高最终 SQL 查询的准确性、执行成功率与稳定性。该研究的目标是解决政务场景下”查准”和”查稳”的问题，为后续数据分析提供可靠的数据输入基础。

(2) **基于分析规划与查询增强的政务数据分析方法（第四章）**。针对政务分析任务中需求约束复杂、查询与分析职责耦合以及结果表达需规范化等问题，本文提出一种协同查询能力的数据分析方法。该方法首先引入分析规划模式 AP-Schema，将自然语言分析需求转化为包含分析类型、指标、维度、筛选条件、统计粒度、操作分层和可视化规格的结构化中间表示；其次，通过查询增强的任务分解机制，将涉及统计口径、模式理解和复杂关联的操作下沉至查询模块完成，而分析模块仅在可靠查询结果上执行排序、占比、透视、标注等轻量分析操作；最后结合分析算子库和可视化规则，生成图表、表格、文字洞察和日志组成的标准化结果包。该研究的目标是解决政务场景下”怎么分析”和”如何规范输出”的问题。

系统实现层面

在上述两项方法创新的基础上，本文进一步完成系统的工程整合与落地验证。

(3) **政务智能问数系统设计与实现（第五章）**。本文将前述查询方法与分析方法组织为一个面向政务场景的智能问数系统。系统采用表示层、应用层、智能服务层和数据层四层架构：应用层以 FastAPI 构建统一 HTTP 入口，支持同步查询与异步分析两类调用方式；智能服务层以 LangGraph 组织查询子图和分析子图，使分析流程通过 AP-Schema 投影查询约束后复用查询能力；数据层利用 PostgreSQL 保存系统元数据和领域知识，利用 Milvus 承载向量索引，利用 Redis 支持缓存和流式事件通道，实现结果产物、执行日志和会话状态的持久化管理。该工作的目标是将方法创新转化为可交互、可追溯和可展示的系统能力，验证所提方法在实际政务场景中的可落地性。

1.4 论文章节安排

本文共分为六章，如图1-3所示，各章内容安排如下。

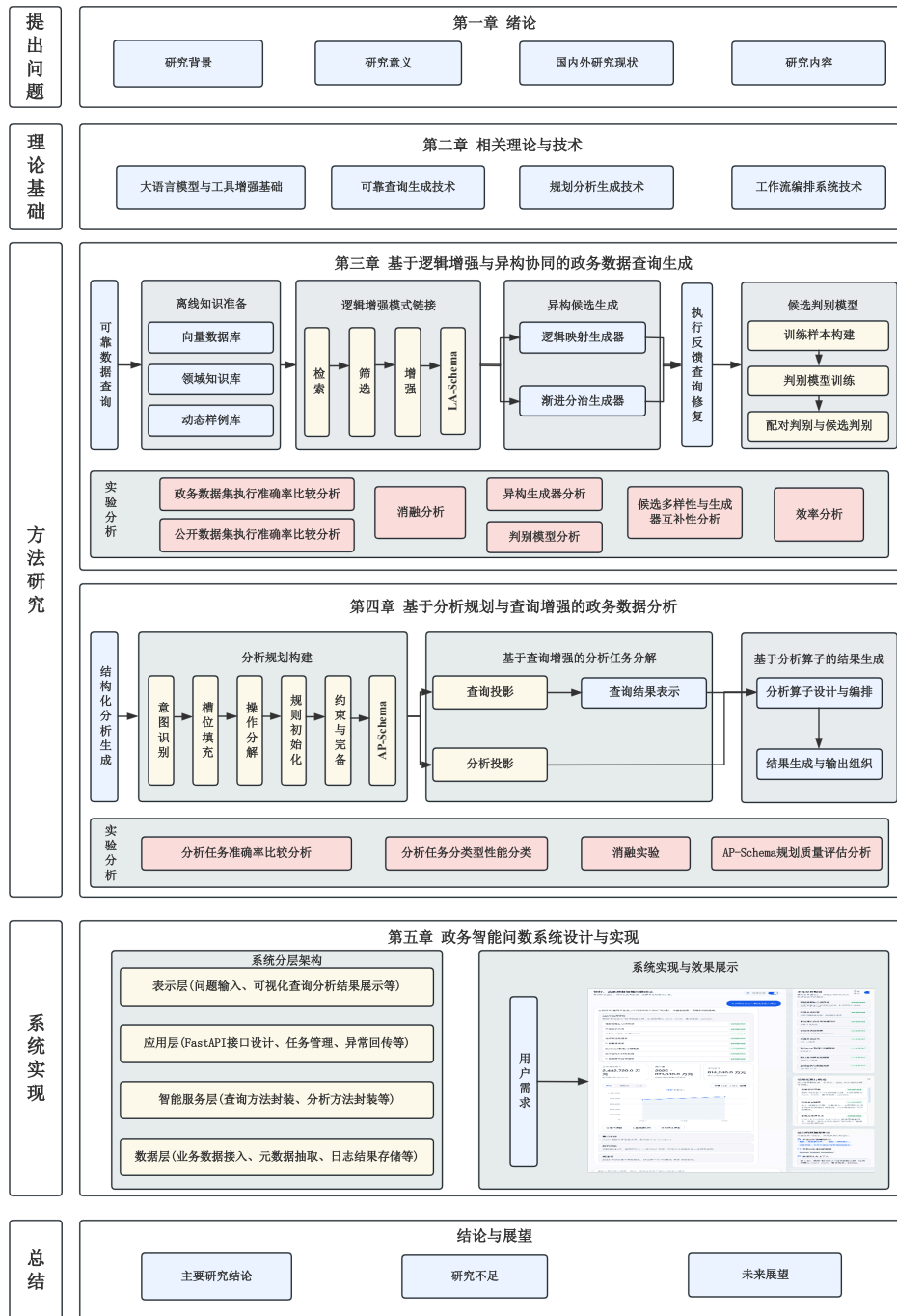


图 1-3 章节内容安排

第一章，绪论。阐述论文的研究背景与意义，综述国内外相关研究现状，归纳当前存在的 key 问题，并给出本文的研究内容与整体结构安排。

第二章，相关理论与技术。介绍论文所依赖的理论基础与关键技术，包括大语言模型与工具增强基础、面向可靠查询生成的关键技术、面向数据分析的规划与

结果生成技术，以及面向系统实现的工作流编排与服务协同技术。

第三章，基于逻辑增强与异构协同的政务数据查询生成方法。围绕政务场景中的可靠取数问题，提出包含离线知识准备、逻辑增强模式链接、异构候选生成、执行反馈修复和候选判别选择在内的 Text-to-SQL 方法，并通过自建政务数据集和公共数据集实验验证方法有效性。

第四章，基于分析规划与查询增强的政务数据分析方法。围绕政务场景中的结构化分析问题，提出 AP-Schema 分析规划模式、查询增强的任务分解机制、分析算子编排与标准化结果组织方法，并通过多维实验验证其性能与规划质量。

第五章，政务智能问数系统设计与实现。在前两章方法基础上，设计并实现一个政务智能问数系统，介绍系统总体架构、应用层与智能服务层实现、系统数据层存储以及系统功能展示与效果。

第六章，结论与展望。总结全文研究工作与主要创新点，分析现有工作的不足，并对后续研究方向进行展望。

1.5 本章小结

本章围绕数字政府建设和政务数据资源利用的现实需求，阐明了本文开展政务智能问数研究的背景、目标与意义；在系统梳理 Text-to-SQL、自动数据分析以及政务数据管理与应用平台相关研究进展的基础上，归纳出领域知识显式增强不足、分析规划与查询协同不足、系统化落地能力不足三个关键问题，并说明了开展本研究既具有现实必要性，也具备明确技术可行性；进而明确了本文以政务数据查询生成方法和政务数据分析方法两项方法创新为核心、以智能问数系统实现为落地验证的研究总体思路，并说明了全文的章节安排，为后续章节展开奠定基础。

第 2 章 相关理论与技术

本章不再对相关研究做展开式现状梳理，而是围绕后续方法与系统实现所依赖的关键技术进行说明。对于政务智能问数任务而言，系统需要同时处理自然语言理解、结构化生成、外部工具执行、流程组织和结果表达等环节^[10]。因此，本章按照模型基础、可靠查询生成、数据分析规划与结果生成、系统编排与服务协同四个层次展开，分别对应第三章至第五章的方法设计与工程实现。

2.1 大语言模型与工具增强基础

大语言模型（Large Language Model, LLM）是在大规模语料上进行预训练，并通过指令对齐获得任务遵循能力的语言模型^[10]。对于本文的政务智能问数任务而言，模型不仅承担通用文本生成功能，还需要支撑需求理解、SQL 生成、分析规划、结构化结果组织以及与外部工具协同执行等职责，因此其可用性取决于统一语义建模、少样本适配、结构化推理和工具调用等基础能力。

2.1.1 Transformer 与指令对齐

Transformer 以自注意力机制为核心，通过并行建模序列中不同位置之间的依赖关系，使模型能够在统一表示空间内同时处理词语语义、上下文关系和结构约束^[54]。这一能力使自然语言问题、数据库模式描述、领域知识片段和结构化输出约束能够被放入同一输入上下文中联合建模，是后续 SQL 生成、分析规划与结果组织的共同基础。

从模型形态看，GPT 类解码器模型^[7,55-57]强化了自回归生成能力，适合直接产出 SQL、代码片段和结构化结果；BERT 类模型^[58]强化了双向语义表示能力，更适合编码与匹配；T5^[59]则提供了统一的文本到文本建模范式，可将多种任务转化为一致的生成问题。对本文而言，第三章的候选 SQL 生成、第四章的 AP-Schema 规划生成以及第五章的结果组织，都更依赖生成式模型对结构化文本的连续建模能力。

仅有预训练还不足以支撑复杂应用。指令微调（Instruction Tuning）通过构造“指令 - 输入 - 输出”样本，使模型学会遵循任务边界、输出格式和领域约束^[60-61]；进一步的对齐训练，如基于人类反馈的强化学习^[62]，则有助于提升输出的稳定性、可用性和一致性。对于政务场景中常见的 SQL 生成、JSON 规划和标

准化结果包生成任务而言,这种“预训练+任务对齐”的技术路线尤其重要,因为其直接影响统计口径表达、输出格式约束和结果可追溯性^[14]。

2.1.2 上下文学习、链式推理与结构化输出

上下文学习(In-Context Learning, ICL)使模型能够在不更新参数的情况下,仅通过提示中的任务说明、数据库模式信息和少量示例完成新任务^[57,63]。在本文场景中,第三章的样例问题-SQL对、数据库模式说明和领域知识片段,第四章的分析类型与结果组织约束,都可以被视为上下文的一部分,用来缩短模型从自然语言到结构化结果之间的推理距离。

链式推理(Chain-of-Thought, CoT)进一步提升了复杂任务中间步骤显式化能力^[64-65]。当用户需求涉及同比、占比、排名、条件筛选和多阶段聚合等隐含逻辑时,模型若直接输出最终SQL或最终结论,容易在中间语义转换中出现遗漏。通过显式展开推理步骤,模型可以先识别指标、维度和约束,再逐步构造查询或分析计划。自一致性^[66]和思维树^[67]等方法进一步说明,多路径推理与结果比较能够有效缓解单路径生成的不稳定问题,这为第三章的多候选生成与判别、第四章的分析规划生成提供了直接启发。

结构化输出则把模型能力从“自由文本生成”进一步推进到“受约束结果生成”。在数据系统中,输出对象通常不是普通自然语言段落,而是SQL语句、JSON计划、代码调用参数或固定槽位结果。只有当模型能够在提示约束下生成字段完整、格式稳定、语义清晰的中间结果时,后续执行、校验和编排才具备可靠基础^[68]。因此,结构化输出既是第四章AP-Schema设计的前提,也是第五章标准化结果包组织的基础。

2.1.3 工具调用与代码执行

大语言模型擅长语义理解与生成,但在精确计算、数据库访问、代码执行和外部状态感知方面存在明显局限^[69]。因此,当前实用系统普遍采用“模型负责推理,工具负责执行”的增强范式。Toolformer^[70]、Gorilla^[71]等研究表明,模型能够学习何时调用外部工具以及如何构造调用参数;ReAct^[72]进一步把推理、行动和观察组织成闭环,使模型能够根据执行结果持续修正自身决策。

这一范式对于本文尤为关键。在查询生成场景中,SQL需要交由数据库执行并根据报错信息或执行结果修正;在数据分析场景中,模型生成的计划需要依赖代码执行环境、统计函数库和可视化组件落地;在系统实现层面,模型需要嵌入由状态对象、服务节点和消息通道组成的执行流程中。即工具调用不是附加能力,

而是把大语言模型从文本生成器转化为查询、分析和系统协同中枢的关键桥梁。

2.2 面向可靠查询生成的关键技术

Text-to-SQL 旨在把用户自然语言问题转换为可在关系数据库上执行的 SQL 语句^[11,73]。对于本文的政务智能问数任务而言,难点不在于“能否生成一条 SQL”,而在于能否在复杂业务语义、隐含统计口径和真实数据库结构之间建立稳定映射。因此,可靠查询生成不仅要求模型产出语法正确的语句,更要求系统具备识别业务术语、引入口径知识、组织样例上下文、借助执行反馈修复错误并从多候选中选择最优结果的能力^[19]。

2.2.1 任务定义与核心挑战

Text-to-SQL 的基本输入是自然语言问题 Q 与目标数据库模式 S , 输出是一条语义等价、语法正确且可执行的 SQL 语句 y ^[11]。在本文面向的政务场景中,这一任务主要面临三类挑战:其一,业务术语与物理字段之间常存在别名、缩写或多对多映射,模型即使理解问题含义,也未必能准确定位字段;其二,同比、占比、预算执行率等核心指标通常依赖隐含计算公式和统计口径,若缺乏显式知识支撑,模型容易生成“语法正确但业务错误”的 SQL;其三,真实业务需求兼具高频固定模式与低频复杂长尾结构,单一路径生成往往难以同时兼顾效率与准确率^[15,19]。

围绕这些问题,已有工作分别从表示、约束和生成策略等方面提出改进。Seq2SQL、SyntaxSQLNet 开启了端到端神经生成路径^[21-22]; RAT-SQL、BRIDGE、PICARD 和 RESDSQL 分别从关系建模、内容锚定、语法约束和 Schema 筛选等方面提升了查询生成可靠性^[23-26];在大模型阶段,DAIL-SQL、DIN-SQL、CHASE-SQL、XiYan-SQL 与 OpenSearch-SQL 则说明,提示组织、样例检索、多路径生成与候选判别已成为提升复杂查询性能的重要手段^[27-31]。

2.2.2 Schema/值检索与领域知识增强

在可靠查询生成链路中,模式链接 (Schema Linking) 承担着“把自然语言中的实体、指标和条件映射到数据库结构”的基础作用。RAT-SQL^[23]通过关系感知编码显式建模问题 Token 与 Schema 元素之间的关系,BRIDGE^[24]利用数据库内容进行语义锚定,RESDSQL^[26]则通过排序与筛选把庞大 Schema 压缩为更适合生成的候选子模式。这些工作共同揭示出一点:高质量 SQL 生成的前提,不是让模型直接猜测字段绑定,而是先把与当前问题最相关的结构信息组织出来。

对于垂直领域系统，仅依赖原始表名、列名和注释通常仍然不够。数据库中常见的字段缩写、别名表达、口径术语和隐含逻辑，往往无法被数据库模式显式刻画，因此结构化领域知识增强成为连接业务语言与物理数据库的必要桥梁。一类方法通过补充别名、样例值、字段说明等语义提示提升模式可读性；另一类方法则进一步把业务术语映射、逻辑公式和值域约束组织为可检索知识单元，并在在线推理时注入问题相关片段^[15,19]。第三章的 LA-Schema 设计正属于这一思路的延伸。

值检索与动态样例也是这一链路的重要组成部分。对包含行政区划、项目名称等实体的问题而言，仅靠模型记忆很难稳定命中正确取值，向量检索和近邻召回可以作为值匹配与样例选择的外部支撑。在大模型 Text-to-SQL 中，DAIL-SQL 与 OpenSearch-SQL 均显示出高相关动态样例对生成质量的重要影响^[27,31]。这说明可靠查询生成并不是单轮提示行为，而是检索，组织，生成的联合过程。

2.2.3 候选生成、执行反馈与结果选择

大模型时代的 Text-to-SQL 系统逐渐从“单答案生成”转向“多候选生成 + 执行反馈 + 最优选择”。其根本原因在于复杂 SQL 问题往往存在较高的推理不确定性：单次生成容易受到提示表述、随机采样和局部推理路径的影响，从而遗漏条件、错误绑定字段或采用错误聚合方式。因此，多路径候选生成不再只是简单重复采样，而是通过不同提示范式、不同分解路径和不同温度设置主动扩大候选空间。

CHASE-SQL^[29]和XiYan-SQL^[30]表明，多生成器和多推理路径能够显著提升复杂问题覆盖度；Alpha-SQL^[32]进一步将 SQL 生成建模为搜索过程，说明复杂查询更适合采用“生成与评估交替”的策略。另一方面，PICARD^[25]等工作说明，语法约束只能保证“写得像 SQL”，却不能保证“问得对业务”，因此执行反馈成为进一步验证候选质量的重要机制。当 SQL 真实执行后，系统可以根据报错信息、空结果、字段不匹配等反馈开展修复，从而把数据库引擎转化为事实校验器。

候选选择是保证最终输出质量的最后一道关口。一致性投票适合处理低复杂度任务，但在存在业务逻辑冲突或执行表现差异时，仍需要更细粒度的判别机制。近年来的偏好优化和候选判别方法表明，结合执行结果、语义一致性和任务约束进行成对比较，通常比简单投票更可靠^[29-30]。这些思路为第三章中异构候选生成、执行反馈修复和判别选择机制提供了直接参考。

2.3 面向数据分析的规划与结果生成技术

如果说 Text-to-SQL 解决的是“把数据取对”，那么面向数据分析的关键问题则是“如何围绕已获取数据开展合适分析并形成可理解结果”。与查询任务相比，

数据分析任务具有更强的开放性：用户通常不会直接给出完整分析步骤，而是以自然语言提出分析目标。系统需要先把需求转化为可执行规划，再调用查询、计算与可视化工具，最终输出图表、表格和文字洞察^[12-13]。这也是第四章方法设计需要重点解决的问题。

2.3.1 分析任务链条与中间规划

已有研究普遍将自动数据分析过程拆分为需求理解、分析规划、工具执行和结果生成等阶段^[12,20]。其中，分析规划处于承上启下的位置：它既要把自然语言中的分析对象、指标、维度、时间范围和展示要求显式化，又要决定哪些步骤应由查询模块完成，哪些步骤保留到分析层处理。若缺乏这一中间层，系统就容易在复杂需求下出现“先取什么数据并不清楚”“展示和计算混在一起”“结果虽能生成但难以验证”等问题。

InsightPilot^[20]将数据探索拆解为一系列分析操作，说明分析任务本质上并非单步问答，而是带有过程结构的决策链。Analysts^[13]则指出，模型在分析规划、统计执行和结果解释上的能力并不均衡，因此需要更强的外部约束来提高规划与输出质量。对于政务场景而言，这种约束尤为重要，因为分析任务通常要遵循固定统计口径、固定展示习惯和更强的可复核要求^[14,51]。

基于此，结构化中间表示成为提高可控性的常见路径。其核心思想不是让模型一步产出最终结论，而是要求模型先输出一个受约束的分析计划，用以描述分析类型、关注指标、分析维度、筛选条件、计算方式以及展示规格。常见中间表示包括计划槽位、任务图、结构化脚本和可解析 JSON 对象等。第四章提出的 AP-Schema 本质上就是面向政务分析任务的结构化中间表示，用于把开放式需求转化为可校验、可复用、可执行的分析对象。

2.3.2 代码生成、分析算子与可视化组织

在分析规划确定之后，系统还需要完成计算和表达两个层面的执行。代码生成模型如 Codex^[39]、Code Llama^[40]、StarCoder^[41]和 WizardCoder^[42]证明了大模型可以根据自然语言描述生成较高质量程序代码，这为自动统计分析、数据变换和图表绘制提供了技术前提。但在实际系统中，若让模型自由生成整段分析代码，往往会带来鲁棒性不足、重复逻辑过多和结果难以复核等问题。

因此，越来越多的系统开始采用“规划 + 算子 + 工具”的组织方式。Data Interpreter^[12]通过代码执行环境完成端到端分析，TaskWeaver^[45]强调代码优先与任务分工，LIDA^[43]和 Chat2VIS^[44]则从可视化生成角度展示了模型如何驱动图表代

码与视觉表达。ChartGPT^[48]进一步说明，自然语言到图表之间并非简单模板映射，而是涉及图表类型选择、坐标映射和表达目标对齐。与此同时，Struc-Bench 表明，若缺少明确槽位和格式约束，模型在结构化结果生成上仍容易出现字段遗漏和格式漂移^[68]。

对于本文而言，这意味着分析结果生成不宜完全依赖自由代码生成，而更适合在分析算子和可视化规则层面引入明确边界：模型负责生成分析计划和调用意图，系统则依据预定义的轻量算子库完成排序、占比、透视、标注和图表组织等稳定操作。这样既保留了大模型的灵活性，又提高了统计过程和展示结果的一致性。

2.3.3 查询与分析分层协同

由于模型在分析规划、统计执行和结果解释上的能力并不均衡^[13]，从系统实现角度看，让分析模块重新承担数据库理解与 SQL 生成职责并不可靠。更稳妥的做法是把“数据获取”和“数据分析”视作两个职责清晰但密切协同的阶段：凡涉及数据库模式理解、复杂字段绑定、跨表关联和统计口径计算的操作，应尽量下沉到查询层完成；凡只依赖已获取结果的重组、排序、对比、占比和注释等操作，则由分析层负责。

这种划分有三点直接好处。第一，它复用了查询模块在模式链接、逻辑增强和执行修复方面的已有能力，避免分析阶段重复犯错。第二，它使分析规划能够围绕统一的中间表示展开，便于对查询约束和分析约束分别建模。第三，它提升了系统整体可解释性，因为每一步究竟是“为了取数”还是“为了分析”都能够被明确追踪。第四章的方法设计正是基于这种分层协同思想展开。

2.4 面向系统实现的工作流编排与服务协同

当查询与分析能力从单点方法发展为可交互系统后，关注点便进一步延伸到工程实现层面。一个实用系统不仅要给出结果，还需要组织任务流程、维护状态、回传异常并沉淀中间产物，以支持复核、追问和问题定位。因此，工作流编排、状态管理、服务协同和角色分工成为第五章系统设计必须解决的问题。

2.4.1 工作流图编排、状态管理与异常回传

大语言模型应用走向复杂系统后，单轮提示已经难以承载完整业务流程。一个典型任务往往需要经历需求解析、知识检索、SQL 生成、执行验证、结果封装和前端回传等多个阶段，因此工作流图编排成为组织模型能力与系统组件的重要

方式。其核心思想是把任务拆分为若干节点，每个节点负责相对单一的职责，再通过显式状态对象和条件边连接形成可追踪、可恢复的执行路径。

ReAct^[72]提供了“推理 - 行动 - 观察”的通用闭环思想，说明模型驱动任务执行更适合放入可循环、可反馈的流程而非一次性输出。对系统实现而言，这一思想需要进一步工程化：系统要把模型调用、数据库执行、文件生成和消息回传等步骤纳入统一状态机中，并对成功、失败、重试和终止等状态进行管理。周祥生等人^[74]关于大模型推理任务调度的研究也说明，随着模型能力嵌入真实业务链路，任务优先级、资源分配和状态调度会直接影响系统稳定性与用户体验。对于政务场景，还需要进一步满足过程留痕、异常回传和服务可审计等要求^[16,52]。

2.4.2 轻量级 Agent 协同与角色分工

在复杂任务中，不同节点承担不同角色：分别负责路由请求、查询生成、分析组织与结果汇总。这类“角色分工 + 消息协同”的设计与近年 Agent 研究存在天然联系。AutoGen^[75]强调通过多角色对话组织工具调用与任务分工，MetaGPT^[76]强调基于标准操作流程的结构化协同，CAMEL^[77]则展示了角色扮演框架下的任务协商能力。郭先会等人^[78]指出，大语言模型智能体的工程价值取决于角色职责是否清晰、工具接口是否稳定。

对真实业务系统而言，更具可操作性的不是重型多智能体社会模拟，而是轻量级服务协同。即系统不必追求大量 Agent 间的开放式对话，而是将查询、分析、调度和结果组织等职责稳定封装为可复用服务角色。李轶夫^[79]指出，生成式大模型智能体形成工程价值的前提，是把角色能力、工具接口和业务场景有机结合，而非停留在概念展示层面。相关政务实践表明，分层设计、模块化集成和模型服务化部署，是保障系统稳定落地的重要路径^[16,51-52]。

2.5 本章小结

本章围绕后续方法与系统实现所依赖的关键技术，对大语言模型基础、可靠查询生成、数据分析规划与结果生成，以及系统编排与服务协同等内容进行说明。

Transformer 架构、上下文学习等和工具增强构成了大语言模型处理复杂数据任务的能力底座。在查询生成方面，Schema 链接、领域知识增强、值检索、动态样例、多候选生成和执行反馈构成了提高 NL2SQL 可靠性的关键技术链。在数据分析方面，结构化规划、代码执行、分析算子和查询分析分层协同体现了从开放式需求到规范化分析结果的实现路径。在系统实现方面，工作流编排、状态管理与轻量级角色协同为复杂智能系统的稳定运行、可追溯性与工程落地提供支撑。

第3章 基于逻辑增强与异构协同的政务数据查询生成方法

3.1 引言

文本到 SQL 生成 (Text-to-SQL) 是将自然语言问题转换为可在关系数据库上执行的结构化查询语言的语义解析任务^[11], 是实现“智能问数”和降低数据分析门槛的关键技术。随着数字政府建设的深入, 海量的政务异构数据急剧增长, 非技术背景的公务人员直接、高效地从数据中获取决策支持显得尤为迫切。

本章提出的 Text-to-SQL 方法, 其任务形式化定义为: 给定用户的自然语言问题 Q 、目标数据库模式 S 、领域知识库 K 以及动态样例库 F , 任务的目标是生成一条可正确执行并返回预期结果的 SQL 查询语句, 即与用户意图最大化一致:

$$y^* \equiv \arg \max_{y \in Y} P_{\theta}(y \mid Q, S, K, F) \quad (3-1)$$

其中 Y 为符合 SQL 语法的候选集合, θ 为模型参数。

本章的研究目标是面向政务数据查询这一具体应用场景, 以系统性解决复杂政务语义下查询生成的正确性、执行成功率与鲁棒性问题。以一个典型的政务查询为例, 当用户提出“2023 年各区县的一般公共预算收入同比增长率是多少?”系统需要理解“一般公共预算收入”对应的物理字段、“同比增长率”隐含的跨年度计算逻辑以及“区县”维度的分组聚合方式, 才能生成正确的 SQL 查询。这一过程涉及的业务知识远超数据库模式本身所能表达的信息边界。

与通用领域的 Text-to-SQL 任务相比, 政务数据查询面临着更为突出的复杂性挑战, 主要体现在以下方面:

(1) 业务术语的语义模糊与别名多样。政府工作存在大量专业术语及其简称、别名和口语化表达。例如, “一般公共预算收入”可能被简称为“一般预算收入”或“公共预算收入”, 而数据库中的物理字段名可能为 `ybsr` 等缩写形式。这种多对一的映射关系显著增加了模式链接的难度。

(2) 指标计算逻辑的隐含性。大量政务指标并非数据库中的直接存储字段, 而需要通过特定的计算公式推导得出。例如“同比增长率”需要通过当期值与上期值的差值除以上期值来计算, “预算执行率”需要执行数除以预算数。这些隐含的计算逻辑未显式存在于数据库模式中, 导致大语言模型在面对此类查询时极易产生“逻辑幻觉”, 即生成语法正确但业务逻辑错误的 SQL 语句。

(3) 查询结构的复杂性与多样性。SQL 查询的复杂度分布呈现明显的长尾特

征：高频需求多为结构相对固定的简单查询，而低频需求则可能涉及多层嵌套子查询、跨表关联、条件聚合等复杂 SQL 结构。单一的生成范式难以同时兼顾这两类需求，对简单查询可能引入不必要的推理冗余，对复杂查询则可能因推理深度不足而导致准确率骤降。

基于上述问题，本章的方法设计围绕以下三个目标展开：第一，通过构建领域知识库并将其显式注入数据库模式，设计一个能快速理解政务领域的数据库模式，以提升模式链接环节对政务语义的理解准确性；第二，通过设计异构协同的双路生成机制，平衡高频标准需求与低频复杂长尾需求的生成效率与质量；第三，通过引入执行反馈修复机制与基于微调的候选判别模型，提升最终查询结果的稳定性与鲁棒性。

3.2 总体框架设计

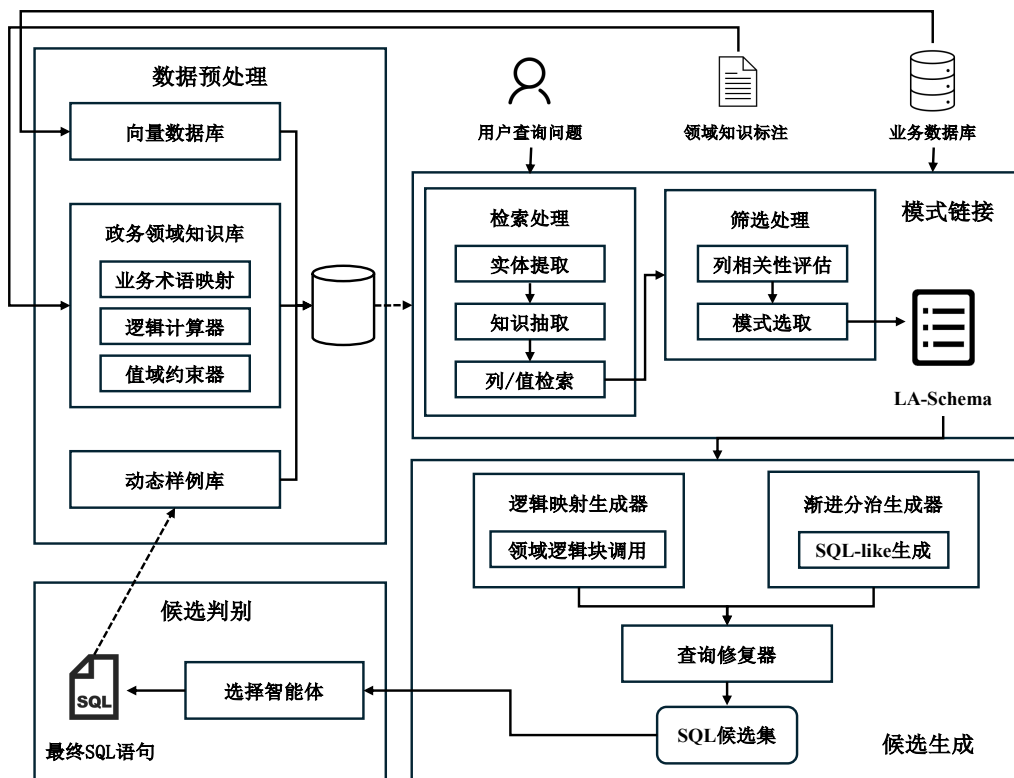


图 3-1 逻辑增强与异构协同的 Text-to-SQL 框架图

如图3-1所示，本章提出的基于逻辑增强与异构协同的 Text-to-SQL 方法包含离线知识准备和在线推理生成两条链路，由四个核心模块组成：离线数据预处理模块、模式链接模块、候选生成模块和候选判别模块。

离线阶段负责为在线推理提供高可用的辅助数据支撑。具体而言，该阶段从

业务数据库中提取字符型列及其取值构建向量索引数据库 (*VecDB*), 为后续的值检索提供基础; 通过人工梳理与结构化组织, 构建包含业务术语映射、逻辑计算公式和值域约束的政务领域知识库 K ; 同时, 利用大语言模型对已有的“问题-SQL”对进行知识解构, 生成包含中间推理链的动态样例库 F 。

在线阶段以用户的自然语言查询为输入, 依次经过以下流程。首先, 模式链接模块从原始数据库模式中检索与问题相关的表、列和值信息, 并注入领域知识库信息, 将原始数据库模式 S 转化为面向当前问题的逻辑增强模式 (LA-Schema) S^* 。随后, 候选生成模块采用异构协同策略, 通过逻辑映射生成器和渐进分治生成器两条并行推理路径, 以生成多样化的 SQL 候选集。对于生成过程中出现的语法错误或执行异常, 查询修复器利用执行反馈信息引导大语言模型进行自反思修复。最终, 候选判别模块通过微调的二分类选择模型对候选集进行配对比较与得分聚合, 选出最优的 SQL 查询语句作为最终输出。

3.3 离线知识准备

离线数据预处理模块的目标是为后续在线推理生成提供低冗余、高可用的辅助数据。本模块通过构建多维度的数据知识支撑体系, 为后续模式链接和候选生成过程提供精确的语义和逻辑基础。该支撑体系包含向量索引数据库、政务领域知识库以及动态样例库三个核心组件。

3.3.1 向量库构建

为保障大语言模型生成的 SQL 语句与底层数据库实际状态的一致性, 本模块构建了高质量的向量索引数据库 *VecDB*, 用于模式链接过程中的值检索。

具体构建流程如下。首先, 从数据库中提取所有字符型列 C 及其对应的取值 v , 构造联合文本序列 $s = [C : v]$ 。通过将值的向量表示携带其所属列的上下文信息, 以帮助在检索时区分不同列下的同名值。

然后, 利用预训练编码模型 BGE-m3^[80]将上述文本序列映射为高维语义向量 h 。此外考虑到政务数据库的规模庞大, 为优化存储开销并提升检索效率, 仅对字符串类型数据进行向量化。所有向量存入基于分层可导航小世界图 (Hierarchical Navigable Small World, HNSW)^[81]的索引结构中。HNSW 通过构建多层跳跃链接的近邻图结构, 在高维空间中实现对数级别的近似最近邻检索, 相较于暴力搜索在大规模向量库上具有显著的效率优势, 同时保持较高的召回率。最终向量库构

建过程可表述为：

$$\begin{cases} \mathbf{h} = \text{BGEEncode}([C : v]) \\ \text{VecDB} = \{(\text{HNSW}(\mathbf{h}), C, v) \mid \forall C, v\} \end{cases} \quad (3-2)$$

向量数据库在在线阶段承担两方面的核心作用：一是通过语义相似度检索实现对用户问题中关键实体的精准召回，即使存在拼写差异或表述变化也能准确定位对应的数据库值；其次是为动态样例检索提供嵌入向量基础，使系统能够根据当前问题的语义特征检索最相关的历史样例。

3.3.2 政务领域知识库构建

针对政务数据查询中业务逻辑复杂且语义模糊的问题，本文构建了结构化的政务领域知识库 $K = \{B, L, V\}$ 。不同于传统的非结构化文本注释方式，该知识库采用结构化三元组形式存储先验业务知识，旨在为后续的模式链接和候选生成过程提供确切的逻辑增强。

知识库由以下三个组件构成：

(1) 业务术语映射 B ：记录政务领域中复杂业务指标与物理数据库字段之间的对应关系。例如，三元组（“一般公共预算收入”， $ybsr$ ，“一般公共预算收入金额字段”）将业务术语映射到具体的数据库字段。

$$B = \{(t_i, c_i, d_i)\}_{i=1}^{|B|} \quad (3-3)$$

其中 t_i 为业务术语， c_i 为对应的物理字段或字段组合， d_i 为转换说明。

(2) 逻辑计算器 L ：存储复杂的计算公式及聚合逻辑，将其表示为 SQL 代数表达式。例如，对于“同比增长率”这一指标，其 SQL 表达式为（当期值 - 上期值）/ 上期值 * 100。通过将复杂的业务计算逻辑预先编码为 SQL 代数表达式并以虚拟列的形式注入数据库模式 Schema ，模型在生成 SQL 时可以直接调用这些预定义的逻辑块，从而规避了模型在处理大规模复杂计算公式时的推理冗余和盲目判断。

$$L = \{(m_j, f_j, e_j)\}_{j=1}^{|L|} \quad (3-4)$$

其中 m_j 为指标名称， f_j 为计算公式的 SQL 表达式， e_j 为计算说明。

(3) 值域约束器 V ：记录特殊业务字段的取值范围或枚举值。例如，（ $xzqh$, {“荣华街道”，“博兴街道”，“经济开发区”，...}）记录行政区划名称字段的合法取值。值域约束的引入能够帮助模型生成包含正确 WHERE 条件的 SQL 语句，避

免因值不匹配导致的查询结果为空。

$$V = \{(c_k, R_k)\}_{k=1}^{|V|} \quad (3-5)$$

其中 c_k 为字段名, R_k 为该字段的合法取值范围或枚举值集合。

知识库的来源包括政务业务术语表、字段文档、业务统计口径规则以及领域专家整理的历史问答知识。其标注过程以人机协同方式,人工标注为主,辅助以大语言模型辅助标注。知识的归一化组织遵循“术语—标准表达—对应字段/取值—业务说明”的统一结构,便于在模式链接阶段进行高效检索与注入。

3.3.3 动态样例库构建

少样本 (Few-shot) 学习是辅助大语言模型进行 SQL 生成的一种重要方法。研究表明,示例问题的质量和与当前问题的相关性对 Text-to-SQL 任务的生成效果具有关键影响^[27,31]。基于此,本模块采用动态少样本学习策略,而非固定的少样本集合,以提高大语言模型在不同查询场景下的适应性和生成质量。

动态样例库的构建流程如图3-2所示。首先,利用大语言模型对原始的“问题-SQL”对进行知识解构,生成中间层的思维链 (Chain-of-Thought, CoT) 信息。该过程将每个样例从 $\langle \text{Query}, \text{SQL} \rangle$ 二元组扩展为包含完整推理逻辑的 $\langle \text{Query}, \text{CoT}, \text{SQL} \rangle$ 三元组。以此构建一个逻辑表达能力更强的样例库。CoT 信息的具体组成如表3-1所示。这种结构化的推理链不仅为后续生成器提供了更丰富的推理线索,也使得模型能够通过模仿样例中的推理过程来提升自身的逻辑表达能力。

表 3-1 思维链 (CoT) 信息的组成要素

要素	标识	说明
查询意图分析	reason	对用户问题的语义理解与意图解读
关键列	columns	查询涉及的数据库列
筛选值	values	WHERE 等筛选条件中使用的具体值
SELECT 内容	SELECT	SELECT 子句应返回的字段与表达式
类 SQL 语句	SQL-like	忽略连接条件的简化 SQL 骨架

而在在线推理阶段,当接收到用户查询 Q 时,系统启动端到端的动态样例检索流程,即从样例库 F 中选取与当前问题最相关的 k 个样例,以构建高质量的少样本提示。该流程融合了掩码问题相似性 (Masked Question Similarity, MQS)^[27]与查询结构相似性的双重度量,从问法相似和 SQL 相似两个维度综合筛选样例。

具体而言,检索过程包含以下步骤。首先,对用户查询 Q 执行掩码操作,将其中的领域特定实体(如表名、列名、具体值)替换为统一占位符,得到掩码后的

查询 Q' :

$$Q' = \text{Mask}(Q) \quad (3-6)$$

随后,同样利用预训练编码模型 BGE-m3 将 Q' 映射为稠密向量表示 $v_{Q'} = \text{Enc}(Q')$, 并在同样经过掩码处理的样例问题向量库中, 使用基于 HNSW 的近似 k 近邻检索 (Approximate Nearest Neighbor, ANN) 算法召回一批候选样例集合 F_{cand} 。掩码操作使得相似度计算更关注问题的结构与语义特征, 而非表面的实体匹配, 从而有效提升跨数据库场景下的样例检索质量。

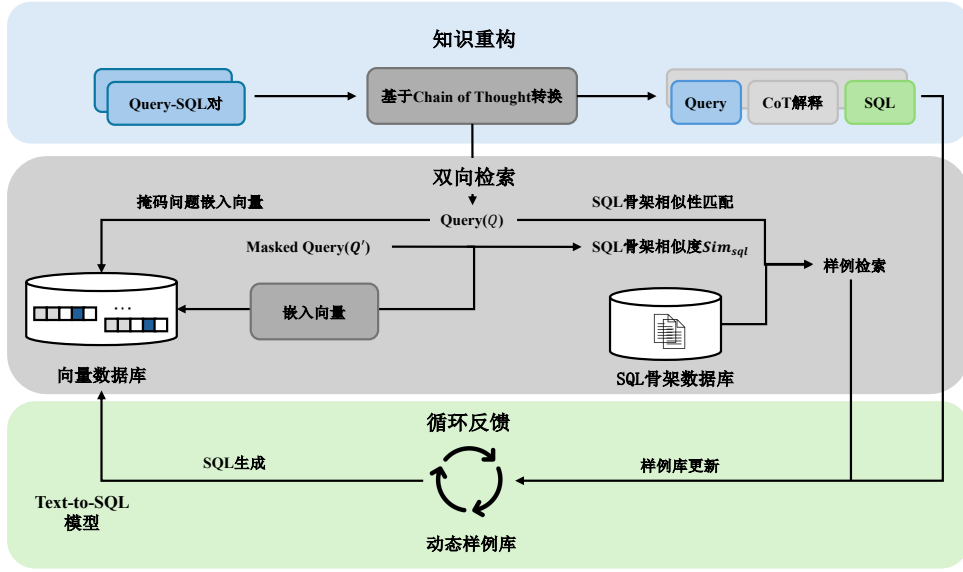


图 3-2 动态样例库构建流程图

其次在候选样例召回的基础上, 系统进一步引入查询结构相似度对候选样例进行精排。具体地, 先利用一个初步预测模型为 Q 生成近似 SQL 查询 $\hat{s}_Q$ 作为目标 SQL 的估计, 然后将 $\hat{s}_Q$ 与每个候选样例的 SQL 查询 s_i 根据 SQL 关键词编码为二元离散语法向量, 计算结构相似度 $\text{Sim}_{\text{sql}}(\hat{s}_Q, s_i)$ 。最终的样例排序综合考虑问题相似度与查询相似度, 并通过预定义阈值 τ 过滤查询相似度不足的样例:

$$F_k = \text{Top-}k \left\{ \text{Sim}_{\text{ques}}(v_{Q'}, v_{Q'_i}) \mid \text{Sim}_{\text{sql}}(\hat{s}_Q, s_i) > \tau \right\} \quad (3-7)$$

其中 Sim_{ques} 为掩码问题向量间的相似度, F_k 即为最终选取的 k 个动态样例。选定的样例以 $\langle \text{Query}, \text{CoT}, \text{SQL} \rangle$ 三元组的形式拼接至提示模板中, 与数据库模式、检索到的相似值及领域规则共同构成完整的少样本提示, 以引导大语言模型生成目标 SQL。

最终的动态样例库具备渐进更新的能力。当渐进分治生成器产生的候选 SQL

被判别模型确认为最优答案后，该查询及其对应的推理过程可作为新的样例加入样例库（可控制是否开启），实现样例库的持续丰富与质量提升。

3.4 基于逻辑增强的模式链接

模式链接是 Text-to-SQL 任务中的关键环节，其目标是将用户的自然语言查询与数据库模式中的元素（包括表、列和值）相关联，从庞大的数据库模式中筛选出最小必要且包含丰富语义信息的数据库子模式^[29]。本节提出一种基于逻辑增强的模式链接方法，通过动态检索元素信息、选取关键模式片段并注入领域逻辑信息，将原始数据库模式 S 转化为面向特定问题 Q 的最小增强模式 S^* 。

该方法的处理流程包含检索、筛选和增强三个阶段，其完整算法描述如算法3.1所示。

算法 3.1: 基于逻辑增强的模式链接算法

Input: 用户查询 Q ，原始数据库模式 S ，列检索召回数 k ，动态样例 $F_u = \{f_1, f_2, \dots, f_k\}$ ，领域知识库 $K = \{B, L, V\}$ ，向量数据库 VecDB，大语言模型 LLM

Output: 最小增强模式 S^*

- 1 初始化：候选模式片段集合 $S_{\text{cand}} \leftarrow \emptyset$;
// 阶段一：检索
- 2 $W_{\text{entity}} \leftarrow \text{LLM}(Q, F_u, K)$; // 实体提取
- 3 **foreach** $w \in W_{\text{entity}}$ **do**
- 4 $C_{\text{topk}} \leftarrow \text{SemanticSearch}(w, C, k)$; // 列检索
- 5 $V_{\text{cand}} \leftarrow \text{HNSW}(w, \text{VecDB})$; // 值近邻召回
- 6 $\text{val} \leftarrow \arg \max_{v \in V_{\text{cand}}} \text{Sim}(\text{Enc}(w), \text{Enc}(v))$; // 语义精排
- 7 $S_{\text{cand}} \leftarrow S_{\text{cand}} \cup \{C_{\text{topk}}, \text{val}, \text{Table}(\text{val})\}$;
- 8 **end**
// 阶段二：筛选
- 9 $S_{\text{filter}} \leftarrow \text{LLM}(Q, S_{\text{cand}})$; // 列相关性评估与筛选
- // 阶段三：增强
- 10 $C_{\text{alias}}, C_{\text{virtual}}, D_{\text{prompt}} \leftarrow \text{GetKnowledge}(W_{\text{entity}}, B, L, V)$;
- 11 $S^* \leftarrow \text{TransLASchema}(S_{\text{filter}}, C_{\text{alias}}, C_{\text{virtual}}, D_{\text{prompt}})$; // 注入知识并转换为
 LA-Schema
- 12 **return** S^*

检索阶段旨在从数据库中搜索与查询潜在关联的值和列。该阶段首先以用户查询 Q 、动态样例 F_u 及领域知识库 K 为输入，利用大语言模型识别用户问题中的关键词与实体，获取实体集合 W_{entity} 。随后，对 W_{entity} 中的每个实体 w 分别执行列检索与值检索。列检索基于实体 w 与列描述之间的语义相似度，筛选每个实体

对应的 Top- k 候选列。值检索基于 HNSW 索引从向量数据库中快速召回语义最近邻候选集 V_{cand} ，并利用嵌入向量的余弦相似度进行精排，以实现目标值及其所属列的精准定位。检索到的候选列、匹配值及其所属表共同构成候选模式片段集合 S_{cand} 。

筛选阶段旨在将检索得到的表、列、值集合缩减为生成 SQL 所需的最小数据库模式。根据检索得到的候选列元素集合，让大语言模型评估每个列与用户查询的相关性，过滤无关列，最终得到最小数据库模式 S_{filter} 。该筛选步骤能够有效降低后续生成阶段的输入规模，减少因冗余信息导致的模型注意力分散和幻觉问题。

增强阶段是本方法的核心创新所在。在获得最小数据库模式后，系统从领域知识库 K 中提取与当前查询相关的业务知识，并将其显式注入模式表示中，形成逻辑增强模式 LA-Schema。LA-Schema 的组织方式参照 M-Schema^[30]的半结构化格式，以层次化的方式展示数据库、表和列之间的结构关系，并在此基础上添加政务领域业务知识的显式表达。具体的增强内容包括三个层面：

(1) **术语归一与别名扩展**：对于知识库 B 中与当前查询匹配的业务术语映射，在对应字段上添加 Business Term 标记，标注该字段在业务语义上的对应关系。

(2) **虚拟逻辑列注入**：对于知识库 L 中匹配的计算指标，在 LA-Schema 中创建虚拟列 (Virtual columns)，并附带预定义的 SQL 计算公式。以便大语言模型在生成 SQL 时可以直接引用预定义的计算逻辑，无需从零开始推理复杂的业务公式。

(3) **列值提示与约束**：对于知识库 V 中匹配的值域约束以及通过值检索获得的匹配值，在对应列上添加 Value Term 标记，提示模型在 WHERE 条件中使用正确的匹配值。

为直观说明 LA-Schema 的增强效果，以下给出一个对比示例。对于查询“帮我找出所有状态为已完工的民生项目，并计算它们的资金结余率。”，原始 DDL Schema、M-Schema 与 LA-Schema 的展示结果如表3-2所示。

可以看到，原始 DDL Schema 仅提供了最基本的列名与数据类型，缺乏任何语义信息；M-Schema 在此基础上补充了中文列描述和示例值，但仍无法表达业务术语映射、值域约束等领域知识。而 LA-Schema 通过显式注入 Business Term、Value Term 与 Virtual Columns 等增强信息，使大语言模型能够准确理解“民生项目”对应的物理字段取值、“已完工”的编码映射以及“资金结余率”的计算公式，从而显著降低模式链接阶段的错误率。

表 3-2 原始 DDL Schema、M-Schema 与 LA-Schema 对比示例

模式类型	表示内容
DDL Schema	<pre>CREATE TABLE t_gov_proj(p_id INTEGER PRIMARY KEY, p_name VARCHAR, amt_tot INTEGER, amt_act INTEGER, p_type TINYINT, status TINYINT, p_year INTEGER)</pre>
M-Schema	<pre>[DB_ID] gover_database [Schema] # Table: t_gov_proj [(p_id: INTEGER, Primary Key, 项目编码, Examples:[1001,1002,1003]), (p_name: VARCHAR, 项目名称, Examples:[" 经开区智慧城市规划服务项目", "T 小区安居提升工程项目", "E 社区服务中心数字化基础设置建设"]), (amt_tot: INTEGER, 总概算, Examples:[1000000,2000000,3000000]), (amt_act: INTEGER, 实际支出, Examples:[100000,200000,300000]), (p_type: TINYINT, 项目类型, Examples:[1,2,3]), (status: TINYINT, 状态, Examples:[0,1]), (p_year: INTEGER, 年份, Examples:[2023,2024,2025])]</pre>
LA-Schema	<pre>[DB_ID] gover_database [Schema] # Table: t_gov_proj [Physical Columns: (p_id: INTEGER, Primary Key, 项目编码, Examples:[1001,1002,1003]), (p_name: VARCHAR, 项目名称, Examples:[" 经开区智慧城市规划服务项目", "T 小区安居提升工程项目", "E 社区服务中心数字化基础设置建设"]), (amt_tot: INTEGER, 总概算, Examples:[1000000,2000000,3000000]), (amt_act: INTEGER, 实际支出, Examples:[100000,200000,300000]), (p_type: TINYINT, 项目类型, Examples:[1,2,3]), Business Term:{" 民生项目": " 民生工程"}, Value Term:{" 民生工程":1, " 基础设施":2, " 产业扶持":3}), (status: TINYINT, 状态, Examples:[0,1]), Value Term:{" 进行中":0, " 已完工":1}), (p_year: INTEGER, 年份, Examples:[2023,2024,2025]) Virtual Columns: (balance_rate: Logical Term, 资金结余率, Examples:[0.1,0.2,0.3], Formula:"(amt_tot-amt_act)/amt_tot")]</pre>

3.5 异构候选生成

在政务数据查询场景中，用户查询的复杂度分布呈现显著的长尾特征^[18]，即高频需求往往是结构相对固定的标准查询，而低频需求可能涉及多层嵌套、多表关联和复杂聚合等高难度 SQL 结构。单一的生成路径在处理这种高度异构的查询分布时存在固有局限：对于简单查询，过度复杂的推理策略会引入冗余步骤并增加出错风险；对于复杂查询，直接映射式的生成方式又难以捕获深层的逻辑关系^[29]。

为此本文设计了一种异构协同生成框架，包含旨在捕获高频业务逻辑的逻辑映射生成器以及处理复杂推理深度的渐进分治生成器。通过并行驱动两个推理范式截然不同的生成器，构建多样化的 SQL 候选池，以提升系统对各种复杂度查询的覆盖能力。生成阶段允许大语言模型在非零温度^[57]设置下执行随机采样，每个生成器并行生成 n_c 个候选 SQL 查询，最终总计产生 $2n_c$ 个多样性的 SQL 候选集。

3.5.1 逻辑映射生成器

逻辑映射生成器深度集成了本章提出的 LA-Schema 架构，其推理机制建立在领域逻辑映射的基础之上。该生成器通过引导大语言模型对 LA-Schema 中的领域先验进行识别与调用，其包括业务术语的语义对齐、虚拟逻辑列的公式引用以及值域约束的条件匹配。以有效地将复杂的 SQL 构造过程从底层的算子推导转化为对预设逻辑块的检索与组装。其完整流程如算法3.2所示。

算法 3.2: 逻辑映射生成算法

Input: 用户查询 Q , LA-Schema 增强模式 S^* , 动态样例 $F_u = \{f_1, f_2, \dots, f_k\}$, 采样数 n_c , 大语言模型 LLM

Output: 候选 SQL 集合 C_{lm}

- 1 初始化: 候选集合 $C_{lm} \leftarrow \emptyset$, 提示 $P \leftarrow \emptyset$;
// 构建提示模板
- 2 $P \leftarrow P \oplus \text{FormatFewShot}(F_u)$; // 拼接动态少样本示例
- 3 $P \leftarrow P \oplus S^*$; // 拼接 LA-Schema 增强模式
- 4 $R \leftarrow \text{BuildRules}()$; // 构建映射规则指令
- 5 $P \leftarrow P \oplus R \oplus Q$; // 拼接规则指令与用户查询
- 6 // 多次采样生成候选 SQL
- 6 **for** $i \leftarrow 1$ **to** n_c **do**
- 7 | $\hat{y}_i \leftarrow \text{LLM}(P, \text{temperature} > 0)$; // 随机采样生成 SQL
- 8 | $C_{lm} \leftarrow C_{lm} \cup \{\hat{y}_i\}$;
- 9 **end**
- 10 **return** C_{lm}

算法中，BuildRules() 构建的映射规则指令明确要求模型遵循以下约束：（1）

优先使用 LA-Schema 中标记为 Virtual Column 的虚拟逻辑列，直接在 SQL 中调用其预定义的计算公式；（2）识别并转换 Business Term 标记的业务术语，使用映射后的物理字段替换原始表达；（3）利用 Value Term 标记的匹配列值，确保 WHERE 条件中使用与数据库一致的标准取值；（4）直接输出最终的可执行 SQL，不作过多中间解释。生成阶段在非零温度下对同一提示执行 n_c 次独立采样，以获得多样化的候选 SQL 集合 $\mathcal{C}_{lm}$ 。

该生成器的核心优势在于效率高、对常见查询模式适应性强。对于政务场景中高频出现的指标查询、条件筛选和简单聚合等标准需求，逻辑映射生成器能够充分利用 LA-Schema 中预置的领域逻辑，快速而准确地完成 SQL 生成，避免了不必要的推理链条展开。

3.5.2 渐进分治生成器

与逻辑映射生成器的直接映射策略不同，渐进分治生成器采用渐进式生成与动态少样本的联合方式，以应对知识库未覆盖的复杂需求，如多层嵌套逻辑、多条件组合聚合以及跨表关联查询等。

渐进式生成策略本质上是一种面向 Text-to-SQL 任务定制化的思维链方法^[64]，其核心在于将复杂的 SQL 生成任务拆解为多维度、结构化的子任务序列，引导大语言模型通过逐步推理深度理解查询逻辑。该策略借鉴了 CHASE-SQL^[29]中的分治思想和 OpenSearch-SQL^[31]中的 SQL-Like 中间表示设计，但针对政务场景做了两方面适配：一是在推理链条中保留了 LA-Schema 的逻辑增强信息，使分步推理能够参考领域知识；二是引入了类 SQL 中间表示，允许模型先构建查询骨架再填充语法细节，降低了一次性生成完整 SQL 的复杂度。其完整流程如算法3.3所示。

算法中，渐进推理指令 BuildCoTInstr() 要求模型在单次生成中按照表3-1所定义的结构化格式，以自回归方式依次输出意图分析 (reason)、涉及列 (columns)、筛选值 (values)、SELECT 内容以及 SQL-like 骨架共五个部分，前序输出自然成为后续生成的上下文，形成递进式的推理链。随后，系统将生成的 SQL-like 骨架作为上下文，再次调用大语言模型补充完整的 JOIN 条件和语法细节，生成最终可执行的 SQL 查询。动态少样本示例以完整的 (Query, CoT, SQL) 三元组呈现，为模型提供渐进推理的范式参考。生成阶段同样在非零温度下执行 n_c 次独立采样，产生多样化的候选集合 $\mathcal{C}_{pd}$ 。

该生成器的核心优势在于对复杂查询的处理能力。当查询涉及多层嵌套逻辑、复杂的条件组合或跨表关联时，分步推理机制能够引导模型逐步构建查询结构，避免因一步到位生成长 SQL 而导致的逻辑遗漏或结构错误。同时，渐进生成器输出

算法 3.3: 渐进分治生成算法

Input: 用户查询 Q , LA-Schema 增强模式 S^* , 动态样例 $F_u = \{f_1, \dots, f_k\}$, 采样数 n_c , 大语言模型 LLM

Output: 候选 SQL 集合 $\mathcal{C}_{pd}$

```

1 初始化: 候选集合  $\mathcal{C}_{pd} \leftarrow \emptyset$ , 提示  $P \leftarrow \emptyset$ ;
   // 构建提示模板
2  $P \leftarrow P \oplus \text{FormatCoTShot}(F_u)$ ;           // 拼接  $\langle \text{Query}, \text{CoT}, \text{SQL} \rangle$  格式示例
3  $P \leftarrow P \oplus S^*$ ;                           // 拼接 LA-Schema 增强模式
4  $P \leftarrow P \oplus \text{BuildCoTInstr}() \oplus Q$ ;      // 拼接渐进推理指令与用户查询
   // 多次采样生成候选 SQL
5 for  $i \leftarrow 1$  to  $n_c$  do
   // 意图分析  $\rightarrow$  涉及列与值  $\rightarrow \text{SELECT} \rightarrow \text{SQL-like}$ 
6    $(\text{reason}_i, \text{cols}_i, \text{vals}_i, \text{sel}_i, \hat{y}'_i) \leftarrow \text{LLM}(P, \text{temperature} > 0)$ ;
7    $\hat{y}_i \leftarrow \text{LLM}(P, \hat{y}'_i)$ ;                 // 基于 SQL-like 生成最终 SQL
8    $\mathcal{C}_{pd} \leftarrow \mathcal{C}_{pd} \cup \{\hat{y}_i\}$ ;
9 end
10 return  $\mathcal{C}_{pd}$ 

```

的结构化推理过程本身也具有可解释性, 有助于后续的错误分析和结果追溯。

此外, 渐进分治生成器与动态样例库之间存在正向反馈关系。当该生成器产生的候选 SQL 经判别后被确认为正确答案时, 其完整的 $\langle \text{Query}, \text{CoT}, \text{SQL} \rangle$ 输出可作为新的高质量样例纳入动态样例库, 为后续相似问题的处理提供更精准的参考。

3.6 基于执行反馈的查询修复

无论是逻辑映射生成器还是渐进分治生成器, 其产生的 SQL 候选集在某些情况下不可避免地存在逻辑或语法错误^[29,34], 这些错误会导致查询无法正确执行或返回空结果。为提升候选集的整体质量, 本文引入了一个基于执行反馈的查询修复模块, 作为候选生成后、判别选择前的鲁棒性增强环节。

在实际政务查询场景中, 常见的 SQL 错误类型包括: (1) 字段名不存在或拼写错误, 通常由模式链接阶段的遗漏或大语言模型的幻觉引起; (2) 表连接关系错误, 如遗漏必要的 JOIN 条件或使用了错误的连接键; (3) SQL 语法错误, 如括号不匹配、关键字使用不当等; (4) 聚合逻辑错误, 如 GROUP BY 子句缺失必要字段或聚合函数使用不当。

查询修复器的工作机制基于自反思 (Self-Reflection) 方法^[82]。整个过程如图3-3所示。对于候选集中的每条 SQL 查询, 系统首先在目标数据库上执行该查询, 捕获执行结果或错误信息。若执行成功则保留该候选; 若执行失败 (如语法错误), 则将执行器的反馈信息包括具体的错误描述以及原始用户问题、LA-Schema 信息

一并构造为修复提示，送入大语言模型进行反思与修复。修复后的 SQL 会再次执行以验证其有效性。

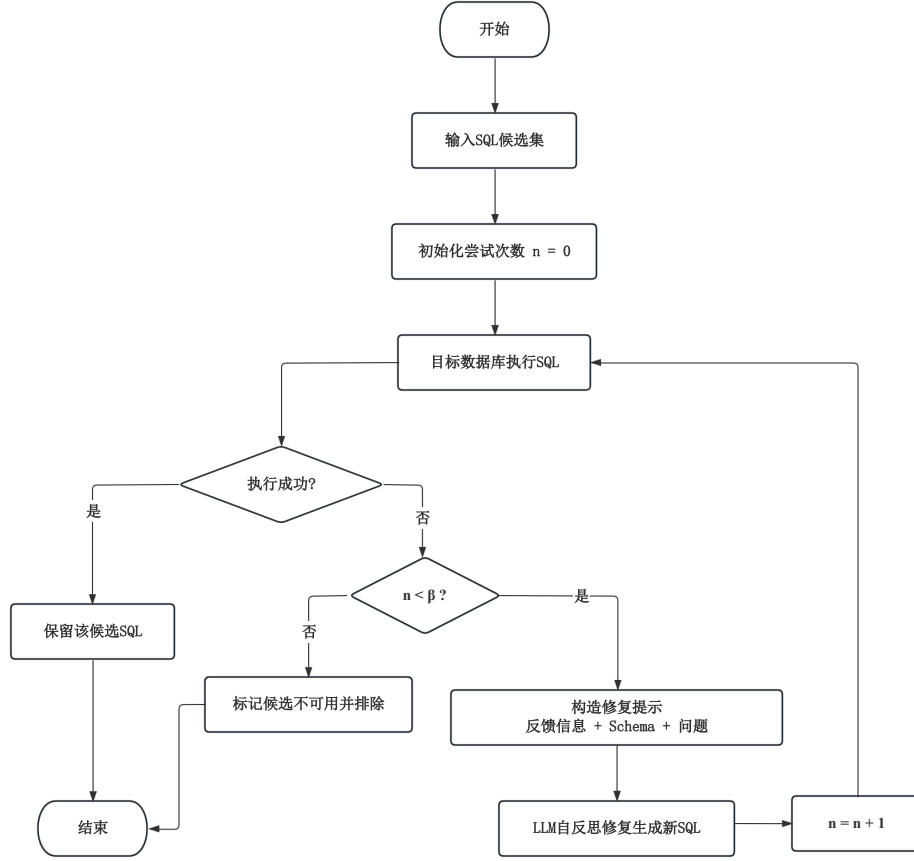


图 3-3 查询修复流程图

为避免无限迭代带来的效率损耗和不收敛风险，修复过程设置了最大尝试次数 β （本文设定 $\beta = 3$ ）。当满足以下任一条件时，修复过程终止：（1）修复后的 SQL 执行成功；（2）达到最大尝试次数 β 。若修复在 β 次尝试后仍未成功，则保留最后一次修复的结果或标记该候选为不可用，在后续判别阶段予以排除。

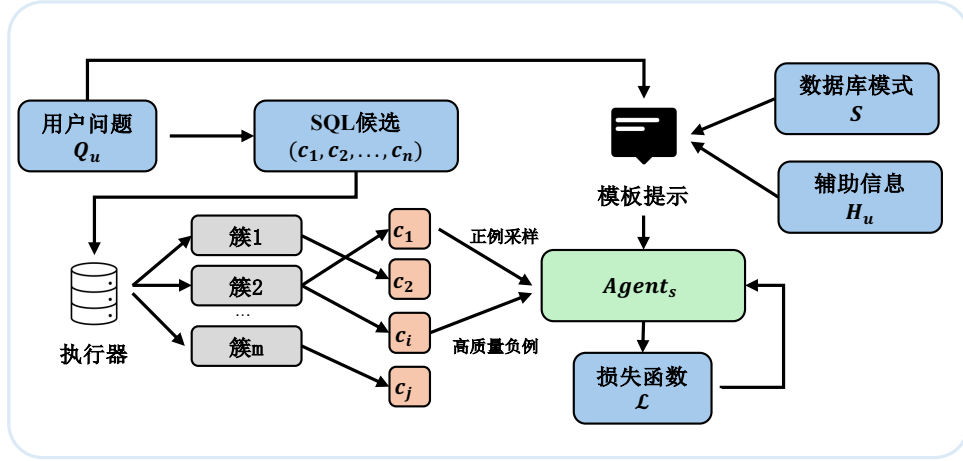
查询修复器的引入有效降低了候选集中的无效查询比例，为后续的候选判别模块提供了更高质量的输入，从而间接提升了最终查询结果的准确率。

3.7 候选判别模型

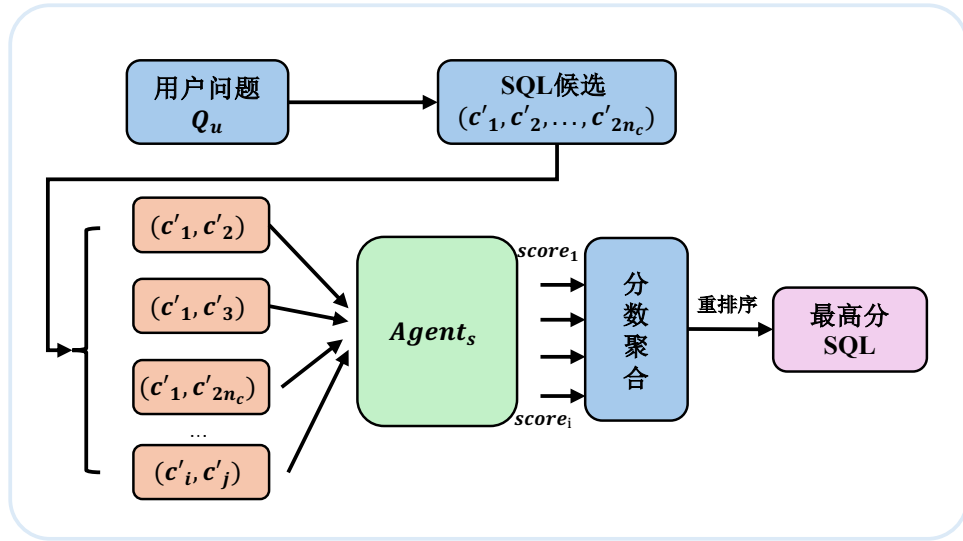
经过异构候选生成与查询修复后，系统可为每个用户问题构建包含 $2n_c$ 个候选 SQL 的查询池。本节的目标是从该候选集中可靠地选出最终正确的 SQL 查询。

现有许多 Text-to-SQL 方法采用候选查询的一致性投票（Self-Consistency）策

略，以得票最多的结果作为最终输出^[31]。然而，在复杂业务查询场景中，最一致的答案并不总是正确答案^[29]，其一致性选择策略与上界精度存在一定的性能差距。为此，本文构建选择智能体 $Agent_s$ ，并将候选选择建模为配对偏好判别问题。具体而言， $Agent_s$ 以用户问题、辅助提示信息、相关增强模式以及一对候选 SQL 为输入，判断两者中哪一个更可能正确，再通过两两比较与得分聚合确定最终输出。整体流程如图3-4所示。



(a) 离线训练



(b) 在线推理

图 3-4 候选判别整体流程

3.7.1 训练样本构建

判别模型的训练数据来源于在训练集上运行候选生成模块所得到的候选 SQL 集合。对于训练集中的每个问题 Q_u ，系统首先利用两个异构生成器产生 $2n_c$ 个候

选 SQL，并在目标数据库上执行所有候选查询。设标准 SQL 的执行结果为 R_u^* ，候选 c_k 的执行结果为 R_k 。系统依据执行结果对候选进行聚类，将执行结果相同的候选划分至同一簇，再将各簇对应的执行结果与 R_u^* 进行比较：若二者一致，则该簇记为正确簇；否则记为错误簇。

在同时存在正确簇与错误簇的情况下，系统从两类簇中抽取候选，构造配对训练样本：

$$T_{ij} = (Q_u, H_u, c_i, c_j, S_{ij}^*, y_{ij}), \quad (3-8)$$

其中， Q_u 表示用户问题， H_u 表示辅助提示信息， c_i 与 c_j 表示两个待比较的候选 SQL， S_{ij}^* 表示 c_i 和 c_j 所涉及增强数据库模式的并集， $y_{ij} \in \{0, 1\}$ 为监督标签。当 c_i 来自正确簇且 c_j 来自错误簇时，记 $y_{ij} = 1$ ；反之记 $y_{ij} = 0$ 。

在样本构建过程中重点挖掘高价值负例。所谓高价值负例，是指执行结果与正确答案接近、但查询逻辑存在关键偏差的错误 SQL，例如筛选条件边界不同、聚合范围不一致或连接关系错误等。相较于随机负例，这类样本更能反映真实业务场景中的判别难点。对于训练集中未能生成正确候选的问题，本文仅在离线样本构造阶段引入标准 SQL 的执行结果，用于引导生成器产生更多与正确答案相似但不完全相同的错误候选，从而提升高价值负例的比例；该信息不作为判别模型在线推理时的输入，以避免信息泄漏。

3.7.2 判别模型训练

系统以 Qwen3-8B^[8]为基础模型构建选择智能体 Agent_s ，并采用指令式监督微调方式^[62](Instruction Supervised Fine-Tuning, ISFT) 进行训练。对于每个训练样本 T_{ij} ，系统将用户问题 Q_u 、辅助提示信息 H_u 、模式并集 S_{ij}^* 以及候选 SQL(c_i, c_j) 按统一模板组织为输入序列，要求模型输出二元判别结果，以表示两者中哪一个更可能为正确查询。由此， Agent_s 学习的是如下条件偏好概率：

$$p_\theta(y_{ij} = 1 \mid Q_u, H_u, S_{ij}^*, c_i, c_j), \quad (3-9)$$

其中 θ 表示大模型经微调后的模型参数。

训练目标采用二分类交叉熵损失函数：

$$\mathcal{L}_{\text{pair}} = -y_{ij} \log p_\theta(y_{ij} = 1 \mid T_{ij}) - (1 - y_{ij}) \log p_\theta(y_{ij} = 0 \mid T_{ij}). \quad (3-10)$$

考虑到高价值负例对模型判别能力的提升更为显著，本文进一步引入样本权重 w_{ij} ，

得到总体优化目标：

$$\mathcal{L} = \sum_{(i,j)} w_{ij} \mathcal{L}_{\text{pair}}. \quad (3-11)$$

其中，对语义接近但逻辑错误的高价值负例赋予更高权重以增强模型对细微差异的敏感性。在样本构建过程中，对每个训练问题的正确簇与错误簇分别进行采样，控制正负样本对的比例约为 1 : 1，以避免类别不平衡对训练过程的干扰。

为兼顾训练效率与模型性能，本文采用 LoRA (Low-Rank Adaptation)^[83] 参数高效微调策略对大模型进行训练。LoRA 通过在预训练权重矩阵旁注入可训练的低秩分解矩阵 $\Delta W = AB$ (其中 $A \in \mathbb{R}^{d \times r}$, $B \in \mathbb{R}^{r \times d}$, $r \ll d$)，仅更新少量新增参数而冻结原始模型权重，从而在大幅减少可训练参数量的同时保留基础模型的言语理解能力。本文对模型自注意力层中的查询投影矩阵 W_q 与值投影矩阵 W_v 注入 LoRA 适配器，该配置下可训练参数量约占模型总参数的 0.24%，在保证判别能力的同时显著降低了显存占用与训练开销。LoRA 的详细超参数设置见表3-3。

表 3-3 LoRA 微调超参数配置

超参数	设置
目标模块	W_q, W_v
秩 r	16
缩放因子 α	32
LoRA Dropout	0.05
可训练参数占比	$\approx 0.24\%$

3.7.3 配对判别与最优候选选择

在线推理阶段,系统基于配对比较与得分聚合机制,从候选集 $C = \{c_1, c_2, \dots, c_{2n_c}\}$ 中选出最终 SQL，其完整流程如算法3.4所示。

对于任意两个不同候选 (c_i, c_j) ，系统首先在目标数据库 DB 上执行二者并比较执行结果。若 $\text{ans}(c_i, DB) = \text{ans}(c_j, DB)$ ，则说明二者在功能上等价，无需调用判别模型；若二者执行结果不同，则构造其相关增强模式并集 $S_{ij}^* = \text{SchemaUnion}(c_i, c_j, S^*)$ ，并调用 Agent_s 进行二元判别。考虑到输入顺序可能影响模型输出，系统对每一对候选同时比较 (c_i, c_j) 与 (c_j, c_i) 两个方向，并将胜者分别累加得分，从而减轻顺序偏差。最终，系统选择累计得分最高的候选作为输出：

$$c_f = \arg \max_{c_i \in C} \text{score}_i. \quad (3-12)$$

若出现平局，系统优先选择执行成功且返回非空结果的候选；若仍无法区分，则进

算法 3.4: SQL 候选集选取算法

Input: 候选集 $C = \{c_1, c_2, \dots, c_{2n_c}\}$, 用户问题 Q_u , 目标数据库 DB , 增强数据库模式 S^* , 选择智能体 Agent_s

Output: 最佳 SQL 查询 c_f

```

1 foreach  $c_i \in C$  do
2   |  $\text{score}_i \leftarrow 0$ ;
3 end
4 foreach  $(c_i, c_j) \in C \times C$  且  $i \neq j$  do
5   | if  $\text{ans}(c_i, DB) = \text{ans}(c_j, DB)$  then
6     |  $\text{winner} \leftarrow i$ ; // 结果一致时, 无需调用判别模型
7   | else
8     |  $S_{ij}^* \leftarrow \text{SchemaUnion}(c_i, c_j, S^*)$ ;
9     |  $\text{winner} \leftarrow \text{Agent}_s(Q_u, S_{ij}^*, c_i, c_j)$ ;
10  | end
11  |  $\text{score}_{\text{winner}} \leftarrow \text{score}_{\text{winner}} + 1$ ;
12 end
13  $c_f \leftarrow \arg \max_{c_i \in C} \text{score}_i$ ;
14 return  $c_f$ 

```

一步选择 SQL 语句长度较短的候选作为最终结果。通过上述机制, 本文将候选选择由一致性投票转化为基于配对偏好的全局竞争过程, 从而更有效地利用候选间的细粒度差异信息, 提高最终 SQL 的选取准确率。

3.8 实验与结果分析

本章首先介绍实验设置, 包含数据集构建与选用、评估指标定义、对比模型。然后针对政务领域数据集上进行对比实验、消融实验、异构协同与判别分析以及效率分析。最后本文还在公共数据集 BIRD^[18]、Spider^[17]上进行实验, 以评估本文方法在泛用数据集上的适用性与稳健型。

3.8.1 数据集构建与实验设置

为全面评估本文方法在政务领域的有效性, 实验采用自建政务数据集进行评估。鉴于政务数据的隐私敏感性, 本文基于“多源采集—逻辑注入—混合标注”的流程构建了面向政务场景的 Text-to-SQL 评测数据集。

数据集的构建流程分为三个阶段:

(1) **多源数据采集与脱敏。**数据来源于某市经济开发区政务平台, 原始数据涵盖财政、公共资源和政务项目三个领域的 13 个数据库, 共计 92 张数据表。为保障数据安全, 对涉及个人隐私的敏感信息 (如身份证号、联系方式等) 进行了加盐哈

希的严格脱敏处理，确保数据在技术上不可逆向还原。

(2) **基于领域知识库的逻辑注入**。通过人工梳理 40 余个核心业务指标，将包括预算执行率、同比增长率、集中采购率等在内的隐性业务逻辑显性化地录入标注元数据中。确保数据集包含足够多的需要领域知识。

(3) **人机协同混合标注**。问答对生成采用人机协同的方式：首先利用 Qwen 大语言模型^[8]根据原始数据库模式生成种子问题和初始 SQL，然后由具备政务业务背景的标注人员对生成结果进行核验和纠正，确保问题的自然性和 SQL 的正确性。

表 3-4 政务数据集 GovSQL 统计信息

划分	领域数	数据库数	数据表数	字段数	问题-SQL 对数	复杂查询占比/%
训练集	3	13	92	563	958	37.8
测试集	3	13	92	563	410	40.2
合计	3	13	92	563	1368	38.5

最终构建的数据集包含 1368 条高质量自然语言问题-SQL 对，按 7:3 划分为训练集和测试集。训练集用于候选判别模型微调，并在训练样本构建阶段产生近 14000 个元组；测试集用于端到端性能评估。数据集中约 38.5% 的问题涉及多表连接、嵌套子查询和复杂聚合等复杂查询，统计信息如表3-4所示。

表 3-5 候选判别模型训练配置

配置项	设置
基础模型	Qwen3-8B
微调方式	LoRA（详见表3-3）
训练框架	PyTorch + PEFT
硬件环境	2 × NVIDIA A100 80GB
优化器	AdamW ^[84]
初始学习率	2×10^{-5}
权重衰减	0.01
学习率调度	余弦退火（Cosine Annealing）
预热比例	总步数的 3%
单卡批大小	4
梯度累积步数	8
等效全局批大小	64
最大序列长度	4096 Token
训练轮次	3 epoch
训练样本数	≈ 14000 对
总训练步数	≈ 660 步

在实验配置方面，本文使用 Qwen3-Coder-30B-A3B-Instruct^[8]作为基础调用大语言模型，用于模式链接、候选生成和查询修复等主要环节；使用 Qwen3-8B^[8]作为

候选判别模型的基础模型进行 LoRA 微调训练。判别模型的完整训练配置如表3-5所示。为保证实验的可复现性。

存储引擎使用 Milvus 作为向量库构建存储, PostgreSQL 作为领域知识库, 并联合使用 Milvus 和 PostgreSQL 实现动态样例库。在工程上使用 Redis 作为热点缓存工具, 提高调用性能。

实现设置上, 动态样例检索的样本数 n_f 设为 5, 此外样例库实验评估阶段关闭在线增量更新, 避免测试数据泄漏。模式链接阶段, 候选列的召回数 k 设为 10。除候选生成外的其他 LLM 调用阶段温度参数 $temperature = 0.3$ 。而在候选生成阶段, 每个生成器在温度参数 $temperature = 0.7$ 的设置下各采样生成 $n_c = 10$ 个候选 SQL。

3.8.2 评估指标与对比基线

本文采用执行准确率 (Execution Accuracy, EX) 作为主要评估指标。EX 通过比较预测 SQL 查询与参考 SQL 查询在目标数据库上的执行结果来判定正确性:

$$EX = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[\text{ans}(SQL_i^{\text{pred}}, DB) = \text{ans}(SQL_i^{\text{gold}}, DB)] \quad (3-13)$$

其中 N 为测试集样本数, $\text{ans}(SQL, DB)$ 表示 SQL 在数据库 DB 上的执行结果。EX 指标关注的是查询的功能等价性而非形式一致性, 即允许 SQL 在写法上存在差异, 只要执行结果相同即视为正确。

此外, 本文在异构协同与候选判别分析和效率分析中还使用了以下辅助指标: 判别模型的二分类准确率以及端到端平均响应时延。

对比基线分为以下三类:

(1) **开源基础大模型**。该类基线采用零样本 (zero-shot) Text-to-SQL 提示直接生成 SQL, 不引入任何额外的优化策略, 旨在衡量大语言模型本身的 SQL 生成能力下限。具体包括: DeepSeek-V3^[9], 本地部署, 采用零样本提示生成 SQL; Qwen3-Coder-30B-A3B-Instruct^[8], 本地部署, 同样采用零样本提示生成 SQL。

(2) **基于提示词的无微调方法**。该方法通过精心设计的提示工程策略提升 SQL 生成质量, 无需对模型参数进行微调。具体包括: DAIL-SQL^[27], 通过为不同问题检索语义相似的问题-SQL 对作为少样本示例, 引导大语言模型生成 SQL; OpenSearch-SQL^[31], 通过动态上下文学习与一致性对齐策略, 以多智能体协同的方式生成 SQL; Alpha-SQL^[32], 基于蒙特卡洛树搜索建模 Text-to-SQL 生成过程, 配合动态生成推理动作与自监督奖励函数进行搜索式 SQL 生成。

(3) **基于监督微调的方法**。该类方法通过在 Text-to-SQL 数据上对模型进行监督微调,使模型习得面向 SQL 生成的专用能力。具体包括: XiYan-SQL^[30], 融合监督微调与上下文学习,并使用 M-Schema 增强模式理解,以多生成器集成的方式生成 SQL; CHASE-SQL^[29], 通过多路径候选生成器产生多样化的 SQL 候选,并使用经偏好优化微调的选择代理进行候选判别; OmniSQL^[33], 通过自动化合成百万级高质量 Text-to-SQL 数据及思维链标注,基于开源大模型进行大规模监督微调。

为保证对比公平性,上述基线方法(除开源基础大模型和 OmniSQL 外)均使用与本文相同的 Qwen3-Coder-30B-A3B-Instruct 作为基础 LLM,实验使用的数据库与知识库也保持一致。

3.8.3 总体性能对比

表3-6展示了各方法在政务数据集上的执行准确率对比结果。本文方法取得了 91.47% 的执行准确率,显著优于所有对比基线。

表 3-6 不同 Text-to-SQL 方法在政务数据集上的性能对比(± 表示 95% 置信区间半宽)

类别	方法	执行准确率/%
开源基础大模型	DeepSeek-V3	64.90 ± 4.62
	Qwen3-Coder-30B-A3B-Instruct	69.17 ± 4.47
基于提示词无微调	DAIL-SQL	74.42 ± 4.22
	OpenSearch-SQL	76.51 ± 4.10
	Alpha-SQL	80.27 ± 3.85
基于监督微调	OmniSQL	71.56 ± 4.37
	CHASE-SQL	81.33 ± 3.77
	XiYan-SQL	83.83 ± 3.56
本文方法	Ours	91.47 ± 2.70

从结果中可以得出以下分析:

开源大模型的零样本方法其准确率普遍处于 65% 至 70% 之间。说明仅依赖基础模型的 SQL 生成能力,难以正确处理政务场景中晦涩的物理列名和复杂业务指标,容易出现列名臆造或计算逻辑错误。

基于提示词工程的方法通过优化上下文样本选择取得了一定提升,其中 Alpha-SQL 达到了 80.27%。但这类方法本质上仍属于隐式推理范式,模型必须在有限的上下文窗口内同时完成语义理解和逻辑推演。在处理高嵌套、跨表计算等复杂政务查询时,上下文中的有效信息容易被稀释,推理链条断裂的概率显著增加。

基于监督微调的 XiYan-SQL 和 CHASE-SQL 表现进一步提升,分别达到了 83.83% 和 81.33%。这表明监督微调在特定领域数据集上确实能带来增益效果。但

即便经过微调，如果模型不具备实时的领域逻辑增强支撑，在面对政务业务中涉及复杂计算公式和动态变化的业务规则时，仍然难以突破 85% 的准确率瓶颈。

相比之下，本文方法较最强基线 XiYan-SQL 提升 7.64 个百分点，较 CHASE-SQL 提升 10.14 个百分点。这一显著改善归因于三方面的协同作用：LA-Schema 的逻辑增强机制有效缓解了政务领域的语义幻觉问题；异构协同生成策略扩大了候选池的多样性和覆盖度；微调判别模型则从多样化的候选中精准筛选出最优答案。

3.8.4 消融实验

为深入探究系统中各核心模块的有效性，本文设计了消融实验。通过在完整模型（Full Model）的基础上分别移除或替换关键模块，观察其在政务数据集上的执行准确率变化。实验结果如表3-7所示，移除任何一个模块均会导致性能下降，证明了各模块的不可或缺性。

表 3-7 移除系统关键模块的消融实验性能比较

模型方法	执行准确率/%	ΔEX /%
Full Model	91.47	-
w/o HNSW	89.32	-2.15
w/o Dynamic Few-shot	85.31	-6.16
w/o LA-Schema	84.62	-6.85
w/o Logical Generator	87.93	-3.54
w/o DC Generator	88.88	-2.59
w/o Query Fixer	89.24	-2.23
w/o Selection Agent	86.12	-5.35

w/o HNSW。在数据预处理阶段，若不使用 HNSW 索引进行向量值检索，而直接采用语义匹配的单阶段值检索，执行准确率降至 89.32%。这证明了 HNSW 索引在处理大规模政务元数据时，能够更高效且精准地召回与查询相关的数据库值。

w/o Dynamic Few-shot。移除动态样例库后，模型仅依赖固定的提示词模板，准确率下降至 85.31%。这表明政务查询具有较强领域多样性，动态检索语义相似的少样本样例，增强系统对特定业务语境的自适应能力。尤其是 $\langle \text{Query}, \text{CoT}, \text{SQL} \rangle$ 格式的样例比传统的 $\langle \text{Query}, \text{SQL} \rangle$ 格式能够提供更丰富的推理线索。

w/o LA-Schema。采用传统 DDL Schema 替代逻辑增强模式后，性能降幅最大，达到 6.85%。该配置的准确率（84.62%）已接近最强基线 XiYan-SQL（83.83%），说明在政务垂直领域，物理字段名的隐晦性和业务逻辑的复杂性是 SQL 生成的核心瓶颈，而 LA-Schema 通过显式注入业务等先验信息，将复杂的逻辑演算简化为模式调用，是系统高性能的核心保障。

w/o Logical Generator & w/o DC Generator。分别去除逻辑映射生成器和渐进分治生成器后，执行准确率分别下降 3.54% 和 2.59%。逻辑映射生成器性能略大于渐进分治生成器，说明政务数据集中包含较多需要领域逻辑映射的业务指标查询。

w/o Query Fixer。去除查询修复器后，准确率下降 2.23% 至 89.24%。虽然修复器的单独贡献相对较小，但其通过基于执行反馈的迭代修正机制，有效提升了候选集中可执行查询的比例，为后续判别模型提供了更高质量的输入。

w/o Selection Agent。若不使用基于执行反馈的选择智能体进行候选判别，而是采用一致性多数投票策略，准确率下降 5.35% 至 86.12%。这说明多数投票无法充分感知 SQL 的执行质量，而选择智能体能够通过执行反馈识别出语义虽不一致但逻辑正确的潜在答案，显著提升了异构候选冲突时的容错率。

3.8.5 异构协同与候选判别分析

为深入剖析异构协同生成机制与候选判别模型这一核心创新的有效性，从生成器独立性能、判别模型精度、候选池多样性三个维度设计了系统性的专项实验。

生成器性能分析

为揭示各生成器的独立贡献，本文在候选判别前对每个生成器的单候选生成能力进行了评估。即将每个生成器生成单条候选 SQL 的准确率与零样本 CoT 基线进行对比，同时考察查询修复器对各生成器的增益效果。结果如图3-5所示。

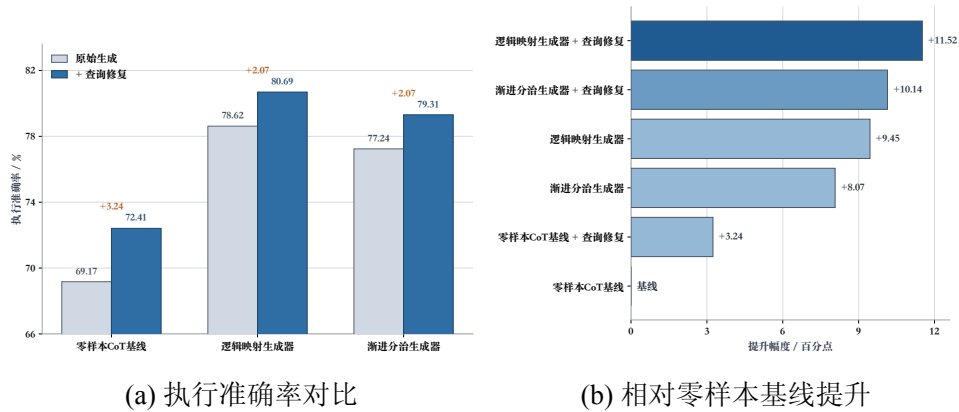


图 3-5 各生成器单候选生成性能对比

从结果中可以得出以下分析。首先，两个生成器的单候选性能均显著优于零样本 CoT 基线，逻辑映射生成器提升了 9.45 个百分点，渐进分治生成器提升了 8.07 个百分点，验证了本文所设计的生成策略在引导大语言模型进行 SQL 生成方面的有效性。其中逻辑映射生成器略优于渐进分治生成器，这主要归因于政务数据集中包含大量可直接利用 LA-Schema 预置逻辑块的业务指标查询。其次，查询修复

器对所有生成方法均带来了约 2%~3% 的性能增益，表明基于执行反馈的迭代修复机制能够有效纠正语法错误和执行异常，提升候选的整体可用性。

判别模型分析

候选判别模型的二分类准确率直接决定了从候选池中选出正确答案的能力。本文对不同选择策略的判别精度进行了对比实验，在配对比较中仅统计一个候选正确、另一个候选错误的高价值情况。结果如表3-8所示。

表 3-8 不同选择策略的二分类判别准确率

选择模型	二分类准确率/%
Qwen3-Coder-30B（未微调）	59.38
Qwen3-8B（未微调）	55.62
Qwen3-8B（微调，本文方法）	72.85

实验结果表明，未经微调的大语言模型在配对判别任务上的准确率仅为 55%~59%，接近随机猜测水平。这说明异构生成器产生的候选 SQL 虽然往往只存在局部差异，但这些差异恰恰决定了业务正确性，未微调模型难以稳定识别。经过高价值正负样本对微调后，Qwen3-8B 的判别准确率提升至 72.85%，比未微调版本高出 17.23 个百分点。

尽管 72.85% 仍意味着存在误判，但这种误判不会等比例传导为端到端错误：在线阶段仅在两个候选执行结果不一致时才调用判别模型；对于执行结果一致的候选，系统直接视为功能等价。此外，本文采用双向配对比较和全局得分聚合，而非单次判别即决定输出，因此局部误判会被部分稀释。这也解释了为何选择智能体相较于一致性多数投票仍能带来 5.35 个百分点的端到端提升。

从误判类型看，判别模型更容易在三类问题上出错。第一类是**筛选边界或时间边界仅有细微差异**，例如“>”与“≥”的区别、统计区间起止月份偏移等；这类候选词项和整体 SQL 骨架上高度相似，但会导致结果集出现局部偏差。第二类是**统计口径或聚合范围不一致**，例如同比与环比或同一指标在汇总层级上存在全量统计与分组统计的差别；此时错误候选往往同样可执行，但模型若仅依据表层语义相似性进行判断，容易忽略隐藏的业务口径差异。第三类是**多表关联下的近似正确候选**，如 JOIN 键选择错误或聚合前后连接顺序不同。这类候选通常语法正确、执行成功，甚至在部分样本上返回与正确答案接近的数值，因此最容易构成高价值负例。所以判别模型的主要难点并非区分“明显错误”和“明显正确”，而是识别那些在表层结构上相近、在业务语义上却存在关键偏差的候选 SQL。

候选池多样性与生成器互补性分析

本文对两个生成器在候选池层面的互补性进行了深入考察。对于测试集中的每个问题，分别统计仅逻辑映射生成器产生正确候选、仅渐进分治生成器产生正确候选以及两者均产生正确候选的情况。如图3-6所示。

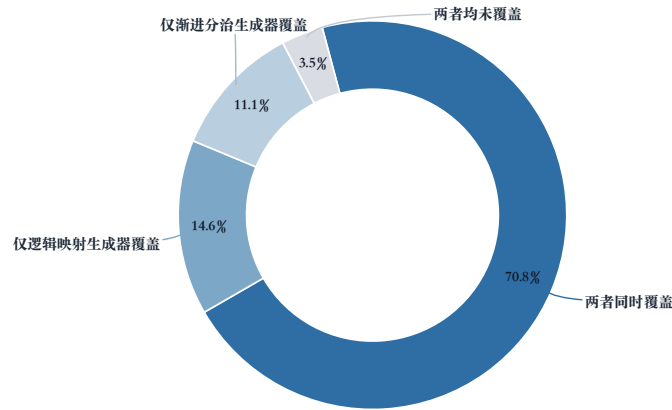


图 3-6 生成器在测试集覆盖统计

在测试样本中，有 14.6% 的问题仅被逻辑映射生成器覆盖，11.1% 仅被渐进分治生成器覆盖，70.8% 被两个生成器同时覆盖，仅有 3.5% 未被任一生成器产生的候选所覆盖。逻辑映射生成器独占的正确案例以包含预定义业务指标的标准化查询为主，涉及需要精确公式调用的问题。渐进分治生成器独占的正确案例则集中在结构复杂的查询上，包括多层嵌套子查询、跨表关联聚合等场景。两种推理范式在查询类型上的互补性是异构协同策略有效的根本原因。

3.8.6 效率分析

表3-9展示了本文方法各模块的平均耗时、时间占比、LLM 调用次数及 Token 消耗情况。

表 3-9 各模块效率分析（平均单样本）

模块	平均耗时/s	占比/%	平均 LLM 调用/次	Token 消耗/个
模式链接	7.8	19.5	3	3000–8000
候选生成（双路并行）	17.1	42.6	30	45000–120000
查询修复	3.8	9.5	0–3	0–6000
候选判别	10.7	26.7	1–15	500–9000
其他（SQL 执行等）	0.7	1.7	0	0
端到端总计	40.1	100.0	34–51	48500–143000

端到端单条查询需 34~51 次 LLM 调用，Token 消耗为 45000~120000 个，具体取决于候选修复轮次和生成器采样多样性。候选生成是最耗时环节，占 42.6%，主要因为两个生成器都需要多次采样并处理较长提示；候选判别次之，占 26.7%，其耗时波动主要来自候选对之间是否需要逐对调用选择智能体。相比之下，模式链接和查询修复的开销相对可控。

对于政务数据查询这一非实时交互的应用场景而言，40 秒左右的响应时间处于可接受范围内。在实际部署中，候选生成的并行化策略（两个生成器同时运行且多线程生成候选 SQL，即 30 次调用耗时约等于 3 次调用）已将该环节的耗时控制在合理水平；同时 SQL 执行结果的缓存大大提高了使用性能。若需进一步降低延迟，可考虑减少每个生成器的采样数量（如降低至 5 或者 3），以在准确率与响应速度之间取得更优的折中。

3.8.7 公共数据集实验结果

BIRD 和 Spider 是两个广受认可的跨域数据集，分有不重叠的训练集、开发集、测试集。BIRD 包含来自 95 个大型数据库的超过 12,751 个独特的问题-SQL 对，涵盖 37 个专业领域，其数据库设计模仿现实世界场景，具有杂乱的数据行和复杂的模式。Spider 包含 10,181 个问题和 5,693 个独特的复杂 SQL 查询，涉及 200 个数据库，覆盖 138 个领域。

为评估本文方法在泛用场景下的适用性与稳健性，本文在两个主流的跨领域公共 Text-to-SQL 基准数据集上进行了实验：BIRD^[18]开发集（BIRD-dev）和 Spider^[17]测试集（Spider-test）。在公共数据集实验中，本文方法未使用政务领域知识库，动态样例库也替换为基于目标数据集训练集构建的通用样例库，以确保评测条件与其他基线方法保持一致。各方法的执行准确率对比结果如表3-10所示。

表 3-10 公共数据集上的性能对比

方法	使用模型	BIRD/%	Spider/%
DeepSeek-V3	DeepSeek-V3	63.80	85.80
Qwen3-Coder	Qwen3-Coder-30B-A3B-Instruct	67.33	88.27
DAIL-SQL	GPT-4	54.76	86.60
OpenSearch-SQL	GPT-4o	69.30	87.10
Alpha-SQL	Qwen2.5-Coder-32B	69.70	-
XiYan-SQL	GPT-4o + Qwen2.5-Coder-32B	73.34	89.65
CHASE-SQL	Gemini-1.5-Pro + Gemini-1.5-Flash	74.90	87.60
OmniSQL	Qwen2.5-Coder-32B	67.00	89.80
本文方法	Qwen3-Coder-30B-A3B-Instruct + Qwen3-8B	71.23	89.25

从结果中可以得出以下分析：

(1) 在 **BIRD-dev** 上，本文方法取得 71.23% 的执行准确率，仅次于 CHASE-SQL (74.90%) 和 XiYan-SQL (73.34%)。其未能延续在政务数据集上的明显领先，主要原因在于 BIRD 覆盖 95 个不同领域数据库，缺乏目标领域知识库支撑时，LA-Schema 的优势无法充分发挥。尽管如此，本文方法仍显著优于 DAIL-SQL (+16.47%)、OmniSQL (+4.23%)，且较相同基础模型的 Qwen3-Coder 零样本基线提升 3.90 个百分点，说明异构协同生成与微调判别机制在通用场景下仍具增益。

(2) 在 **Spider-test** 上，本文方法取得 89.25% 的执行准确率，仅次于 OmniSQL (89.80%) 和 XiYan-SQL (89.65%)，并超过 CHASE-SQL (87.60%)。Spider 的查询结构更规范、领域歧义更少，因此各方法差距较 BIRD 进一步收窄。本文方法在未做特定适配的情况下仍保持竞争力，说明异构候选生成框架和配对判别机制具有较好的通用适用性。值得注意的是，OmniSQL 依赖百万级合成数据的大规模监督微调，而本文仅对判别模型进行了小规模微调，在训练成本上更具优势。

(3) 综合来看，本文方法在公共数据集上保持了稳定竞争力，验证了异构协同生成与微调判别框架的通用性；而其在政务数据集上的更大领先幅度，也进一步说明 LA-Schema 在具备领域知识支撑的垂直场景中能够发挥更强价值。

3.9 本章小结

本章针对政务数据查询中业务语义模糊、计算逻辑隐含及查询结构复杂等挑战，提出了基于逻辑增强与异构协同的 Text-to-SQL 方法。该方法通过构建结构化领域知识库并以 LA-Schema 形式显式注入数据库模式，提升政务领域的业务理解能力；通过逻辑映射与渐进分治双路径异构生成机制，平衡了高频标准需求与低频复杂长尾需求；通过微调二分类判别模型与执行反馈修复机制，提升了最终查询的准确性与鲁棒性。最终在政务数据集和公开数据集验证了本章所提模型的有效性。为构建智能问数系统提供了核心技术支撑。

第4章 基于分析规划与查询增强的政务数据分析方法

4.1 引言

第三章围绕政务智能问数中的可靠取数问题，提出了基于逻辑增强与异构协同的 Text-to-SQL 方法，重点解决自然语言查询到结构化查询语句的准确映射问题。然而，在真实政务业务场景中，用户需求往往并不止于“查到数据”，而是进一步希望系统能够围绕查询结果完成趋势识别、横向比较、结构刻画和结论归纳等分析任务。因此仅有可靠的 SQL 生成能力仍不足以支撑完整的政务数据洞察。

基于此，本章聚焦于：如何在第三章可靠查询能力的基础上，将自然语言分析需求转化为结构化分析计划；如何将分析任务拆解为查询层与分析层两个相互协同但职责清晰的阶段；以及如何基于查询结果生成规范化的图表、表格和文字洞察。换言之，第三章解决查准，本章进一步解决围绕查询结果开展规范分析。

本章将分析过程定义为：给定自然语言分析需求 x 、目标数据库模式 S 、已构建的领域知识库 K 、动态样例库 F 、查询生成 workflow G_q 、分析算子库 O_a 以及可视化规则集合 $\mathcal{V}$ ，系统最终生成分析规划 A 和结果包 R ，使得分析结果与用户意图最大化一致，即：

$$(A^*, R^*) = \arg \max_{A \in \mathcal{A}, R \in \mathcal{R}} P(A, R \mid x, S, K, F, G_q, O_a, \mathcal{V}) \quad (4-1)$$

其中， $\mathcal{A}$ 表示候选分析规划集合， $\mathcal{R}$ 表示候选结果集合。为支撑上层应用展示与后续系统实现，本章将输出统一组织为

$$\text{ResultPackage} = \{\text{charts}, \text{tables}, \text{insights}, \text{logs}\} \quad (4-2)$$

其中 `charts` 表示图表文件或图表描述集合，`tables` 表示结构化分析表格结果，`insights` 表示文字洞察摘要，`logs` 表示执行状态与异常信息。

结合政务场景中的高频需求，本章将目标任务限定为四类典型分析类型，各类型的核心逻辑与适用场景如表4-1所示。

与查询任务相比，面向分析的结果生成面临三类挑战。其一，分析需求通常隐含指标、维度、时间粒度、统计口径和展示形式等多重约束，若缺乏中间规划过程，系统容易遗漏关键分析步骤；其二，若分析环节重新承担数据库理解与 SQL 生成任务，则会再次暴露 Schema 歧义、字段映射错误和逻辑幻觉等问题；其三，分析结果不仅要正确，还要展示规范，因此需要在方法层面形成统一的结果组织方式，而具体的服务部署等工程实现将在系统实现章节展开。

表 4-1 四类典型分析类型及其适用场景

分析类型	核心逻辑	适用场景
趋势分析	沿时间维度追踪指标的变化轨迹与演进规律	财政收入增减趋势、企业注册量波动、工单受理量走势等
对比分析	沿区域、部门、年份等维度对关键指标进行横向比较	不同区县经济指标差异、部门绩效对比、年度预算同比等
排名分析	按指定指标排序并识别头部与尾部对象	项目投资额排名、预算执行率排名、办件效率排名等
结构分析	计算整体内部各组成部分的占比与分布	财政支出结构、产业类型分布、收入来源构成等

基于上述考虑，本章以“分析规划、查询增强、算子生成、规范输出”为总体思路展开研究。首先，通过构建结构化中间表示 AP-Schema，将开放式分析需求转化为面向执行的分析规划；第二，通过查询增强的任务分解机制，将分析任务拆分为查询层操作与分析层操作，确保复杂业务口径仍由数据查询模块统一处理；第三，引入轻量分析算子库与可视化规则，在查询结果上完成稳定的分析结果生成；最后，将输出统一组织为标准化结果包，为后续的系统实现提供清晰的方法基础。通过上述设计，数据查询模块与数据分析模块共同构成从可靠取数到规范分析的完整技术链路。

4.2 总体框架设计

基于上述目标，本文设计了一个面向政务洞察的分析规划与查询增强协同框架。其整体流程由需求解析、AP-Schema 构建、查询需求投影、取数调用、分析算子执行以及结果组织六个阶段组成。与第三章的 Text-to-SQL 流程相比，本章不再重新研究 SQL 生成，而是将第三章的方法作为标准化数据获取子模块嵌入分析流程中，实现“先查准、再分析”的层级协同。

框架的核心数据流可表示为：

$$\begin{cases} A = \text{BuildPlan}(x, K), \\ Q_A = \pi_q(A), P_A = \pi_a(A), \\ D = G_q(x, Q_A; S, K, F), \\ R = \text{Generate}(P_A, D; \mathcal{O}_a, \mathcal{V}) \end{cases} \quad (4-3)$$

其中， A 表示由分析规划模块生成的 AP-Schema； $\pi_q(\cdot)$ 表示从 AP-Schema 中投影出取数需求子结构的操作； $\pi_a(\cdot)$ 表示投影出分析层操作与展示要求的操作； S 为

目标数据库模式； K 为3.3.2中构建的领域知识； F 为3.3.3构建的动态样例库； G_q 表示对第三章查询生成管线与数据库执行过程的上层抽象。具体而言，系统首先通过模板化组装函数将原始分析需求 x 与查询投影 Q_A 合成为受约束的查询子问题 $x_q = \text{Assemble}(x, Q_A)$ ，再结合目标数据库模式 S 、领域知识库 K 和动态样例库 F ，由数据查询方法（3.4至3.7）完成模式链接、候选生成与最优 SQL 查询 c_f 选择，最后在目标数据库上执行 c_f 并封装返回结果； $\mathcal{D}$ 表示该管线返回的数据结果、字段元数据与执行状态； R 表示最终结果包。

整个框架如图4-1所示：

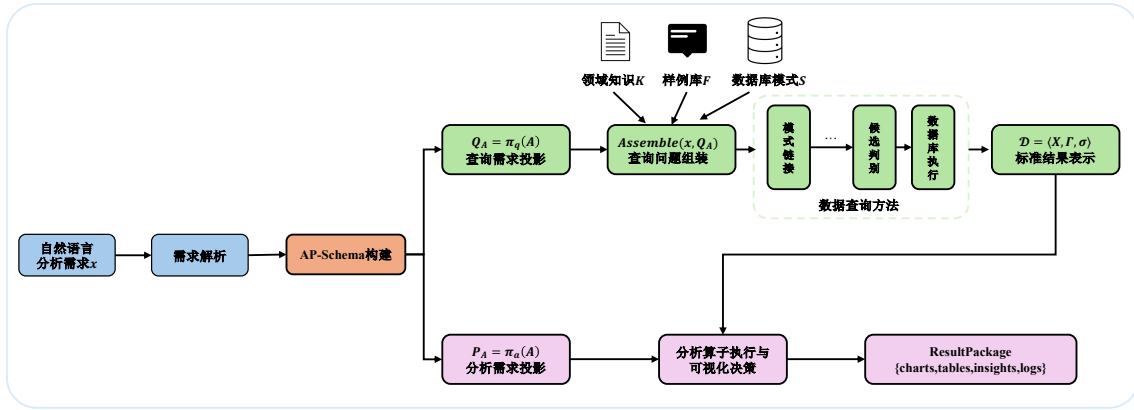


图 4-1 分析规划与查询增强协同数据分析框架图

需求解析阶段负责从用户输入中识别分析对象、指标、时间范围、分组维度和期望展示形式，为后续规划提供语义基础。

AP-Schema 构建阶段将自然语言需求转化为结构化分析表示，用于明确“分析什么”“通过哪些数据分析”“如何展示结果”三个核心问题。该中间表示是本章的关键方法载体，也是连接第三章查询方法与第五章系统实现的重要桥梁。

查询需求投影阶段从 AP-Schema 中抽取与数据获取相关的指标、维度、过滤条件、时间粒度和查询层操作，并调用查询生成管线完成可靠取数。

取数调用阶段依据 $x_q = \text{Assemble}(x, Q_A)$ 将受约束查询子问题送入第三章查询生成管线，以确保字段绑定、逻辑指标求解和 SQL 执行仍由查询模块统一完成。

分析算子执行阶段在获取到查询结果后，根据分析类型与任务要求，从轻量分析算子库中选择合适的操作链，对结果数据完成排序、占比、透视、标注等分析处理，并结合可视化规则生成规范化图表与表格。

结果组织阶段将图表、表格、摘要和日志统一封装为 ResultPackage，以便后续系统展示、日志记录和误差分析。

通过上述设计，本章的方法在保持大语言模型语义理解能力的同时，引入了

清晰的中间表示、稳定的任务分解机制和规范化的结果生成流程，从而应对政务场景下对可靠性、规范性与可落地性的综合要求。

4.3 分析规划构建

分析规划并非对用户需求的简单转写，而是将分析对象、统计口径、执行边界与展示要求统一编码为可验证的中间表示。为此，本节围绕 AP-Schema（Analysis Plan Schema）展开，依次说明其模式定义、字段约束与完备性校验机制，以及从自然语言需求到结构化规划实例的构建过程，从而为后续查询投影与分析算子编排提供统一基础。AP-Schema 的整个构建与约束机制过程如图4-2所示。

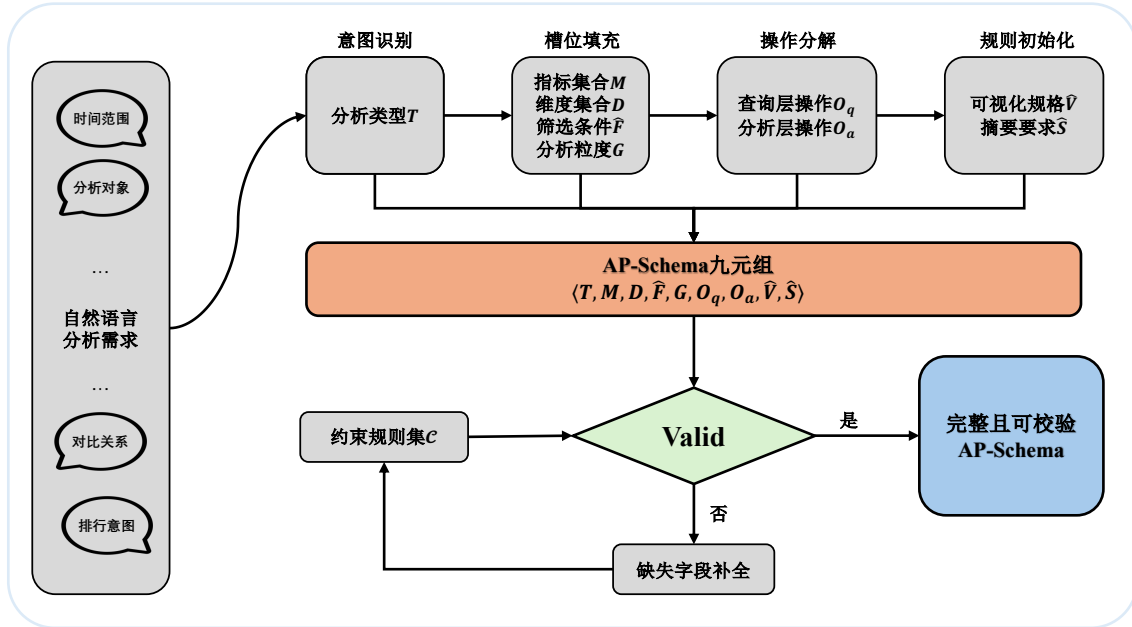


图 4-2 AP-Schema 构建与约束机制图

4.3.1 规划模式定义

为统一表达分析需求并为后续查询投影与算子编排提供结构化基础，本文引入分析规划模式 AP-Schema，并将其定义为：

$$\begin{cases} A = \langle T, M, D, \hat{F}, G, O_q, O_a, \hat{V}, \hat{S} \rangle, \\ T \in \{\text{trend, comparison, ranking, structural}\} \end{cases} \quad (4-4)$$

表4-2给出了 AP-Schema 各字段的语义定义。与原始自然语言需求相比，AP-Schema 将分析任务中的关键约束显式化，使后续的查询投影、分析算子组织和结果验证能够围绕同一结构化表示协同展开。

表 4-2 AP-Schema 字段定义

组成部分	符号	含义	典型取值
分析类型	T	任务所属的分析类别	trend、ranking 等
指标集合	M	待分析指标及其聚合方式	预 算 收 入/SUM、 项 目 数/COUNT
维度集合	D	用于分组、对比或结构刻画的语义维度	区县、部门、时间、支出领域
筛选条件	$\hat{F}$	时间范围与对象过滤条件	年份 =2025、区县 $\in \{A,B\}$
分析粒度	G	时间或统计粒度	年度、季度、月度、类别级
查询层操作	O_q	取数阶段的口径与关联操作	同比口径、跨表关联
分析层操作	O_a	结果上的展示与整理操作	sort→topk→annotate
可视化规格	$\hat{V}$	图表类型与坐标映射要求	折线图、柱状图、饼图
摘要要求	$\hat{S}$	文字洞察需覆盖的要点	峰值月份、占比前三、最大增幅

4.3.2 约束与完备性

AP-Schema 的九个字段并非相互独立，而是存在由分析类型 T 驱动的结构性约束。不同的 T 取值对维度集合 D 、分析粒度 G 和默认可视化类型 $\hat{V}$ 等字段提出了不同的最低要求，从而将后续生成过程约束在合理的决策空间内。表4-3列出了四类分析类型对关键字段的典型约束。

表 4-3 分析类型对 AP-Schema 关键字段的约束

T 分析类型	D 必须包含	G 典型取值	$\hat{V}$ 默认	O_a 常用算子
趋势分析	时间维度	月度/季度/年度	折线图	sort, annotate
对比分析	至少一个对比维度	按对比维度聚合	柱状图	pivot, sort
排名分析	排序维度	按维度聚合	横向柱状图	sort, topk
结构分析	分类维度	类别级汇总	饼图/堆叠柱	share, annotate

这些约束在规划生成阶段起到完备性校验的作用。例如，若系统识别出 $T = \text{trend}$ 却未在 D 中包含时间维度，则规划应视为不完备并触发补全；若识别出 $T = \text{structural}$ 却未发现任何分类维度字段，系统需要回溯至意图识别阶段重新提取维度信息。本文将该校验形式化为：

$$\text{Valid}(A) = \bigwedge_{c_i \in \mathcal{C}} c_i(A) \quad (4-5)$$

其中， $\mathcal{C}$ 表示由表4-3推导的约束规则集合。当 $\text{Valid}(A)$ 不成立时，系统将尝试从

用户输入中二次抽取缺失信息或使用领域知识推断默认值。

4.3.3 规划生成

AP-Schema 的构建过程本质上是从开放式自然语言分析需求到结构化规划表示的语义映射。该映射并非要求大语言模型自由输出分析结论，而是先通过固定槽位提示模板生成规划草案 $A^{(0)}$ ，再依次经过意图识别、槽位填充、操作分解与规格初始化四个子步骤，对草案中的字段进行解析、校验和补全，从而逐层完成对分析需求的结构化。

意图识别阶段负责确定分析类型 T 。系统通过大语言模型对用户输入执行语义解析，识别其中显式或隐式包含的分析意图线索，包括趋势词（“变化”“走势”“波动”）、比较词（“对比”“差距”“哪个更高”）、排序词（“排名”“前三”“最多”）和结构词（“占比”“构成”“分布”）。当输入中同时包含多类意图线索时，系统采用优先级规则进行消歧：若同时出现时间变化语义和对比语义，则优先判定为趋势分析（ $T = \text{trend}$ ），因为对比通常是趋势分析的附属展示而非独立任务。

槽位填充阶段负责从用户输入中提取 M 、 D 、 $\hat{F}$ 、 G 四个核心字段。该阶段结合领域知识库 $K = \{B, L, V\}$ 进行语义归一。具体而言，指标词通过业务术语映射 B 归一为标准业务指标及其默认聚合方式；时间范围和对象范围通过值域约束 V 确认取值合法性；分析粒度 G 根据时间词和分析类型联合推断。例如，当用户提出“分析近两年各区一般公共预算收入的月度趋势”时，“一般公共预算收入”先通过 B 归一为标准业务指标“一般公共预算收入/SUM”，而不在本阶段直接绑定到具体物理字段；“近两年”通过 V 映射为年份 $\in \{2024, 2025\}$ ，“月度”确定 $G = \text{month}$ 。后续在查询需求投影阶段，系统再将这些约束连同原始需求交由数据查询模块结合数据库模式 S 完成字段绑定与 SQL 生成。

操作分解阶段负责将用户需求中涉及的操作划分为查询层操作 O_q 与分析层操作 O_a 。其判断依据是：凡依赖数据库模式理解或改变统计口径的操作归入 O_q ，凡仅对已获取结果进行重组或展示的操作归入 O_a 。例如，上述需求中“并给出同比变化”涉及逻辑计算器 L 中预定义的跨年度计算公式，应映射为查询层逻辑指标（ $O_q = \{\text{同月同比口径}\}$ ），而非在分析层进行自由计算。

规格初始化阶段根据已确定的 T 、 D 、 G 以及可视化规则库 $\mathcal{V}$ 生成初始可视化规格 $\hat{V}$ 和摘要要求 $\hat{S}$ 。此阶段利用表4-3中的默认映射关系确定初始图表类型，并根据指标数量和维度数量调整坐标轴分配与标题内容。

上述映射过程可统一表述为：

$$A^{(0)} = \text{LLM}_{\text{plan}}(x, K, \mathcal{C}), \quad A = \text{BuildPlan}(x, K) \quad (4-6)$$

AP-Schema 的完整生成流程如算法4.1所示。

算法 4.1: AP-Schema 构建算法

Input: 自然语言分析需求 x , 任务类型集合 $\mathcal{T}$, 领域知识 $K = \{B, L, V\}$, 分析算子库 $\mathcal{O}_a$, 可视化规则库 $\mathcal{V}$, 约束规则集 $\mathcal{C}$, 大语言模型 LLM

Output: 分析计划 $A = \langle T, M, D, \hat{F}, G, O_q, O_a, \hat{V}, \hat{S} \rangle$

```

1 初始化:  $A \leftarrow \emptyset$ ;
2  $A^{(0)} \leftarrow \text{LLM}(\text{PromptTemplate}(x, K, \mathcal{C}))$ ; // 按固定槽位输出规划草案
3  $E \leftarrow \text{ParseDraft}(A^{(0)})$ ; // 解析草案中的指标、维度、条件、粒度与展示意图
4  $T \leftarrow \text{IdentifyType}(E, \mathcal{T})$ ; // 识别分析类型
5  $(M, D, \hat{F}, G) \leftarrow \text{ParseSlots}(E, K)$ ; // 归一化指标、维度、条件与粒度
6  $(O_q, O_a) \leftarrow \text{DecomposeOps}(T, M, D, \hat{F}, G, K, \mathcal{O}_a)$ ; // 区分查询层与分析层操作
7  $\hat{V} \leftarrow \text{InitVis}(T, D, G, \mathcal{V})$ ; // 生成初始可视化规格
8  $\hat{S} \leftarrow \text{BuildSummarySpec}(T, M, \hat{V})$ ; // 生成摘要要求
9  $A \leftarrow \langle T, M, D, \hat{F}, G, O_q, O_a, \hat{V}, \hat{S} \rangle$ ;
10 if  $\neg \text{Valid}(A, \mathcal{C})$  then
11    $(M, D, \hat{F}, G) \leftarrow \text{CompleteFields}(x, K, T)$ ; // 基于约束规则触发补全
   // 再次执行重新获取字段信息
12    $(O_q, O_a) \leftarrow \text{DecomposeOps}(T, M, D, \hat{F}, G, K, \mathcal{O}_a)$ 
13    $\hat{V} \leftarrow \text{InitVis}(T, D, G, \mathcal{V})$ 
14    $\hat{S} \leftarrow \text{BuildSummarySpec}(T, M, \hat{V})$ 
15    $A \leftarrow \langle T, M, D, \hat{F}, G, O_q, O_a, \hat{V}, \hat{S} \rangle$ ;
16 end
17 return  $A$ 

```

固定槽位模板要求模型按照 $T \rightarrow M \rightarrow D \rightarrow \hat{F} \rightarrow G \rightarrow O_q/O_a \rightarrow \hat{V}/\hat{S}$ 的顺序输出草案, 从而避免大语言模型在规划阶段直接生成冗长叙述文本。上述流程在返回前需进行基于公式(4-5)的完备性校验。当校验发现规划中存在字段缺失或类型不匹配时, 系统回溯至槽位填充阶段进行二次提取, 并对时间粒度、默认图表类型和摘要要点等字段执行规则化补全, 从而提升规划的鲁棒性。

通过上述四步, 系统可将“分析近两年各区一般公共预算收入的月度趋势, 并给出同比变化”这一自然语言需求映射为表4-4所示的完整 AP-Schema 实例。

最终 AP-Schema 的引入带来三方面收益。其一, 它将分析任务中的隐式约束显式化, 减少系统在后续步骤中遗漏关键分析要求的风险; 其二, 它为数据查询与数据分析模块之间建立了统一接口, 使“取什么数据”和“如何组织分析结果”能够在同一规划框架下协同工作; 其三, 字段约束关系和完备性校验为规划质量提供了可检查的保障机制, 使系统在面对表述不完整或存在歧义的用户输入时仍

表 4-4 自然语言分析需求到 AP-Schema 的映射示例

字段	映射结果
T	trend
M	{一般公共预算收入/SUM, 同比增长率/逻辑指标}
D	{区县, 时间}
$\hat{F}$	{年份 $\in \{2024, 2025\}$ }
G	月度 (month)
O_q	{按区县和月度汇总, 同月同比口径}
O_a	[sort, annotate]
$\hat{V}$	折线图, x 轴 = 时间, y 轴 = 收入金额, 系列 = 区县, 辅助指标 = 同比增长率
$\hat{S}$	趋势摘要: 整体走势、峰值月份、同比最大增幅

能生成结构完整的分析规划。

4.4 基于查询增强的分析任务分解方法

上一节完成了从自然语言需求到 AP-Schema 的结构化规划构建。本节进一步讨论如何将规划中的操作分配给查询层与分析层，并给出两层之间的结果衔接方式。具体而言，本节依次说明查询需求投影与分析需求投影的定义（4.4.1节）、操作归属的判定准则与执行流程（4.4.2节），以及查询层输出的标准结果表示（4.4.3节）。

4.4.1 查询层与分析层的任务划分

据^[13]研究发现以及实践表明，若查询与分析耦合职能划分不明确，则会出现逻辑幻觉重复计算等问题。故本章采用如下划分原则：涉及统计口径、复杂关联或底层数据库计算逻辑的操作，交由第三章查询生成管线完成；本章分析层执行作用于查询结果组织、展示与解释的操作。查询生成在给定用户问题 Q 、数据库模式 S 、领域知识库 K 和动态样例库 F 的条件下完成模式链接、候选生成与候选判别；本章则不直接展开 SQL，而是将与取数相关的约束显式化后交由查询层求解。

为此，本文从 AP-Schema 中分别定义查询需求投影函数与分析需求投影函数：

$$\begin{cases} Q_A = \pi_q(A) = \langle M, D, \hat{F}, G, O_q \rangle \\ P_A = \pi_a(A) = \langle O_a, \hat{V}, \hat{S} \rangle \end{cases} \quad (4-7)$$

其中， Q_A 表示分析规划中与取数相关的结构化约束，描述待查询的指标与聚合要求 (M)、分组维度 (D)、过滤条件 ($\hat{F}$)、时间粒度 (G) 以及需在查询阶段完成的业务口径操作 (O_q)； P_A 表示分析层操作链 (O_a)、可视化规格 ($\hat{V}$) 和摘要要

求 ($\hat{S}$)。

任务划分的执行流程可概括为四步，如图4-3所示。首先，从 AP-Schema 中抽取指标、维度、条件、粒度和候选操作，形成待判定的取数约束与分析约束；其次，围绕模式依赖性、统计口径依赖性和查询结构复杂性依次判断各操作的归属；再次，将满足查询层条件的部分汇总为 Q_A ，将仅依赖结果组织的部分汇总为 P_A ；最后，由查询层输出标准结果表示 $\mathcal{D}$ ，分析层再据此执行算子调用、可视化映射和摘要生成。由此，任务划分从原则性描述转化为可执行的过程表示。

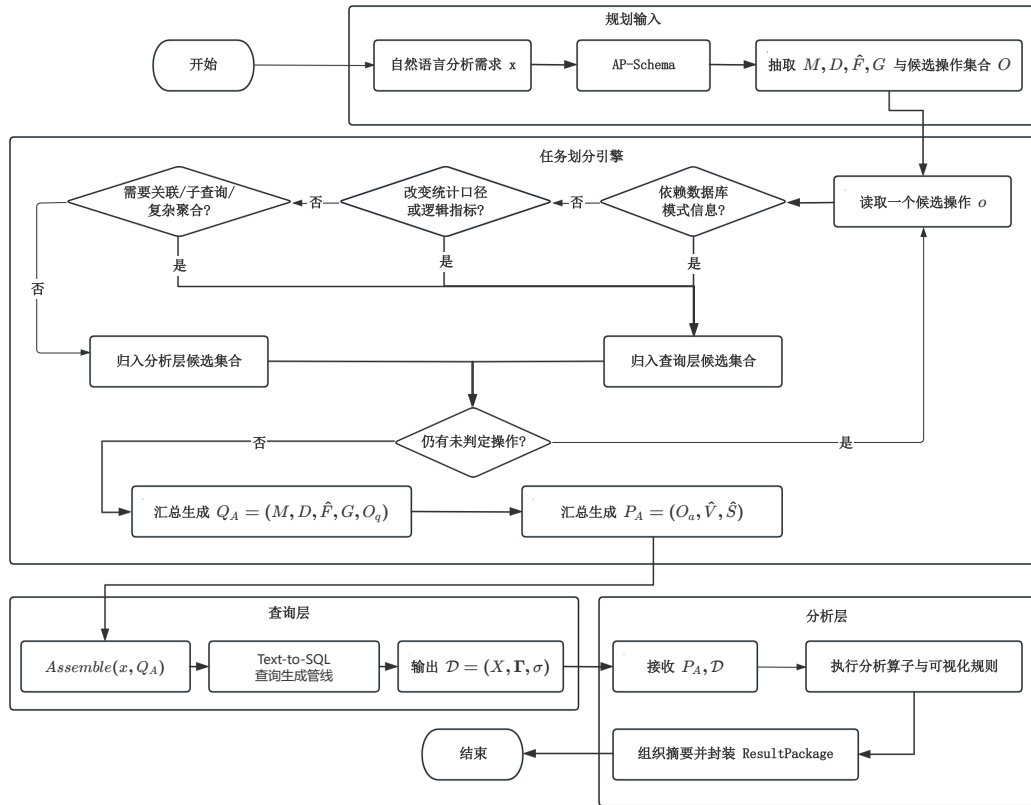


图 4-3 查询层与分析层任务划分执行流程图

4.4.2 任务分解准则与层级判定

为了在具体操作层面系统地区分 O_q 与 O_a ，本文给出如下判定准则：

$$\phi(o) = \begin{cases} \text{query-layer,} & \text{若 } o \text{ 改变统计口径或依赖底层模式与关系计算} \\ \text{analysis-layer,} & \text{若 } o \text{ 仅对已获取结果集进行重组、展示或解释} \end{cases} \quad (4-8)$$

该准则可进一步落实为四个判定维度：操作是否依赖数据库模式信息、是否改变统计口径、是否需要复杂查询结构支持，以及其直接作用对象是数据库还是结果

集。满足前述任一查询层特征的操作应下沉到查询层，否则保留在分析层。表4-5给出了相应的层级判定矩阵。

表 4-5 任务分解操作的层级判定矩阵

判定维度	查询层特征	分析层特征	判定依据
模式依赖性	需要字段、表关系或类型信息	不再访问底层模式	是否必须依赖 S 完成语义绑定
统计口径依赖性	改变指标定义或聚合口径	不改变既定统计口径	是否需要 L 或预定义业务规则支持
查询结构复杂性	需要关联、子查询或复杂聚合	仅进行轻量结果变换	是否必须通过 SQL 结构求解
操作对象	直接作用于数据库取数过程	直接作用于结果集 $\mathbf{X}$	操作输入是数据库还是查询返回结果

边界情形主要出现在粒度不具体的任务中。例如，“按季度汇总”在已返回月度数据时既可由查询层通过调整分组实现，也可由分析层进一步聚合。针对这类情形，本文采用优先查询层的策略，以减少分析层对原始统计口径的二次加工，并保持结果的一致性与可验证性。

4.4.3 查询结果表示与层间衔接

在完成任务划分后，查询层需要向分析层返回统一的结果表示。本文将 G_q 视为“受约束查询意图进入完整 Text-to-SQL 管线并执行”的上层封装，并将其输出记为：

$$\mathcal{D} = G_q(x, Q_A; S, K, F) = \langle \mathbf{X}, \Gamma, \sigma \rangle \quad (4-9)$$

其中， $\mathbf{X} \in \mathbb{R}^{n \times m}$ 表示以结构化表格形式封装的查询结果， n 为记录数， m 为字段数； $\Gamma = \{(\gamma_j^{\text{name}}, \gamma_j^{\text{type}}, \gamma_j^{\text{unit}}, \gamma_j^{\text{grain}}) \mid j = 1, \dots, m\}$ 表示各字段的名称、数据类型、单位和粒度等元数据； $\sigma \in \{\text{success}, \text{failure}\}$ 表示查询执行状态。

这一结果表示承担三项作用： $\mathbf{X}$ 为分析算子提供直接操作对象， Γ 为可视化映射和展示规则提供字段语义信息， σ 为流程控制提供执行依据。当 $\sigma = \text{failure}$ 时，分析流程应终止并在结果包的 logs 中记录失败原因；当查询执行成功但结果中存在空值、异常粒度或字段说明不足时，系统不再额外设置 partial 状态，而是将相关信息写入 Γ 与结果包的 logs 中，以保持层间接口简洁稳定。

通过 Q_A 与 $\mathcal{D}$ 两个接口，第三章与第四章形成清晰分工：前者负责可靠取数，后者负责规范分析。图4-4进一步展示了操作判定与层间衔接的整体流程。

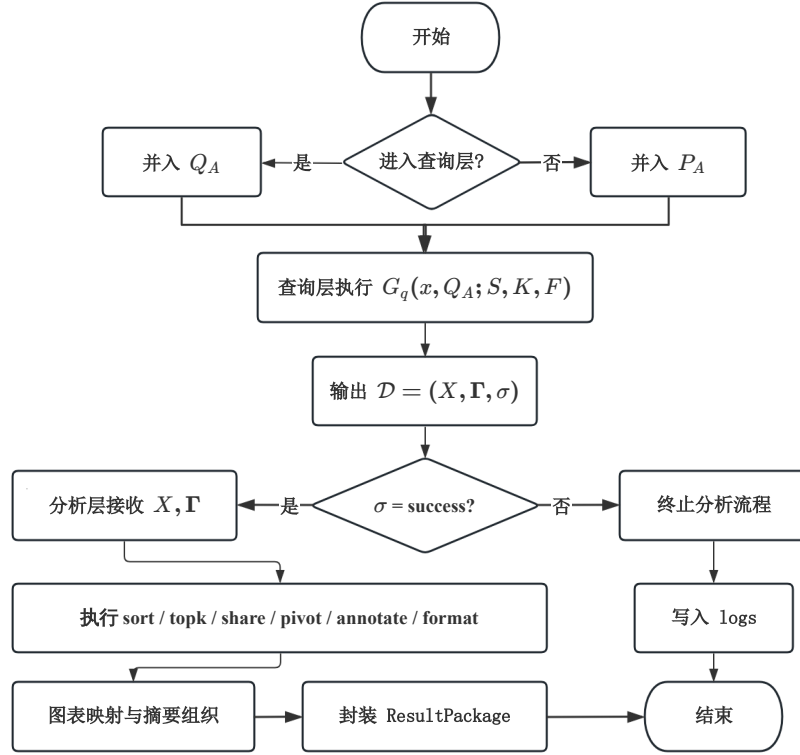


图 4-4 层间衔接流程图

4.5 基于分析算子的结果生成方法

在查询层返回结果表示 $\mathcal{D}$ 后，系统需要将结构化数据进一步转化为图表、表格和文字洞察。本节首先说明分析算子的设计与编排机制，随后给出结果生成流程与输出组织方式。

4.5.1 分析算子设计与编排

为保持分析层的可控性，本文引入面向结果组织的轻量分析算子库

$$\mathcal{O}_a = \{\text{sort}, \text{topk}, \text{share}, \text{pivot}, \text{annotate}, \text{format}\} \quad (4-10)$$

其中，**sort** 用于排序，**topk** 用于提取头部或尾部结果，**share** 用于补充占比计算，**pivot** 用于重组对比结果，**annotate** 用于标记关键点，**format** 用于统一数值格式、单位与标签表达。算子仅作用于查询结果的组织与展示，不重新定义业务指标。

为保持查询层与分析层的职责边界，**share** 仅在查询层已返回构成项汇总值的前提下计算占比，不重新定义分子和分母；**annotate** 仅从结果表中抽取峰值、最低值、拐点和 Top-k 对象等显式特征；**format** 仅对数值单位、百分比形式和标签文本进行规范化处理。

针对四类典型分析任务，本文设计如下基础算子链：

$$\Psi(T) = \begin{cases} [\text{sort}, \text{annotate}], & T = \text{trend} \\ [\text{pivot}, \text{sort}, \text{annotate}], & T = \text{comparison} \\ [\text{sort}, \text{topk}, \text{format}], & T = \text{ranking} \\ [\text{share}, \text{sort}, \text{annotate}], & T = \text{structural} \end{cases} \quad (4-11)$$

其中，趋势分析强调时间顺序与关键点标注，对比分析强调结果对齐与差异突出，排名分析强调排序和头尾识别，结构分析则在组成部分已完整返回的前提下补充占比与重点类别标注。

设调整后的算子链记为 $\text{chain} = [o_1, o_2, \dots, o_l]$ ，则分析层对查询结果的处理过程可统一表示为：

$$\mathbf{Y} = o_l \circ o_{l-1} \circ \dots \circ o_1(\mathbf{X}, \Gamma) \quad (4-12)$$

其中 $\mathbf{Y}$ 表示算子链执行后的结果表。

在图表选择上，本文采用“初始规格 + 数据后校正”的策略。规划阶段给出初始可视化规格 $\hat{V}$ ，结果返回后再结合类别数、字段类型和时间粒度确定最终图表类型：

$$v^* = \begin{cases} \text{line_chart}, & T = \text{trend} \\ \text{bar_chart}, & T \in \{\text{comparison}, \text{ranking}\} \\ \text{pie_chart}, & T = \text{structural} \text{ 且类别数} \leq 6 \\ \text{stacked_bar_chart}, & T = \text{structural} \text{ 且类别数} > 6 \text{ 或存在时间对比} \end{cases} \quad (4-13)$$

在具体编排时，系统以上游规划给出的分析类型 T 、分析需求投影 $P_A = \langle O_a, \hat{V}, \hat{S} \rangle$ 以及查询结果表示 $\mathcal{D} = \langle \mathbf{X}, \Gamma, \sigma \rangle$ 为输入，先由 $\Psi(T)$ 确定基础算子链，再结合 O_a 中的显式要求以及 Γ 中的字段类型、类别数和粒度信息，对算子顺序和图表规格进行轻量调整，最终得到可执行的算子序列与可视化配置。由此，分析层既保留了类型驱动的稳定性的，又能够对真实数据特征作出必要响应。

4.5.2 结果生成流程与输出组织

在完成算子编排与可视化决策后，系统进入结果生成阶段。该阶段以分析需求投影 $P_A = \langle O_a, \hat{V}, \hat{S} \rangle$ 和查询结果表示 $\mathcal{D} = \langle \mathbf{X}, \Gamma, \sigma \rangle$ 为输入，经过执行状态检查、算子链执行、图表规格修正、摘要生成与结果封装五个步骤，最终输出标准化结果包 R 。算法4.2给出了该流程的详细步骤。

为减少摘要生成过程中的自由发挥，本文将文字洞察生成拆解为“结构化事实抽取”和“摘要组织”两个顺序阶段：

$$\begin{cases} F^* = \text{ExtractFacts}(\mathbf{Y}, \hat{S}), \\ I = \text{Verbalize}(F^*, \hat{S}) \end{cases} \quad (4-14)$$

其中， F^* 表示从结果表中抽取的峰值、增幅、排名和占比等结构化事实集合， I 表示在摘要要求约束下形成的文字洞察。

算法 4.2: 分析结果生成算法

Input: 分析需求投影 $P_A = \langle O_a, \hat{V}, \hat{S} \rangle$ ，查询结果表示 $\mathcal{D} = \langle \mathbf{X}, \Gamma, \sigma \rangle$ ，分析类型 T ，可视化规则库 $\mathcal{V}$

Output: 结果包 $R = \{\text{charts}, \text{tables}, \text{insights}, \text{logs}\}$

```

1 if  $\sigma = \text{failure}$  then
2    $R \leftarrow \{\text{charts} = \emptyset, \text{tables} = \emptyset, \text{insights} = \emptyset, \text{logs}(\sigma, \text{错误原因})\};$ 
3   return  $R$ ; // 查询失败，直接终止
4 end
   // 步骤 1: 选择并执行算子链
5  $\text{chain} \leftarrow \Psi(T);$  // 由分析类型确定基础算子链
6  $\text{chain} \leftarrow \text{Adjust}(\text{chain}, O_a, \Gamma);$  // 结合显式要求与字段元数据调整
7  $\mathbf{Y} \leftarrow \text{Apply}(\text{chain}, \mathbf{X}, \Gamma);$  // 依次执行算子，得到处理后结果
   // 步骤 2: 修正可视化规格
8  $n_{\text{cat}} \leftarrow \text{CountCategories}(\mathbf{Y}, \Gamma);$  // 统计实际类别数
9  $\hat{V}' \leftarrow \text{RefineVis}(\hat{V}, \mathbf{Y}, \Gamma, n_{\text{cat}}, \mathcal{V});$  // 按公式(4-13)校正图表类型
   // 步骤 3: 抽取结构化事实并生成文字洞察
10  $F^* \leftarrow \text{ExtractFacts}(\mathbf{Y}, \hat{S});$  // 抽取峰值、增幅、Top-k、占比等事实
11  $I \leftarrow \text{Verbalize}(F^*, \hat{S});$  // 按摘要要求组织文字洞察
   // 步骤 4: 封装结果包
12  $R \leftarrow \{\text{charts}(\hat{V}', \mathbf{Y}), \text{tables}(\mathbf{Y}), \text{insights}(I), \text{logs}(\sigma, \Gamma)\};$ 
13 return  $R$ 
```

算法4.2有四个关键设计要点。第一，执行状态 σ 的前置检查确保了分析层不会在错误数据上盲目执行算子：当查询失败时直接终止并保留日志，从而避免在无效数据上继续分析。第二，算子链的执行过程采用公式(4-12)所示的确定性组合方式，使系统既遵循类型驱动的基础编排 $\Psi(T)$ ，又能根据 AP-Schema 中用户显式指定的 O_a 和真实数据的字段特征进行微调。第三，文字洞察生成采用公式(4-14)所示的“事实抽取优先”策略，先从结果表中提取峰值、增幅、Top-k 和占比等结构化事实，再在摘要要求 $\hat{S}$ 的约束下组织文本，以降低自由生成导致的结论漂移。第四，可视化规格修正依据公式(4-13)，在真实类别数 n_{cat} 等数据特征的基础上对初

始图表类型进行后校正，避免仅凭自然语言需求过早决定图表类型所带来的偏差。

经过上述流程，系统最终输出的结果包 R 满足公式(4-2)所定义的统一结构，为第五章的系统实现提供了标准化的方法输出接口。

4.6 实验与结果分析

本节旨在验证本章方法的三项核心主张：其一，基于 AP-Schema 的分析规划是否能够提升分析任务完成率与结果稳定性；其二，复用查询生成管线是否能够显著提升数据获取正确性并最终改善分析结果质量；其三，方法中各子模块是否各自具有不可替代的独立贡献。与面向 Text-to-SQL 的大规模基准评测不同，本章实验以中等规模任务集上的端到端验证为主，并通过分类型对比、细粒度模块消融和规划质量评估等多维度实验，体现方法的协同效果与各模块的独立作用。

4.6.1 实验设置与评价指标

实验任务基于第三章构建的同源政务数据库场景展开（见表3-4），仍使用财政预算、公共资源和政务项目等领域的 5 个数据库、33 张数据表作为底层数据来源。在此基础上，本文构建了 168 个自然语言分析任务，其中趋势分析 48 个、对比分析 42 个、排名分析 42 个、结构分析 36 个。每个任务均配备人工核验的参考分析目标，包括目标指标、期望图表类型和关键结论要点。

为保持与第三章的衔接，本章在分析规划阶段采用与第三章相同的 Qwen3-Coder-30B-A3B-Instruct 作为基础模型；数据获取环节直接复用第三章的完整查询生成管线。文字洞察组织阶段仍采用同一基础模型，但其输入被约束为公式(4-14)抽取的结构化事实集合。实验评价采用以下指标。

端到端评价采用四个指标：

流程成功率（Pipeline Success Rate, PSR），表示系统能够完成从分析规划、查询获取到结果生成整个流程并返回有效结果包的比例：

$$PSR = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[\text{pipe}_i = 1] \quad (4-15)$$

分析任务完成率（Task Completion Rate, TCR），表示不仅流程执行成功，而且输出图表、表格和摘要在逻辑上满足原始分析意图的比例：

$$TCR = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[\text{task}_i = 1] \quad (4-16)$$

结果正确率 (Result Accuracy, RA), 表示在任务完成样本中, 系统输出与人工参考结果一致的比例。对于数值结果, 若相对误差不超过 $\epsilon = 5\%$ 则视为正确:

$$RA = \frac{1}{N_c} \sum_{i=1}^{N_c} \mathbb{I}[\delta(y_i^{pred}, y_i^{ref}) \leq \epsilon] \quad (4-17)$$

其中 N_c 表示任务完成的样本数, $\delta(\cdot)$ 表示相对误差或人工一致性判别函数。

图表恰当性 (Chart Appropriateness, CA), 由 3 名标注者从图表类型选择、坐标映射、标题与标签完整性三个维度进行 1 到 5 分评分, 取平均值:

$$CA = \frac{1}{3N_c} \sum_{i=1}^{N_c} \sum_{j=1}^3 \text{score}_{ij} \quad (4-18)$$

为单独评价 AP-Schema 构建阶段的规划质量, 本文引入以下三个辅助指标。

意图识别准确率 (Intent Accuracy, IA), 表示正确识别分析类型 T 的比例:

$$IA = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[T_i^{pred} = T_i^{ref}] \quad (4-19)$$

槽位填充完备率 (Slot Completeness, SC), 表示 AP-Schema 中各必填字段被正确填充的比例。对于每个任务 i , 设 AP-Schema 共有 $|\mathcal{S}|$ 个必填槽位, 其中正确填充的槽位数为 s_i :

$$SC = \frac{1}{N} \sum_{i=1}^N \frac{s_i}{|\mathcal{S}|} \quad (4-20)$$

约束满足率 (Constraint Satisfaction Rate, CSR), 表示生成的 AP-Schema 通过公式(4-5)完备性校验的比例:

$$CSR = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[\text{Valid}(A_i) = \text{true}] \quad (4-21)$$

为验证本章方法的有效性, 本文设置如下五组对比方法:

(1) **Code-Gen Agent**: 代表“LLM 生成分析代码”范式^[46]。使用同一基础模型 (Qwen3-Coder-30B-A3B-Instruct), 给定数据库模式与分析任务描述, 由大语言模型直接生成包含 SQL 取数、Pandas 数据处理和 Matplotlib 可视化的端到端 Python 分析脚本。该范式广泛应用于主流数据分析产品中;

(2) **ReAct Agent**: 代表“LLM + 工具迭代调用”范式^[72]。使用同一基础模型, 配置 SQL 执行工具和绘图工具, 以 ReAct (Reasoning-Acting) 循环方式驱动大语

言模型交替进行推理与工具调用，迭代完成数据分析任务；

(3) **Direct-Analysis**: 直接将自然语言分析需求端到端映射为分析过程与结果，不经过显式的 AP-Schema 规划，也不显式复用第三章查询生成管线；

(4) **Plan+Free-SQL**: 采用 AP-Schema 和结果生成规范，但在数据获取阶段不调用第三章查询生成管线，而是在分析流程中自由生成 SQL；

(5) **Ours**: 本文完整方法，即“AP-Schema 规划 + 查询增强取数 + 轻量分析算子结果生成”。

其中，Code-Gen Agent 和 ReAct Agent 均使用与本文方法相同的基础模型和数据库环境，以确保对比的公平性。

4.6.2 结果对比

图4-5展示了五种方法在 168 个政务分析任务上的总体结果。

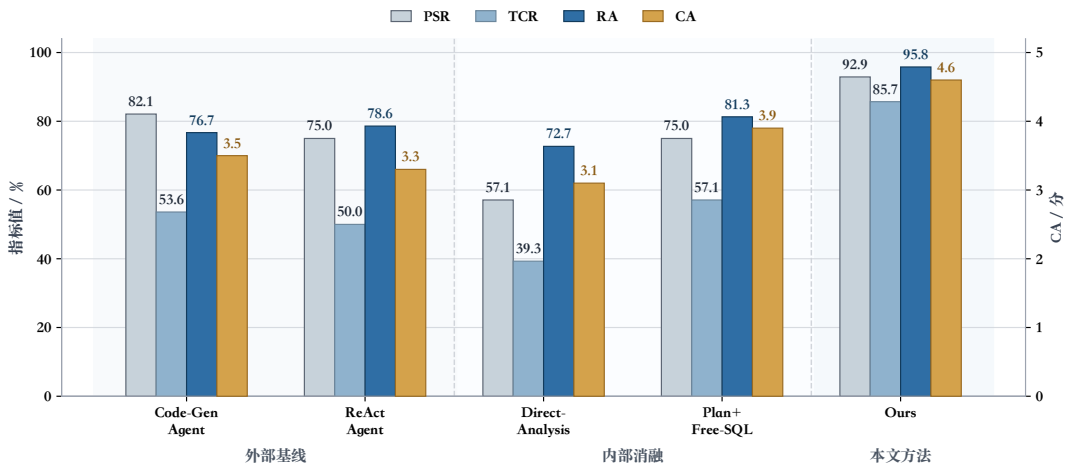


图 4-5 不同分析方法的总体实验结果

从结果中可以得出以下分析：

本文方法显著优于外部基线。Code-Gen Agent 的 PSR 较高 (82.1%)，说明代码生成范式在流程执行层面具有一定的鲁棒性；但其 TCR 仅为 53.6%，RA 也仅为 76.7%，主要原因在于：生成的 Python 脚本缺乏对政务业务指标和统计口径的结构化理解，导致分析逻辑不完整或关键维度遗漏。ReAct Agent 虽然可以通过迭代调用工具自主纠错，但其 PSR (75.0%) 和 TCR (50.0%) 均低于 Code-Gen Agent，因为结构化规划的缺失下迭代过程容易陷入无效循环或工具调用错误，流程稳定性不足。相比之下，本文方法的 RA 达到 95.8%，TCR 达到 85.7%，在所有指标上均大幅领先，证明了 AP-Schema 规划与查询增强机制的联合优势。

分析规划显著提升了任务完成稳定性。与 Direct-Analysis 相比，Plan+Free-SQL

在 PSR 上提升了 17.9 个百分点,在 TCR 上提升了 17.8 个百分点。这表明 AP-Schema 能够有效约束系统对指标、维度、时间粒度和展示形式的理解,减少流程中遗漏关键分析要求的情况。值得注意的是,Plan+Free-SQL 的 TCR (57.1%) 已高于 Code-Gen Agent (53.6%) 和 ReAct Agent (50.0%),说明仅引入结构化规划即可超越主流代码生成和工具调用范式。

复用查询生成管线是提升整体性能的关键。在已经采用分析规划的前提下,Ours 相较于 Plan+Free-SQL 在 TCR 上进一步提升 28.6 个百分点,在 RA 上提升 14.5 个百分点。这说明若在分析阶段重新自由生成 SQL,仍会重新暴露字段误解、连接错误和过滤条件偏差等问题;而查询生成管线能够提供更可靠的数据输入,使分析方法将注意力集中在分析本身。

图表恰当性受益于轻量算子与规则约束。Ours 在 CA 上达到 4.6 分,显著高于 Code-Gen Agent (3.5 分) 和 ReAct Agent (3.3 分)。Code-Gen Agent 通过 Matplotlib 生成图表虽然功能可用,但缺乏类型驱动的图表选择规则,标题和坐标映射常不规范;ReAct Agent 的图表质量更不稳定,因为迭代生成过程中图表规格容易被多次修改导致不一致。本文方法通过任务类型到图表类型的规则映射以及结合数据特征的后校正机制,在图表类型选择、标题描述和坐标配置上均更加规范。

4.6.3 分类型性能对比

为进一步揭示不同分析场景下各方法的表现差异,本文按四种分析类型分别统计五种方法的 TCR 和 RA。表4-6给出了分类型的实验结果。

表 4-6 不同分析类型下各方法的 TCR 与 RA 对比

方法	趋势分析		对比分析		排名分析		结构分析	
	TCR	RA	TCR	RA	TCR	RA	TCR	RA
Code-Gen Agent	43.8	66.7	57.1	79.2	64.3	81.5	50.0	77.8
ReAct Agent	37.5	72.2	52.4	81.8	59.5	80.0	52.8	78.9
Direct-Analysis	31.3	60.0	42.9	83.3	50.0	71.4	33.3	75.0
Plan+Free-SQL	50.0	75.0	64.3	88.9	64.3	77.8	50.0	83.3
Ours	87.5	92.9	85.7	100.0	92.9	92.3	75.0	100.0

趋势分析对分析规划的依赖最为明显。趋势分析要求系统正确识别时间维度、统计粒度(月度/季度)和同比口径等多重约束。Code-Gen Agent 和 ReAct Agent 在此类型上的 TCR 分别仅为 43.8% 和 37.5%,说明代码生成和工具迭代范式在处理多约束分析任务时容易遗漏关键时间粒度或同比要求。引入 AP-Schema 后(Plan+Free-SQL),TCR 提升至 50.0%;而本文完整方法进一步将 TCR 提升至 87.5%,表明结

构化规划与查询增强机制的联合能够有效保障逻辑指标的正确计算。

对比分析与结构分析受益于查询增强取数最为显著。在这两类任务中，本文方法的 RA 均达到 100.0%，而 Plan+Free-SQL 的 RA 分别为 88.9% 和 83.3%。对比分析要求系统在同一口径下获取多个对象的数据，结构分析需要获取完整的组成项汇总值，二者对查询结果的完整性和一致性有较高要求。自由生成 SQL 时容易出现过滤条件偏差或聚合范围不一致，导致数据缺失或口径错误。

排名分析的 TCR 最高，外部基线在此类型上表现相对最好。本文方法在排名分析上的 TCR 达到 92.9%，在四类任务中最高。这主要因为排名分析的结构相对规整（排序+Top-k），AP-Schema 的约束和算子链 $\Psi(\text{ranking}) = [\text{sort}, \text{topk}, \text{format}]$ 能够直接映射为稳定的执行流程。值得注意的是，Code-Gen Agent 在排名分析上的 TCR 达到 64.3%，是其四类任务中的最高值，说明代码生成范式对结构相对简单的排序类任务具有一定的天然适应性。

结构分析的 TCR 相对偏低，但 RA 最高。结构分析的 TCR 在四类任务中最低，主要原因是部分结构分析任务的分类维度较复杂，规划阶段有时将分类维度误判为对比维度，导致生成的 AP-Schema 类型不匹配。而一旦任务完成其 RA 达到 100.0%，说明在规划正确的前提下，查询增强机制和占比算子能够可靠地产出正确结果。

4.6.4 模块消融实验

总体结果对比已验证了分析规划与查询增强两大核心设计的整体效果。为更精确地量化各子模块的独立贡献，本文在完整方法的基础上分别关闭五个关键子模块，观察端到端指标的变化。与宏观方案对比不同，本节所有消融配置均保留完整的方法框架（AP-Schema + 查询增强 + 结果生成），仅逐一移除特定子模块，以隔离其独立作用。表4-7给出了消融实验结果。

表 4-7 模块消融实验结果

模型方法	PSR/%	TCR/%	RA/%	CA/分
w/o Completeness Validation	87.5	75.0	92.9	4.4
w/o Task Decomposition	89.3	78.6	86.4	4.1
w/o Operator Chain $\Psi(T)$	91.1	80.4	93.3	3.8
w/o Chart Post-correction	92.9	83.9	95.8	3.9
w/o Fact Extraction	92.9	82.1	93.5	4.0
Full Model	92.9	85.7	95.8	4.6

w/o Completeness Validation（去除完备性校验）。保留 AP-Schema 的结构化

表示,但跳过公式(4-5)的约束校验与字段补全机制。去除后,TCR 和 PSR 性能下降。分析发现,失败案例主要集中在趋势分析和结构分析中:前者因缺少时间维度校验导致 G 字段缺失,后者因分类维度未通过约束检查而遗漏关键分组条件。这表明完备性校验是 AP-Schema 从“结构化表示”走向“可靠规划”的关键保障。

w/o Task Decomposition (去除查询/分析层分解)。保留 AP-Schema 和查询增强取数,但不再将操作显式区分为 O_q 和 O_a ,而是将所有操作统一交由大语言模型自行决定分配。去除后,RA 降幅达 9.4 个百分点。错误显示,系统在缺乏显式分层时容易将本应由查询层处理的统计口径操作错误地放入分析层自由计算,导致数值偏差。这说明公式(4-8)所定义的层级判定准则是保证数据正确性的重要机制。

w/o Operator Chain $\Psi(T)$ (去除类型驱动算子链)。保留查询增强取数,但分析层不使用公式(4-11)定义的预设算子链,改由大语言模型自由组织结果展示。去除后,CA 降幅最为明显;TCR 也下降至 80.4%。自由组织模式下,系统在排名分析中经常遗漏 Top-k 截取,在结构分析中未执行占比计算,导致结果不完整。这表明类型驱动的算子链为分析层提供了稳定的执行骨架,避免结构性遗漏。

w/o Chart Post-correction (去除图表后校正)。保留所有其他模块,但图表类型始终使用规划阶段初始化的 $\hat{V}$,不再根据实际类别数等数据特征进行校正。去除后,PSR 和 RA 均未受影响,但 CA 从 4.6 分下降至 3.9 分。典型错误包括:当结构分析返回的类别数超过 6 个时仍使用饼图(可读性差),以及当对比维度仅有两个对象时使用多系列柱状图而非更清晰的分组柱状图。这说明公式(4-13)的后校正机制虽不影响数据正确性,但对图表展示的规范性具有显著提升。

w/o Fact Extraction (去除结构化事实抽取)。保留所有其他模块,但文字洞察不经过公式(4-14)的两阶段管线,而由大语言模型直接从结果表生成摘要文本。去除后,TCR 下降至 82.1%,CA 从 4.6 分下降至 4.0 分,RA 也略降至 93.5%。自由摘要模式下,系统偶尔在洞察中引入结果表中不存在的数值(如编造排名位次或增幅百分比),导致标注者判定为任务未完成或结果不正确。这表明“先抽取事实、再组织文本”的约束策略能有效抑制大语言模型在摘要生成中的结论漂移。

4.6.5 AP-Schema 规划质量评估

上述实验从端到端角度评价了整体方法效果,但无法直接反映 AP-Schema 构建过程本身的质量。为此,本文单独评估规划阶段的输出,以验证固定槽位模板与完备性校验机制对规划质量的提升作用。

实验对比两种配置:(1) LLM 直接生成,即不使用固定槽位模板,也不执行约束校验,由大语言模型以自由文本形式输出分析规划;(2) 本文方法,即采用固

定槽位模板引导草案生成，并通过公式(4-5)进行约束校验与字段补全。评估基于168个分析任务，每个任务的参考 AP-Schema 由人工标注确定。表4-8给出了两种配置在意图识别准确率（IA）、槽位填充完备率（SC）和约束满足率（CSR）三个指标上的结果。

表 4-8 AP-Schema 规划质量评估结果

配置	IA/%	SC/%	CSR/%
LLM 直接生成	78.6	67.3	53.6
本文方法	94.6	91.8	91.1

固定槽位模板显著提升了意图识别准确率。LLM 直接生成配置的 IA 为 78.6%，错误主要集中在趋势分析与对比分析的混淆（如将“各区收入变化对比”误判为对比分析而非趋势分析），以及排名分析与结构分析的边界模糊。本文方法通过固定槽位模板强制模型先输出 T 字段再填充后续字段，使 IA 提升至 94.6%。

槽位填充完备率的提升体现了结构化模板的约束作用。LLM 直接生成时，SC 仅为 67.3%，常见遗漏包括分析粒度 G 、查询层操作 O_q 和摘要要求 $\hat{S}$ 等非显式字段。本文方法通过固定槽位顺序和领域知识库辅助填充，将 SC 提升至 91.8%。

约束校验机制是保障规划可执行性的核心。LLM 直接生成的 CSR 仅为 53.6%，即近半数规划不满足表4-3定义的类型约束（如趋势分析缺少时间维度、结构分析缺少分类维度）。本文方法通过完备性校验与二次补全将 CSR 提升至 91.1%，使绝大多数规划在进入查询投影阶段前即满足结构完备性要求。

4.7 本章小结

本章针对政务智能问数中“查到数据后如何规范分析”的问题，提出了基于分析规划与查询增强的政务数据分析方法。该方法引入 AP-Schema 将开放式分析需求转化为结构化规划，通过查询增强的任务分解机制复用第三章的可靠取数能力，并借助轻量分析算子库与可视化规则实现规范化的结果生成。实验表明，本文方法在任务完成率、结果正确率和图表恰当性上均显著优于对比方法；分类型对比、细粒度模块消融和规划质量评估三组实验进一步验证了各子模块的独立贡献与设计必要性。由此，第三章解决可靠取数，本章解决规范分析，二者共同为第五章的系统实现提供完整的方法基础。

第 5 章 政务智能问数系统设计与实现

5.1 引言

在第三章和第四章查询与分析方法构建的基础上，本文实现了一个面向政务场景的智能问数系统，并将查询生成方法与分析生成方法整合到统一后端服务之中。该系统能够完成从自然语言输入、可靠取数、结构化分析到结果组织与追溯的完整流程。

系统核心能力主要包括：**（1）自然语言问数**。支持用户以自然语言提交查询类或分析类请求，并由应用层完成统一接入与任务分发；**（2）可靠查询生成**。复用第三章的 LA-Schema 增强、异构候选生成、查询修复与候选判别能力，输出高可信度 SQL 及其结果表；**（3）自动化分析生成**。复用第四章的 AP-Schema 构建、查询增强取数和分析算子编排能力，生成图表、表格与文字洞察；**（4）结果留痕与追溯**。完整保存任务状态、中间产物、执行日志和结果文件，以支持后续复核、追问与问题定位。

围绕这一目标，本章从系统总体架构、应用层与智能服务层实现、系统数据层存储以及系统功能实现与效果展示四个方面展开。

5.2 系统总体架构

为满足查询、分析、会话管理和结果追溯等工程需求，本文实现的政务智能问数系统采用由表示层、应用层、智能服务层和数据层组成的四层架构，如图5-1所示。该分层方式使交互界面、HTTP 接入、智能编排与底层存储相互解耦，利于保持模块边界清晰。

表示层直接面向用户，负责承接自然语言输入、呈现 SQL 与查询结果、展示分析图表与文字洞察，并提供历史会话与结果追溯入口。表示层通过统一接口调用应用层服务，并将后端返回的结构化结果可视化呈现。

应用层采用基于 Python 的 Web 服务栈实现，以 FastAPI 作为统一 HTTP 入口，配合 Pydantic 完成请求对象建模、参数校验和响应序列化。本层同时承担任务创建、状态维护、统一异常处理和 SSE（Server-Sent Events）流式输出等职责，使查询请求和分析请求都以标准化接口进入系统。

智能服务层是系统的核心执行层。本研究采用 LangGraph 的 StateGraph 实现服务级编排，并将查询能力与分析能力分别封装为查询子图和分析子图。其

中，查询服务封装第三章提出的完整查询链路，对外返回 `QueryResponse`；分析服务封装第四章提出的分析规划、查询增强取数和结果生成链路，对外返回 `AnalysisResultResponse`，其结果主体为 `ResultPackage`。对于分析请求，系统先在分析子图中生成 `AP-Schema`，再投影得到查询约束，并由查询调用节点复用查询子图完成取数；查询结果返回后，再进入分析算子、图表构建和洞察生成节点完成结果组织。在这一层中，本地大模型不直接暴露给前端，而是由 HTTP 调用器以受约束方式供各节点复用。

数据层为上层服务提供稳定的数据与知识支撑。政务业务库保存财政、项目、公共资源等原始业务数据；PostgreSQL 保存系统元数据、领域知识库与会话状态；Milvus 承载值检索与样例检索向量索引；Redis 保存热点缓存、任务热状态和流式事件通道；图表与导出文件则以文件路径方式登记到结果产物表中。其中，应用层和智能服务层通过 SQLAlchemy 统一管理元数据库等核心对象。

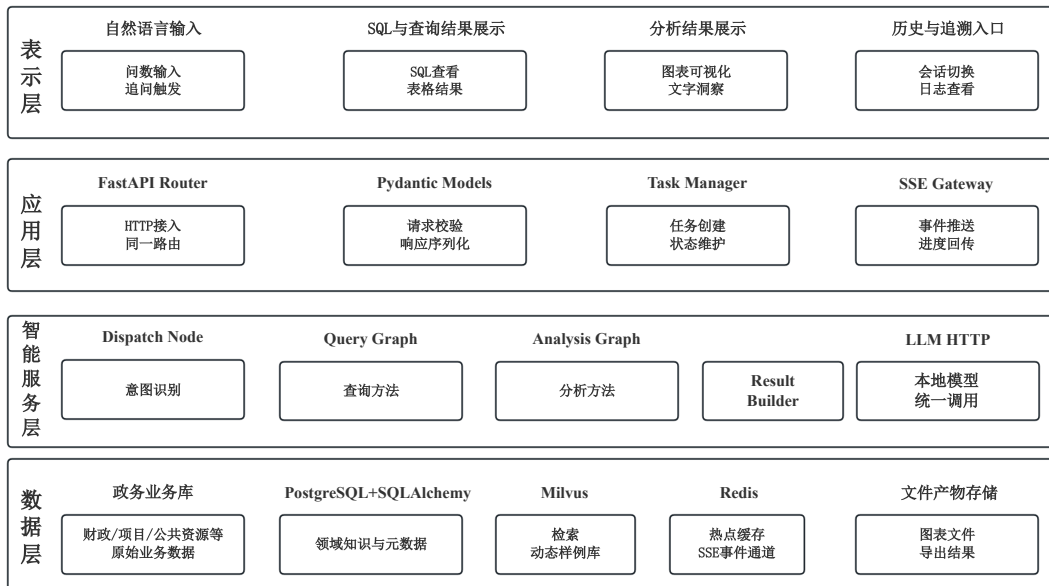


图 5-1 系统总体分层架构图

5.3 应用层与智能服务层实现

5.3.1 应用层接口设计

应用层采用 FastAPI 构建统一 HTTP 服务入口，以 Pydantic 完成请求校验与响应序列化，并围绕查询和分析两类核心业务对外提供标准化接口。

在接口组织上，系统采用同步查询、异步分析的调用策略。同步查询接口在请求上下文中直接执行查询子图并返回 `QueryResponse`，同时支持 SSE 流式模

式，将各节点进度实时推送给前端；异步分析接口采用“先建任务、后执行”的模式，创建任务后由后台执行器触发分析子图，任务完成后再通过结果接口返回 `AnalysisResultResponse`，其中核心结果对象为 `ResultPackage`。此外，系统提供统一的任务状态查询、结果获取和事件订阅接口，使查询与分析的结果回传共享同一交互范式。

在查询与分析的链路衔接上，分析请求并不会绕过查询服务直接生成图表或结论，而是先由分析规划节点生成 **AP-Schema**，再由查询投影节点提取指标、维度、筛选条件和统计粒度等查询约束，并由查询调用节点复用查询子图。查询子图复用第三章中的 **LA-Schema** 增强、异构候选生成、查询修复与候选判别流程，返回结果表及执行状态；分析子图再基于该结果完成分析算子执行、图表构建、文字洞察和结果封装，从而生成 **ResultPackage**。

在异常处理上，应用层通过全局异常拦截器将参数错误、权限异常、任务状态异常等规范化为统一响应结构，并为每个请求生成唯一追踪标识，与任务标识一同贯穿服务层和日志系统，为后续错误定位与结果回放提供依据。

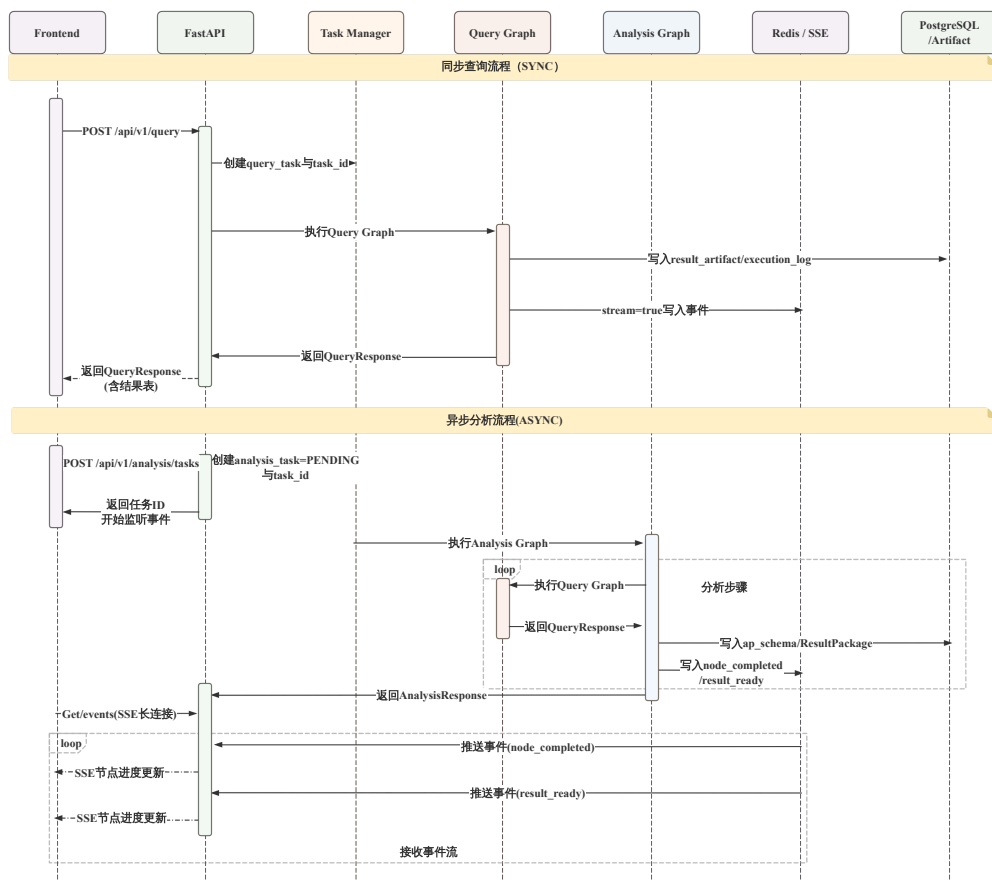


图 5-2 同步查询、异步分析与流式回传时序图

5.3.2 任务状态管理与异常回传

为直观说明同步查询、异步分析与流式回传三种实现方式之间的关系，图5-2给出了应用层和智能服务层的时序交互。

系统在任务层维护由 PENDING、RUNNING、SUCCESS 和 FAILED 组成的统一状态机。查询同步接口在用户视角上即时返回，但内部仍经历完整的“建任务—执行—写日志—回传结果”状态流转；分析异步接口则明确依次经历各状态阶段。所有状态转移均同步写入持久层并通过 SSE 事件推送给前端。

异常处理与结果回传机制如图5-3所示。

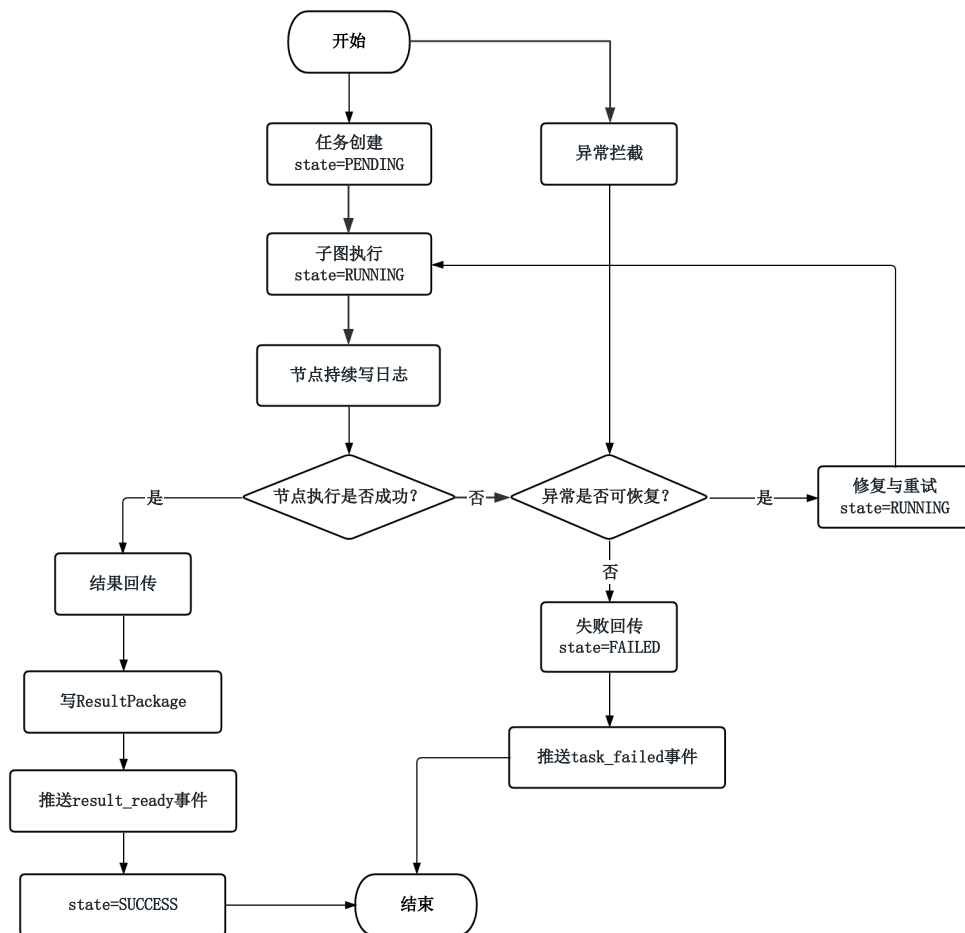


图 5-3 异常处理与结果回传实现图

在异常分类上，**应用层异常**由全局异常处理器拦截，涵盖参数校验失败、权限不足和非法状态访问等；**服务层异常**由子图节点内部捕获，涵盖 SQL 执行错误、分析算子异常和大模型调用超时等。对于可恢复错误，系统在节点内执行有限轮次的自动修复并记录重试日志；对于不可恢复错误，系统将任务标记为失败并向

前端推送失败事件。系统最终通过结果产物表和结构化日志保存关键中间对象和最终结果，使前端能够基于任务标识完成结果回放和链路追踪。

5.4 系统数据层存储

为使上述应用层接口和智能服务层编排稳定运行，还需要有针对性的存储机制来承接数据源接入、元数据抽取、任务状态保存和结果产物管理。故本节从业务数据源接入与元数据抽取、系统元数据库实现以及结果与缓存管理三个方面说明数据存储实现。

5.4.1 业务数据源接入与元数据抽取

在工程实现上，系统通过 JDBC/ODBC 等标准数据库连接方式建立到外部数据源的连接，并在 `data_source` 表中维护数据源标识、数据库类型、连接配置和同步策略等元信息。应用层中的数据源管理接口负责维护这些配置，而后台元数据同步任务则定期触发表结构扫描和索引更新。

此外系统还必须对外部数据源执行结构化元数据抽取。如图5-4所示，元数据抽取流程包含表结构抽取、字段属性抽取、主外键关系抽取以及样例值和枚举值抽取四个环节。

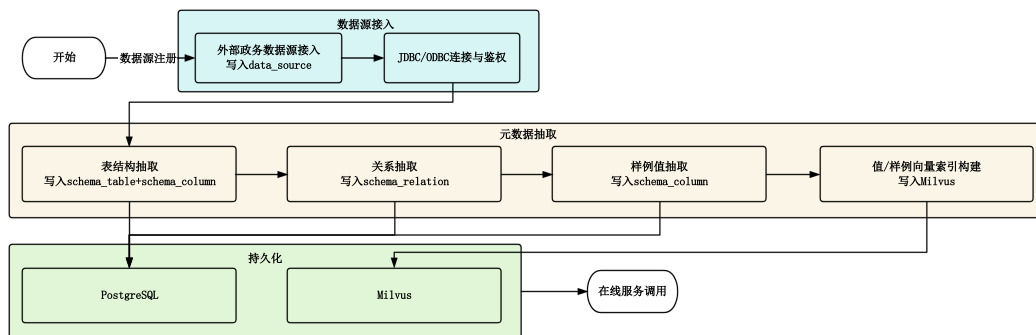


图 5-4 业务数据源接入与元数据抽取实现图

其中，**表结构抽取**负责获得库、表和字段的基础信息，包括表名、表注释、字段名、数据类型、主键标识和字段注释等；**关系抽取**负责显式发现数据库中的主外键关系，并将其转化为统一的 `schema_relation` 记录，以支撑后续多表 JOIN 推理；**样例值抽取**面向行政区划、项目状态、支出类别等典型枚举型或文本型字段，抽取少量样例值或完整值域，用于值匹配与语义提示；**向量索引构建**则将关键字段值与动态样例检索所需内容编码后写入 Milvus，为第三章中的值检索与样例检索提供在线支持。

经过上述过程后，系统在 PostgreSQL 中形成 data_source、schema_table、schema_column 和 schema_relation 等标准元数据表，在 Milvus 中形成面向值检索与样例检索的向量索引。在线查询时，schema_retriever 从 schema_* 表中读取与当前问题相关的结构元数据，knowledge_retriever 从领域知识库 $K = \{B, L, V\}$ 中读取术语映射、逻辑公式和值域约束，example_retriever 则基于动态样例库 F 及其向量索引完成相似样例召回。三类信息在查询子图中共同用于构建 LA-Schema。因此，元数据抽取模块实际上承担了从“外部数据库原始结构”到“可被查询方法使用的统一知识表示”之间的桥梁作用。

5.4.2 系统元数据库实现

系统自身维护一个独立的元数据库来持久化保存会话、任务、产物和日志等状态对象。应用层和智能服务层通过 SQLAlchemy 统一管理 query_task、analysis_task、execution_log 和 result_artifact 等核心表，本文实现了如表5-1所示的系统核心数据表，并在图5-5中给出了它们之间的主要关联关系。

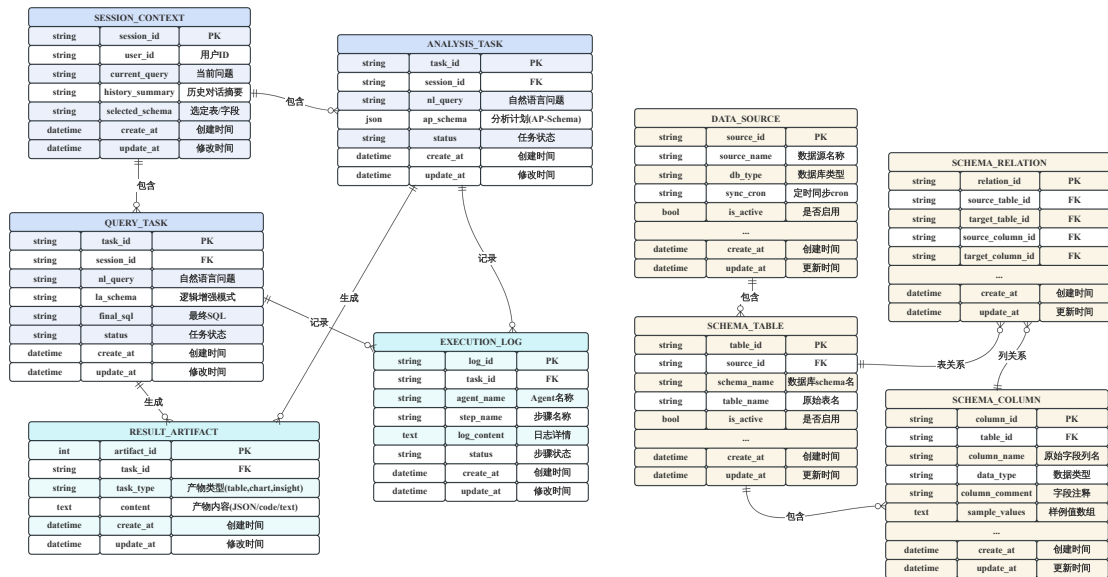


图 5-5 系统元数据库 ER 图

从结构关系看，源数据侧由 data_source、schema_table、schema_column 和 schema_relation 共同构成统一结构描述；会话侧由 session_context、query_task 和 analysis_task 共同构成交互过程的状态管理；产物侧则由 result_artifact 和 execution_log 承接任务级结果沉淀与运行留痕。由于查询任务和分析任务都可能生成多个结果文件和多条日志记录，因此系统将产物和日志分别实现为独立从表，并通过 task_type 与 task_id 完成关联。

表 5-1 系统元数据库核心表结构

表名	关键字段	主要说明
data_source	source_id source_name db_type	保存外部政务数据源配置和同步信息，是元数据抽取的起点。
schema_table	table_id source_id table_name	保存库级与表级结构信息，供模式链接和多库定位使用。
schema_column	column_id table_id column_name sample_values	保存字段类型、注释和样例值，是 LA-Schema 构建的直接输入。
schema_relation	relation_id source_table_id target_table_id	保存主外键关系或显式 JOIN 关系，为复杂查询生成提供连接先验。
session_context	session_id history_summary current_query	保存会话级上下文摘要、最近任务产物和用户反馈，是多轮交互的核心状态表。
query_task	task_id session_id nl_query final_sql	保存查询任务主记录，记录原始问题、最终 SQL 和执行状态。
analysis_task	task_id session_id nl_query ap_schema	保存分析任务主记录，其中 ap_schema 采用 JSON 对象存储。
result_artifact	artifact_id task_type task_id content	统一登记查询结果表、图表文件、文本洞察和导出文件等产物。
execution_log	log_id task_type task_id agent_name	按步骤记录执行状态、异常信息和关键信息快照，是问题定位与结果追溯的基础。

5.4.3 结果、日志与缓存管理

在持久化存储策略上，系统根据对象类型将其分别落到最合适的组件中。具体而言，PostgreSQL 负责保存系统元数据、任务主记录和领域知识库；Milvus 负责保存字段值向量和样例向量索引；Redis 负责热点缓存、任务热状态以及 SSE 事件通道；图表文件、导出文件和大体积结果文件则采用文件路径方式登记到 result_artifact 中。

在结果管理方面，系统将查询结果表、图表文件、洞察文本、导出文件统一抽象为结果产物。对于查询任务，重点保存 final_sql、结果表摘要和原始结果文件；对

于分析任务，重点保存 `ap_schema`、图表描述、分析表格和 `insights` 文本。若用户在已有查询结果基础上继续发起分析任务，系统仅在当前分析任务与已有查询结果在指标、维度、筛选条件和统计粒度上兼容时优先复用相应 `result_artifact`；若不兼容，则由分析服务重新触发查询子图完成取数。

在日志管理方面，`execution_log` 采用结构化写法，至少记录 `task_id`、节点名称、执行步骤、状态、日志内容和时间戳。与仅保存文本日志不同，结构化日志可以支持按任务、代理或节点维度检索，能够快速定位错误是发生在模式链接、候选判别、分析算子、图表生成还是结果封装环节。

在缓存管理方面，系统主要缓存两类内容。第一类是变化频率较低但访问频繁的元数据与知识对象，如表结构摘要、字段样例值和部分领域知识片段；第二类是高频相同请求对应的查询结果摘要、分析结果引用以及流式事件缓冲。由于政务问数场景并不追求毫秒级实时更新，因此在保证缓存失效策略合理的前提下，引入 Redis 可以有效降低重复请求对数据库和大模型服务的压力。

5.5 系统功能实现与效果展示

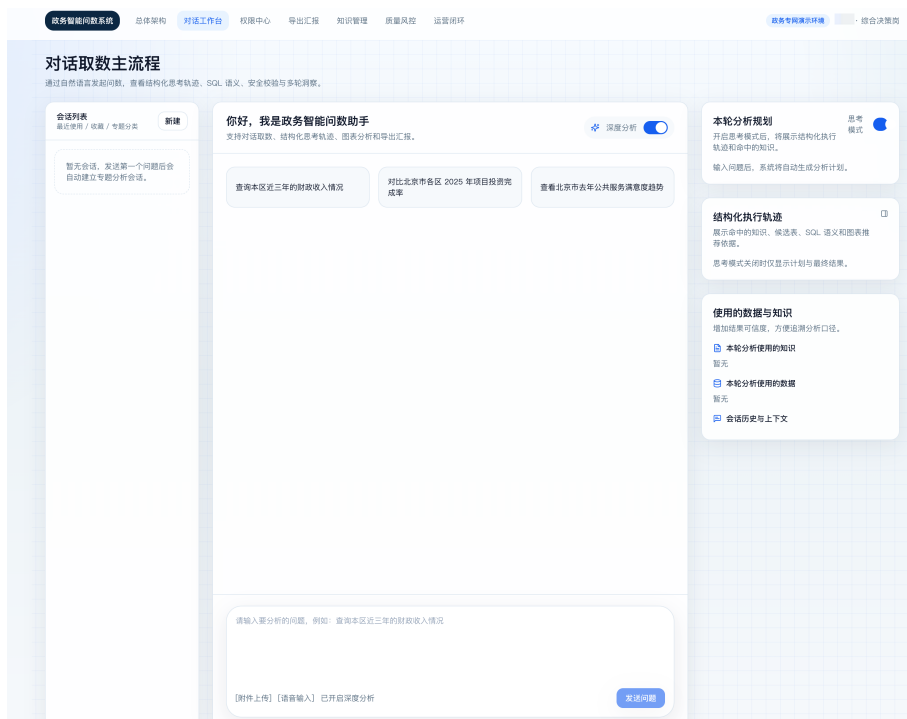


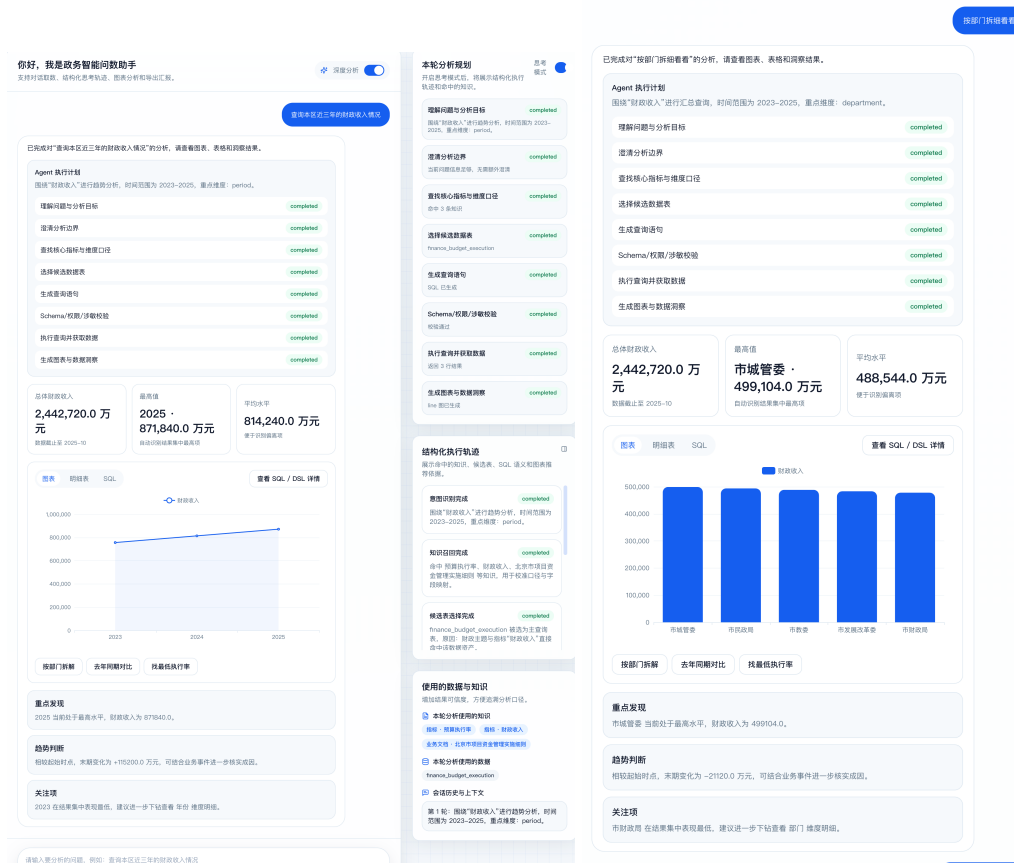
图 5-6 系统对话主页面

基于上述架构、应用层接口和智能服务层实现，本文系统为此展示对应自然语言输入、SQL 与结果查看、分析结果展示等内容。

以下页面概括展示已实现系统的核心界面结构。

自然语言输入对话主页面。图5-6是系统的主要交互入口，底部提供自然语言输入框，左侧提供历史会话列表，主区域展示用户与系统的交互内容。右侧包含相关细节的展示，包含有分析规划、执行轨迹、数据与知识信息等。用户可以直接输入查询类或分析类需求，也可以在已有会话中继续追问。

分析图表与文字洞察。如图5-7a。当任务属于分析类请求，或用户在查询结果基础上发起分析时，系统在该页面展示图表、分析表格和文字洞察。在输出过程中会展示分析规划（计划）和执行轨迹。同时支持用户针对已有的分析结果进行追问，系统会根据追问内容生成新的分析规划和执行轨迹。如图5-7b。



(a) 分析需求问答展示

(b) 分析需求追问

图 5-7 分析图表与文字洞察

SQL 与查询结果展示。如图5-8。当任务被判定为查询类请求或涉及到查询并成功执行后，系统在该页面展示查询计划与最终 SQL 语句和结果表（明细表）。结果表区域则为后续人工复核、导出或发起一键分析提供基础。

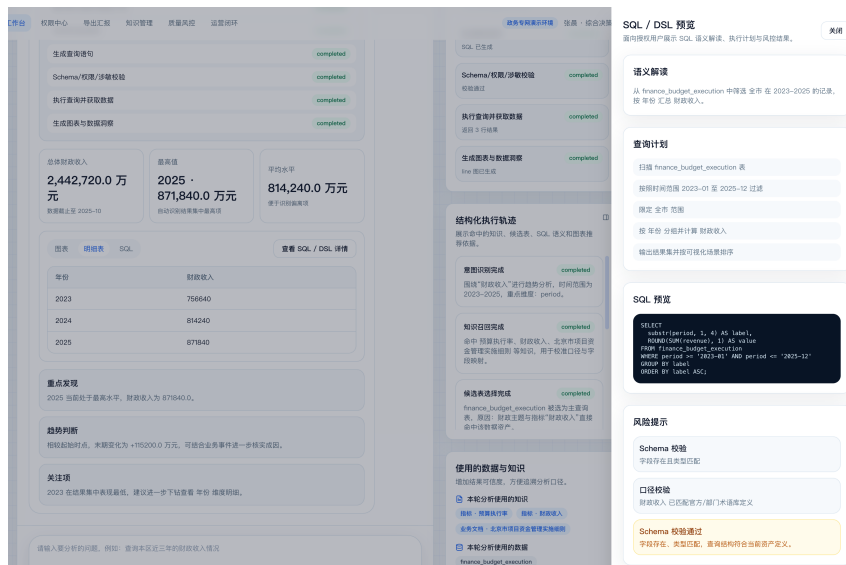


图 5-8 SQL 与查询结果展示

数据结果追溯。如图5-9。每次查询或者分析任务，都要在主页面的右侧展示使用的数据与知识，方便进行追溯分析。



(a) 执行轨迹

(b) 使用数据与知识

图 5-9 问答执行轨迹与数据追溯

系统功能页面并不是独立于方法之外的界面拼装，而是把第三章的查询能力、第四章的分析能力以及第五章的应用层接口、智能服务层编排和结果追溯机制，统一组织到一个已经实现的政务智能问数系统中。用户在界面层面看到的是一次自然的问数与分析过程，而在系统内部则对应着 FastAPI 接口接入、LangGraph 子图

编排、Redis 事件推送和结构化状态流转的工程实现。

5.6 本章小结

本章从工程实现角度说明了政务智能问数系统的落地方式。系统采用表示层、应用层、智能服务层和数据层四层架构，由应用层统一接入自然语言请求，并在智能服务层编排查询子图与分析子图，完成同步查询、异步分析以及任务状态与结果的统一管理。

第三章提供可靠取数能力，第四章提供规范分析能力，第五章则将二者组织为一个可交互、可追溯的统一系统：分析请求先形成 **AP-Schema** 并投影查询约束，再复用查询子图完成取数，最终生成图表、表格和文字洞察。由此，全文形成了从方法设计到系统实现的完整闭环。

结论与展望

6.1 主要研究结论

本文围绕政务场景中自然语言问数与数据分析的实际需求，面向“如何可靠查数、如何规范分析、如何系统落地”三个核心问题开展研究，形成了从方法创新到系统落地的完整技术链路。全文主要研究结论如下。

在方法层面，本文提出两项核心创新。第一，围绕政务术语歧义明显、统计口径隐含、复杂查询长尾分布突出等问题，本文提出了基于逻辑增强与异构协同的政务数据查询生成方法。该方法通过构建领域知识库、动态样例库与 LA-Schema，将业务术语映射、逻辑计算公式和值域约束显式融入查询生成过程，并结合逻辑映射生成器、渐进分治生成器、执行反馈修复与候选判别模型，提升了复杂政务场景下查询生成的正确性与稳定性。实验结果表明，本文方法在自建政务数据集 GovSQL 上取得了 91.47% 的执行准确率，较最强基线 XiYan-SQL 提升 7.64 个百分点；在 BIRD 与 Spider 公共数据集上分别达到 71.23% 和 89.25%，表明所提出方法在垂直场景和通用场景中均具有较强竞争力。

第二，围绕开放式分析需求约束隐含、查询与分析职责耦合以及结果表达需规范化等问题，本文提出了基于分析规划与查询增强的政务数据分析方法。该方法引入 AP-Schema 作为结构化分析规划模式，将自然语言分析需求显式映射为分析类型、指标、维度、筛选条件、粒度、查询层操作、分析层操作、可视化规格和摘要要求等字段；进一步通过查询增强的任务分解机制，将涉及统计口径、字段绑定和复杂关联的操作下沉至查询层完成，在可靠取数基础上由轻量分析算子链与可视化规则生成规范化结果。实验结果表明，在政务分析任务上，本文方法的流程成功率、任务完成率、结果正确率和图表恰当性分别达到 92.9%、85.7%、95.8% 和 4.6 分，显著优于代码生成（Code-Gen Agent）和工具迭代（ReAct Agent）等主流分析范式；在规划质量评估中，AP-Schema 的意图识别准确率、槽位填充完备率和约束满足率分别达到 94.6%、91.8% 和 91.1%。这说明“先查准、再分析”的协同机制能够有效提升政务数据分析的稳定性与规范性。

在系统实现层面，本文完成了从方法到系统的工程整合。

第三，在前述查询方法和分析方法基础上，本文设计并实现了一个面向政务场景的智能问数系统。系统采用表示层、应用层、智能服务层和数据层四层架构，基于 FastAPI、LangGraph、PostgreSQL、Milvus 和 Redis 等技术组件，支持同步查

询、异步分析、任务状态管理、执行日志留痕与结果追溯。系统实现表明，本文提出的查询方法与分析方法不仅能够在实验环境中取得有效结果，而且能够被组织为可交互、可复核、可追踪的统一系统，从而完成从自然语言需求理解、可靠取数到规范分析输出的闭环落地。

总体而言，本文构建了面向政务智能问数的完整研究框架，为大语言模型在复杂业务场景中的数据查询、分析与系统落地提供了可参考的方法路径。

6.2 研究不足

本文仍存在以下不足。

第一，数据与任务验证范围仍然有限。本文构建的 GovSQL 数据集及分析任务主要来源于同源政务业务场景，虽然能够较好反映财政预算、公共资源和政务项目等典型需求，但对于跨地区、跨部门、跨制度口径的复杂业务场景，其泛化能力仍有待进一步验证。

第二，领域知识库构建、动态样例整理和部分训练标注仍依赖人工参与。当前方法虽然通过结构化知识注入显著提升了垂直场景性能，但知识获取、校验与维护成本相对较高，面对业务规则频繁调整、数据结构持续变化的政务环境时，自动更新与自适应能力仍显不足。

第三，本文分析任务的覆盖范围仍以趋势、对比、排名和结构四类典型任务为主。对于更复杂的预测分析、诊断归因、多轮深度分析以及更严格的安全权限约束控制，本文尚未展开系统研究，相关能力距离真实复杂政务应用场景仍存在一定差距。

6.3 未来展望

第一，构建更大规模、多区域、多领域的政务查询与分析评测数据集，覆盖更多制度口径与数据库形态，进一步提升方法的跨场景泛化验证能力，并为后续模型比较与持续优化提供更充分的评测基础。

第二，研究更自动化的领域知识获取、知识更新与样例演化机制。可结合元数据同步、历史查询日志挖掘、人机协同校验与持续学习等方式，降低领域知识库与样例库的人工维护成本，增强系统对动态业务规则和模式变化的适应能力。

第三，拓展分析任务类型与系统能力边界。在方法层面，可进一步增强多轮交互分析、复杂洞察生成与多源异构数据协同分析能力；在系统层面，可进一步完善权限控制、部署治理与服务稳定性设计，推动大语言模型问数系统在真实政务场景中的安全应用与持续落地。

参考文献

- [1] 国务院. 国务院关于加强数字政府建设的指导意见 [R]. 北京: 国务院, 2022.
- [2] 中共中央, 国务院. 数字中国建设整体布局规划 [R]. 北京: 中共中央, 国务院, 2023.
- [3] 国家发展改革委, 国家数据局, 工业和信息化部. 国家数据基础设施建设指引 [R]. 北京: 国家发展改革委, 国家数据局, 工业和信息化部, 2025.
- [4] 孟天广. 数字治理全方位赋能数字化转型 [J]. 政策瞭望, 2021, (03): 33–35.
- [5] 陈兰杰, 黄海波. 从“政务信息”到“公共数据”: 我国政府数据治理的演进逻辑与发展趋势 [J]. 西华大学学报 (哲学社会科学版), 2026, 45(01): 84–93.
- [6] Wang L, He H, Pan Z, et al. An efficient system for large-scale collaborative tasks in government [J]. Human-Centric Intelligent Systems, 2024, 4: 406–416.
- [7] OpenAI. GPT-4 technical report [EB/OL]. arXiv, 2023, [2026-03-24]. arXiv:2303.08774, <https://arxiv.org/abs/2303.08774>.
- [8] Yang A, Li A, Yang B, et al. Qwen3 technical report [EB/OL]. arXiv, 2025, [2026-03-24]. arXiv:2505.09388, <https://arxiv.org/abs/2505.09388>.
- [9] DeepSeek-AI. DeepSeek-V3 technical report [EB/OL]. arXiv, 2024, [2026-03-24]. arXiv:2412.19437, <https://arxiv.org/abs/2412.19437>.
- [10] Zhao W X, Zhou K, Li J, et al. A survey of large language models [EB/OL]. arXiv, 2023, [2026-03-24]. arXiv:2303.18223, <https://arxiv.org/abs/2303.18223>.
- [11] Liu X, Shen S, Li B, et al. A survey of text-to-sql in the era of llms: Where are we, and where are we going? [J]. IEEE Transactions on Knowledge and Data Engineering, 2025.
- [12] Hong S, Lin Y, Liu B, et al. Data interpreter: an LLM agent for data science [C]. Findings of the Association for Computational Linguistics: ACL 2025, Vienna, Austria, 2025: 19796–19821.
- [13] Tang Z, Wang W, Zhou Z, et al. Llm/agent-as-data-analyst: A survey? [EB/OL]. arXiv, 2025, [2026-03-24]. arXiv:2509.23988, <https://arxiv.org/abs/2509.23988>.
- [14] 王昀, 胡珉, 塔娜, 等. 大语言模型及其在政务领域的应用 [J]. 清华大学学报 (自然科学版), 2024, 64(04): 649–658.
- [15] 沈宇, 黄卫东, 叶文武. 基于大模型 nl2sql 的交通系统运行智能问数助手技术研究 [J]. 信息通信技术, 2025, 19(03): 29–36.
- [16] 兰洪浩, 高长伟, 房秉毅, 等. 政务大模型在民生服务领域的创新应用与价值创造 [J]. 信息通信技术, 2025, 19(3): 9–13.

-
- [17] Yu T, Zhang R, Yang K, et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task [C]. Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, 2018: 3911–3921.
- [18] Li J, Hui B, Qu G, et al. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls [C]. Proceedings of the 37th International Conference on Neural Information Processing Systems, NIPS '23, Red Hook, NY, USA, 2023: 42330–42357.
- [19] 刘喜平, 舒晴, 何佳壕, 等. 基于自然语言的数据库查询生成研究综述 [J]. 软件学报, 2022, 33(11): 4107–4136.
- [20] Ma P, Ding R, Wang S, et al. InsightPilot: An LLM-empowered automated data exploration system [C]. Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, Singapore, 2023: 346–352.
- [21] Zhong V, Xiong C, Socher R. Seq2SQL: generating structured queries from natural language using reinforcement learning [EB/OL]. arXiv, 2017, [2026-03-24]. arXiv:1709.00103, <https://arxiv.org/abs/1709.00103>.
- [22] Yu T, Yasunaga M, Yang K, et al. SyntaxSQLNet: Syntax tree networks for complex and cross-domain text-to-SQL task [C]. Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, 2018: 1653–1663.
- [23] Wang B, Shin R, Liu X, et al. RAT-SQL: Relation-aware schema encoding and linking for text-to-SQL parsers [C]. Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, Online, 2020: 7567–7578.
- [24] Lin X V, Socher R, Xiong C. Bridging textual and tabular data for cross-domain text-to-SQL semantic parsing [C]. Findings of the Association for Computational Linguistics: EMNLP 2020, Online, 2020: 4870–4888.
- [25] Scholak T, Schucher N, Bahdanau D. PICARD: Parsing incrementally for constrained autoregressive decoding from language models [C]. Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, Online and Punta Cana, Dominican Republic, 2021: 9895–9901.
- [26] Li H, Zhang J, Li C, et al. Resdsql: decoupling schema linking and skeleton parsing for text-to-sql [C]. Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence and Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence and Thirteenth Symposium on Educational Advances in Artificial Intelligence, 2023: 13067–13075.

- [27] Gao D, Wang H, Li Y, et al. Text-to-sql empowered by large language models: A benchmark evaluation [J]. *Proc VLDB Endow*, 2024, 17(5): 1132–1145.
- [28] Pourreza M, Rafiei D. Din-sql: decomposed in-context learning of text-to-sql with self-correction [C]. *Proceedings of the 37th International Conference on Neural Information Processing Systems*, Red Hook, NY, USA, 2023: 36339–36348.
- [29] Pourreza M, Li H, Sun R, et al. Chase-sql: Multi-path reasoning and preference optimized candidate selection in text-to-sql [C]. *International Conference on Learning Representations (ICLR)*, 2025.
- [30] Gao Y, Wang Y, Li Y, et al. XiYan-SQL: a multi-generator ensemble framework for text-to-SQL [EB/OL]. *arXiv*, 2024, [2026-03-24]. *arXiv*:2411.08599, <https://arxiv.org/abs/2411.08599>.
- [31] Xie X, Xu G, Zhao L, et al. Opensearch-sql: Enhancing text-to-sql with dynamic few-shot and consistency alignment [J]. *Proc ACM Manag Data*, 2025, 3(3): 1–24.
- [32] Li B, Zhang J, Fan J, et al. Alpha-sql: Zero-shot text-to-sql using monte carlo tree search [C]. *International Conference on Machine Learning (ICML)*, 2025.
- [33] Li H, Wu S, Zhang X, et al. Omnisql: Synthesizing high-quality text-to-sql data at scale [J]. *Proc VLDB Endow*, 2025, 18(11): 4695–4709.
- [34] Talaei S, Pourreza M, Chang Y C, et al. Chess: Contextual harnessing for efficient sql synthesis [C]. *International Conference on Machine Learning (ICML)*, 2025.
- [35] Li H, Zhang J, Liu H, et al. Codes: Towards building open-source language models for text-to-sql [J]. *Proc ACM Manag Data*, 2024, 2(3): 1–28.
- [36] 申一凯. 基于大语言模型的 Text-to-SQL 方法研究与应用 [D]. 合肥: 中国科学技术大学, 2025: 1–80.
- [37] 李豎飞. 基于大语言模型的文本转 SQL 方法研究 [D]. 上海: 东华大学, 2025: 1–71.
- [38] 黄铠乘. 面向政务数据赋能的 SQL 查询自动生成技术研究 [D]. 佛山: 佛山大学, 2025: 1–96.
- [39] Chen M, Tworek J, Jun H, et al. Evaluating large language models trained on code [EB/OL]. *arXiv*, 2021, [2026-03-24]. *arXiv*:2107.03374, <https://arxiv.org/abs/2107.03374>.
- [40] Rozière B, Gehring J, Gloeckle F, et al. Code Llama: open foundation models for code [EB/OL]. *arXiv*, 2023, [2026-03-24]. *arXiv*:2308.12950, <https://arxiv.org/abs/2308.12950>.
- [41] Li R, Allal L B, Zi Y, et al. StarCoder: may the source be with you! [EB/OL]. *arXiv*, 2023, [2026-03-24]. *arXiv*:2305.06161, <https://arxiv.org/abs/2305.06161>.

- [42] Luo Z, Xu C, Zhao P, et al. Wizardcoder: Empowering code large language models with evol-instruct [C]. International Conference on Learning Representations (ICLR), 2024.
- [43] Dibia V. LIDA: A tool for automatic generation of grammar-agnostic visualizations and infographics using large language models [C]. Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations), Toronto, Canada, 2023: 113–126.
- [44] Maddigan P, Susnjak T. Chat2vis: Generating data visualizations via natural language using chatgpt, codex and gpt-3 large language models [J]. IEEE Access, 2023, 11: 45181–45193.
- [45] Qin B, Chen S, Zhu Y, et al. TaskWeaver: a code-first agent framework [C]. Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics: System Demonstrations, 2024: 262–273.
- [46] Zhang W, Shen Y, Lu W, et al. Data-copilot: Bridging billions of data and humans with autonomous workflow [C]. International Conference on Learning Representations (ICLR), 2024.
- [47] Huang A, Zha D, Shen Y, et al. TableGPT2: a large multimodal model with tabular data integration [EB/OL]. arXiv, 2024, [2026-03-24]. arXiv:2411.02059, <https://arxiv.org/abs/2411.02059>.
- [48] Tian Y, Cui W, Deng D, et al. Chartgpt: Leveraging llms to generate charts from abstract natural language [J]. IEEE Transactions on Visualization and Computer Graphics, 2025, 31(3): 1731–1745.
- [49] 孟庆国, 鞠京芮. 人工智能支撑的平台型政府: 技术框架与实践路径 [J]. 电子政务, 2021, (9): 37–46.
- [50] 赵大燕, 何华均, 李宇平, 等. 面向政务协同的访问控制模型 [J]. 计算机应用, 2025, 45(1): 162–169.
- [51] 王薇, 庞晓博. 政务大模型应用策略研究 [J]. 信息通信技术与政策, 2025, 51(11): 74–80.
- [52] 吴重庆, 杨思哲, 宁庭勇, 等. 上海政务垂类大模型应用探索 [J]. 信息通信技术与政策, 2025, 51(8): 78–83.
- [53] 肖庆. 面向政务垂直领域的大语言模型技术与应用研究 [D]. 成都: 电子科技大学, 2025: 1–113.
- [54] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need [C]. Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS’17, Red Hook, NY, USA, 2017: 6000–6010.
- [55] Radford A, Narasimhan K, Salimans T, et al. Improving language understanding by generative pre-training [R]. San Francisco: OpenAI, 2018.

- [56] Radford A, Wu J, Child R, et al. Language models are unsupervised multitask learners [R]. San Francisco: OpenAI, 2019.
- [57] Brown T B, Mann B, Ryder N, et al. Language models are few-shot learners [C]. Proceedings of the 34th International Conference on Neural Information Processing Systems, NIPS '20, Red Hook, NY, USA, 2020: 1877–1901.
- [58] Devlin J, Chang M W, Lee K, et al. BERT: Pre-training of deep bidirectional transformers for language understanding [C]. Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), Minneapolis, Minnesota, 2019: 4171–4186.
- [59] Raffel C, Shazeer N, Roberts A, et al. Exploring the limits of transfer learning with a unified text-to-text transformer [J]. J Mach Learn Res, 2020, 21(1): 5485–5551.
- [60] Wei J, Bosma M, Zhao V, et al. Finetuned language models are zero-shot learners [C]. International Conference on Learning Representations (ICLR), 2022.
- [61] Chung H W, Hou L, Longpre S, et al. Scaling instruction-finetuned language models [J]. J Mach Learn Res, 2024, 25(1): 3381–3433.
- [62] Ouyang L, Wu J, Jiang X, et al. Training language models to follow instructions with human feedback [C]. Proceedings of the 36th International Conference on Neural Information Processing Systems, 35, Red Hook, NY, USA, 2022: 27730–27744.
- [63] Dong Q, Li L, Dai D, et al. A survey on in-context learning [C]. Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, Miami, Florida, USA, 2024: 1107–1128.
- [64] Wei J, Wang X, Schuurmans D, et al. Chain-of-thought prompting elicits reasoning in large language models [C]. Advances in neural information processing systems, 35, 2022: 24824–24837.
- [65] Kojima T, Gu S S, Reid M, et al. Large language models are zero-shot reasoners [C]. Advances in neural information processing systems, 35, 2022: 22199–22213.
- [66] Wang X, Wei J, Schuurmans D, et al. Self-consistency improves chain of thought reasoning in language models [C]. International Conference on Learning Representations (ICLR), 2023.
- [67] Yao S, Yu D, Zhao J, et al. Tree of thoughts: deliberate problem solving with large language models [C]. Advances in neural information processing systems, 36, 2024: 11809–11822.

- [68] Tang X, Zong Y, Phang J, et al. Struc-bench: Are large language models good at generating complex structured tabular data? [C]. Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 2: Short Papers), Mexico City, Mexico, 2024: 12–34.
- [69] Qin Y, Hu S, Lin Y, et al. Tool learning with foundation models [J]. ACM Comput Surv, 2024, 57(4): 1–40.
- [70] Schick T, Dwivedi-Yu J, Dessì R, et al. Toolformer: language models can teach themselves to use tools [C]. Advances in neural information processing systems, 36, 2024: 68539–68551.
- [71] Patil S G, Zhang T, Wang X, et al. Gorilla: large language model connected with massive apis [C]. Proceedings of the 38th International Conference on Neural Information Processing Systems, Red Hook, NY, USA, 2024: 126544–126565.
- [72] Yao S, Zhao J, Yu D, et al. ReAct: synergizing reasoning and acting in language models [C]. International Conference on Learning Representations (ICLR), 2023.
- [73] Shi L, Tang Z, Zhang N, et al. A survey on employing large language models for text-to-sql tasks [J]. ACM Comput Surv, 2025, 58(2): 1–37.
- [74] 周祥生, 肖萍, 韩普. 大模型推理任务调度系统设计与优化研究 [J]. 信息通信技术, 2025, 19(3): 37–44.
- [75] Wu Q, Bansal G, Zhang J, et al. Autogen: Enabling next-gen llm applications via multi-agent conversation [C]. International Conference on Learning Representations (ICLR), 2024.
- [76] Hong S, Zhuge M, Chen J, et al. Metagpt: Meta programming for a multi-agent collaborative framework [C]. International Conference on Learning Representations (ICLR), 2024.
- [77] Li G, Hammoud H A A K, Itani H, et al. CAMEL: communicative agents for “mind” exploration of large language model society [C]. Advances in Neural Information Processing Systems, 36, 2024: 51991–52008.
- [78] 郭先会, 张梦姣, 马军. 基于大语言模型的智能体构建综述 [J]. 通信技术, 2024, 57(9): 873–879.
- [79] 李轶夫. 基于生成式大模型的 ai 智能体技术及应用发展趋势 [J]. 信息通信技术, 2025, 19(3): 14–21.
- [80] Chen J, Xiao S, Zhang P, et al. M3-embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation [C]. Findings of the Association for Computational Linguistics: ACL 2024, Bangkok, Thailand, 2024: 2318–2335.

- [81] Malkov Y A, Yashunin D A. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs [J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2020, 42(4): 824–836.
- [82] Shinn N, Cassano F, Gopinath A, et al. Reflexion: language agents with verbal reinforcement learning [C]. Advances in neural information processing systems, 36, 2024: 8634–8652.
- [83] Hu E J, Shen Y, Wallis P, et al. LoRA: Low-rank adaptation of large language models [C]. International Conference on Learning Representations(ICLR), 2022.
- [84] Loshchilov I, Hutter F. Decoupled weight decay regularization [C]. International Conference on Learning Representations(ICLR), 2019.

攻读硕士学位期间的成果

学术 论文	序号	作者排名	发表刊物名称	论文发表年份	发表刊物类别 ※
	1	1	计算机应用文摘	2026	普通期刊
	2	3	Proceedings of the ACM Web Conference 2024	2024	国际会议论文 CCFA
专 利	序号	排名	专利类型	专利授权年份	
获 奖 情 况	序号	排名	授奖单位	授奖级别	授奖年份
其 他	序号	排名	类别（标准或项目）	级别	年份

- 注：1. “创新成果列表”中，不得出现学术论文题目、专利名称、专利号等可定位到个人及导师身份的信息；
2. “发表刊物类别”一栏请注明SCI、EI、CSCD、CSSCI、核心期刊（以北京大学出版社《中文核心期刊要目总览》为准）、会议论文（国际会议或国内会议）等；
3. “获奖情况”一栏请注明获得国家级、省部级（含国家一级学会、协会）科技奖级别；
4. “其他”请填写已正式出版或报批的国际、国家及行业标准；或者参与的国家级重大、重点工程项目课题。